studocu

Algodat SS 2019 Skript

Algorithmen und Datenstrukturen (Friedrich-Schiller-Universität Jena)



Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Algorithmen und Datenstrukturen

Modul	Algorithmen und Datenstrukturen
Dozent	Dr. Jana Grajetzki
Semester	Sommersemester 2019

Dieses Dokument (Stand 16. Mai 2022) stellt eine studentische Mitschrift währendder Veranstaltungen zu oben aufgeführtem Modul dar. Obwohl es nach bestemWissen und Gewissen angefertigt worden ist, können Fehler enthalten sein,welche Sie gerne als Issue im Git-Projekt vermerken können.

This document is available on studocu

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Inhaltsverzeichnis

1 Einführung 5
1.1 Maschinenmodell und Effizienzbetrachtungen 6
RAM-Modell 6
Laufzeitanalyse 7
1.2 Entwurf und Analyse eines einfachen Algorithmus – Sortieren durch Einfügen 8
Entwurf 8
Korrektheit 8
Analyse 9
1.3 Asymptotische Notationen 11
Definitionen 11
Eigenschaften 12
Langsame und schnelle Algorithmen 14
1.4 Schnittpunkte von horizontalen und vertikalen Liniensegmenten 14
Orthogonale Liniensegmente 14
Verallgemeinerter Fall 16
2 Rekursionsgleichungen 17
2.1 Iterationsmethode 17
2.2 Rekursionsbaum-Methode 18
2.3 Substitutionsmethode 19
2.4 Variablentransformation 19
2.5 Mastertheorem 20
3 Sortieren und Selektieren 21
3.1 Mergesort 21
3.2 Heapsort 23
Die Datenstruktur Heap 23
Build Heap 27
Heapsort 29
3.3 Quicksort 30

Downloaded by Maja Müller (blackcatgirl@freenet.de)

3.4 Selection . . . 33
3.5 Eine untere Schranke für das Sortieren . . . 35
3.6 Sortieren in linearer Zeit . . . 37
Countingsort . . . 37
Radixsort . . . 39
4 Suchbäume . . . 39
4.1 Grundlegende Operationen . . . 40
Traversierung von binären Bäumen . . . 40
Suchen . . . 43
Minimum und Maximum . . . 43
Vorgänger und Nachfolger . . . 44
Einfügen . . . 45
Löschen . . . 46
4.2 Rot-Schwarz-Bäume . . . 48
Definition und grundlegende Eigenschaften . . . 48
Rotationen . . . 51
Einfügen . . . 52
Löschen . . . 58
5 Hashing . . . 63
5.1 Hashing mit Verkettung . . . 64
5.2 Hashfunktionen . . . 65
Verschiedene Hashfunktionen . . . 65
Universelles Hashing . . . 66
5.3 Offene Adressierung . . . 66
Lineares Sondieren . . . 68
Quadratisches Sondieren . . . 68
Doppeltes Hashing . . . 69
6 Binomialheaps . . . 70
6.1 Binomialbaum . . . 70
6.2 Binomialheap . . . 71
Darstellung von Binomialheaps . . . 72

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Operationen auf Binomialheaps 72
7 Amortisierte Kostenanalyse 75
7.1 Einführende Beispiele 75
Stapeloperationen 75
Binärzähler 76
7.2 Potentialmethode 77
Stapeloperationen 78
Binärzähler 78
8 Fibonacci-Heaps 78
8.1 Basis-Operationen auf Fibonacci-Heaps 80
Heap erstellen und Minimum finden 80
Einfügen 80
Vereinigen zweier F-Heaps 81
Entfernen des Minimums 82
8.2 Knoten löschen und Schlüssel verringern 87
8.3 Wie groß kann $D(n)$ werden? 90
9 Union-Find-Strukturen 92
9.1 Disjoint-Set-Forest 92
10 Graphenalgorithmen 96
10.1 Breitensuche 98
10.2 Kürzeste Wege 103
10.3 Tiefensuche 103
10.4 Topologisches Sortieren 108
10.5 Minimale Spannbäume 111
Algorithmus von Kruskal 115
Algorithmus von Prim 116
11 Dynamische Programmierung 120
11.1 Längste gemeinsame Teilfolge 121
Phase 1 121
Phase 2 122
Phase 3 123

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Phase 4 124
11.2 Summe maximaler Teilfolgen 124
11.3 Anzahl der aussagenlogischen Klammerungen 126
11.4 Dame auf dem Schachbrett 128
12 Material aus den Übungsterminen 130
12.1 Algorithmik 130
12.2 Landau-Notation 131
12.3 Rekursionsgleichungen 133
12.4 Suchen und Sortieren 135
12.5 Amortisierte Analyse 143
12.6 Graphenalgorithmen 153

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

1 Einführung

Was ist ein Algorithmus?

- wohldefinierte Rechenvorschrift mit einer Größe oder einer Menge als Eingabe und einer Größe oder einer Menge als Ausgabe (anders: Folge von Rechenschritten, die eine Eingabe in eine Ausgabe umwandelt)
- spezifische Rechenvorschrift (löst ein genau festgelegtes Rechenproblem)

Für die Aussage „Es gibt keinen Algorithmus, der das Problem ... löst" benötigt man eine präzise formale Fassung (Stichworte Churchsche These und Turingmaschine).

Wozu werden Algorithmen gebraucht?

- Sortieralgorithmen
- Wegeplanung im Straßennetz (Graphenalgorithmen)
- geometrische Algorithmen (z.B. konvexe Hülle, nächste Nachbarn, Voronoi-Diagramm)
- Internet: große Mengen von Informationen schnell zugänglich machen (Algorithmen, die große Datenmengen handhaben können)
- Kryptologie (Verschlüsselung)
- numerische Algorithmen
- zahlentheoretische Algorithmen

Geschichte der Algorithmen:

- Konzepte der Informatik sind älter als die Geschichte der Computer.
- Diese Konzepte sind auch ohne existierende Computer gültig.
- Sie erreichen ihre praktische Relevanz erst durch den Siegeszug der Computer.
- 300 v. Chr. Euklids Algorithmus zur Bestimmung des größten gemeinsamen Teilers
- 800 n. Chr. „Liber Algorithmi" – Algorithmensammlung für Kaufleute und Testamentsvollstrecker
- 1574 Adam Ries Rechenbuch – mathematische Algorithmen
- 1644 erste Logarithmentafeln werden algorithmisch berechnet
- 1703 Leibniz führt binäre Zahlsysteme ein
- 1815 Ada Lovelace geboren, entwirft Programme für erste Rechenmaschinen

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- 1822 Charles Babbage entwickelt „Difference Engine“, später „Analytical Engine“
- 1931 Gödels Unvollständigkeitssatz
- 1936 Chruchsche These vereinheitlicht die Welt der Sprachen zur Notation von Algorithmen
- danach: Mit Realisierung der ersten Computer erfolgte der Ausbau der Algorithmentheorie

Womit beschäftigen wir uns?

- Entwurf von Algorithmen
- Korrektheitsbeweise
- Effizienzbetrachtung

1.1 Maschinenmodell und Effizienzbetrachtungen

RAM-Modell

Die Effizienz eines Algorithmus wird bestimmt durch Speicherplatz und Laufzeit. Beides ist für eine konkrete Implementierung auf konkretem Rechner messbar. Das lässt sich aber nicht auf andere Implementierungen und andere Rechner übertragen.

Dafür gibt es zwei Auswege:

1. Messen von Laufzeit und Speicherplatz auf idealisiertem Modellrechner: RAM (Random Access Machine)
   - verfügt über einige Register (abzählbar unendlich viele einzeln adressierbare Speicherzellen)
   - Register und Speicherzellen können je eine unbeschränkt große ganze (oder gar reelle Zahl) aufnehmen.
   - Befehle:
     - Transportbefehle zum Laden von und Speichern in direkt oder indirekt adressierten Speicherzellen (Befehle zum Löschen, Speichern, Kopieren)
     - arithmetische Befehle zur Verknüpfung zweier Registerinhalte mit den üblichen für ganze und reelle Zahlen definierten Operationen (Addition, Subtraktion, Division, Modulo, unteres Ganzes, oberes Ganzes, Potenzieren)
   - Speicherplatz: Anzahl der benötigten RAM-Zellen

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Laufzeit: Anzahl der ausgeführten RAM-Befehle
- Dass die Menge der Speicherzellen abzählbar unendlich ist, ist eine idealisierte Annahme und im Einzelfall zu bedenken.

2. Ermittlung einiger charakteristischer Parameter, z.B. Laufzeit eines Sortieralgorithmus wird durch die Zahl der Vergleiche und Vertauschungen bestimmt.

Laufzeitanalyse

Die Eingabegröße eines Algorithmus hängt vom Problem ab und muss konkret angegeben werden.

Beispiel 1

Sortieren – Zahl der Elemente

Graphenalgorithmen – Knoten- und Kantenzahl

Die Laufzeit ist die Zahl der ausgeführten Grundoperationen.

- pro Zeile im Pseudocode konstante Zeit
- Zeile $i \rightarrow$ Laufzeit c_i

Unterschied Problem zu konkreter Eingabe (Problem Sortieren der Eingabe $7, 3, 8, 4, 5$). Die Laufzeit kann für konkrete Eingaben gemessen werden.

Definition 1

Sei E die Menge aller zulässigen Probleminstanzen. $t(e)$ mit $t : E \rightarrow \mathbb{N}$ gibt die Zahl der ausgeführten Grundoperationen bei Eingabe e an.

Wir wollen Laufzeiten für alle Eingaben der Größe n betrachten. Diese können natürlich verschieden sein.

Der Durchschnitt scheint sinnvoll, aber worüber wird er gebildet? Dass jede Instanz gleichtwahrscheinlich ist, ist in der Praxis nicht gerechtfertigt.

Argumente für den worst case:

- obere Schranke für die Laufzeit einer beliebigen Eingabe
- schlechte Fälle treten oft häufig auf (z.B. erfolglose Suche)

Daher:

$$T(n) = \max_{\substack{e \in E, \\ |e|=n}} \{t(e)\}$$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

1.2 Entwurf und Analyse eines einfachen Algorithmus – Sortieren durch Einfügen

Entwurf

Algorithmus 1 INSERTIONSORT:

- Input: Eine Folge von Zahlen $\langle a_1, ..., a_n \rangle$.
- Output: Eine Permutation $\langle a'_1, ..., a'_n \rangle$ der Eingabefolge, sodass $a'_1 \leq a'_2 \leq ... \leq a'_n$ gilt.

algorithm INSERTIONSORT(A)

```
1    require Feld A[1...n] enthält a_1, ..., a_n
2    for j = 2 to n do
3        s = A[j]
4        ▷ Füge A[j] in die sortierte Liste A[1...j − 1] ein
5        i = j − 1
6        while i > 0 and A[i] > s do
7            A[i + 1] = A[i]
8            i = i − 1
9        end while
10       A[i + 1] = s
11   end for
end algorithm
```

Korrektheit

Definition 2 Ein Algorithmus heißt *korrekt*, wenn er für jede gültige Eingabeinstanz mit der korrekten Ausgabe stoppt.

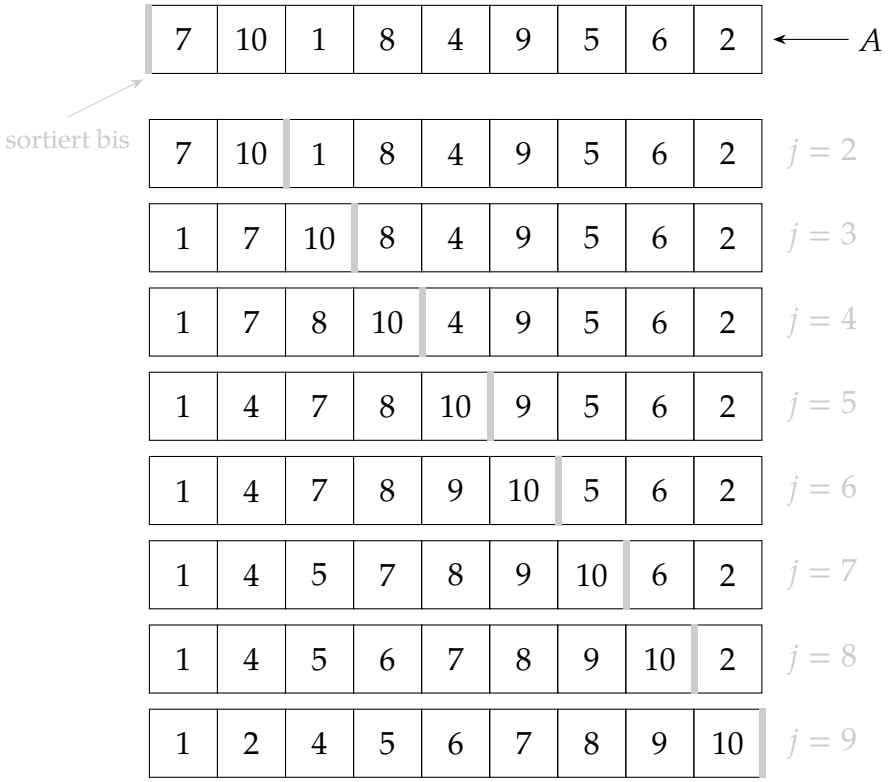
Bemerkung 1 Nicht korrekt bedeutet, es gibt mindestens eine Eingabe, bei der der Algorithmus nicht stoppt oder ein falsches Ergebnis liefert.

Sei j der Index der aktuell einzusortierenden Zahl. Wir wählen eine Schleifeninvariante: Zu Beginn jeder Iteration der For-Schleife (Zeilen 2–2) besteht $A[1...j-1]$ aus $a_1, ..., a_{j-1}$ in geordneter Reihenfolge.

Beweis:

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 1 Beispiel zur Funktionsweise von InsertionSort auf der Eingabe $A = [7, 10, 1, 8, 4, 9, 5, 6, 2]$



1. Initialisierung: Die Behauptung ist vor der ersten Iteration wahr. Anfangs ist $j = 2$. $A[1...j-1]$ ist $A[1]$. Dieses enthält a_1 und ist sortiert.
2. Fortsetzung: Ist die Behauptung vor einer Iteration wahr, so ist sie auch vor der nächsten Iteration wahr. Was passiert: $A[j-1], A[j-2]$, ... werden jeweils um eine Position nach rechts geschoben und $A[j]$ an den freien Platz eingefügt. Daraus folgt, dass $A[1...j]$ $a_1, ..., a_{j-1}, a_j$ enthält. $A[1...j]$ ist sortiert, denn die Reihenfolge von $A[1], ..., A[j-1]$ wurde nicht geändert und in Zeile 10 gilt $A[i] \leq s < A[i+1]$.
3. Terminierung: Wann bricht die Schleife ab? Nach dem Durchlauf $j = n$ bricht der Algorithmus ab. Das Teilfeld nach dem Schleifendurchlauf ist $A[1...n]$ und enthält nach Invariante $a_1, ..., a_n$ in sortierter Reihenfolge.

Der Algorithmus ist korrekt.

Analyse

Wir notieren für jede Zeile Kosten für die einmalige Ausführung und die Zahl der Durchläufe.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Sei t_j die Anzahl der Tests in Zeile 6 für $2 \leq j \leq n$.

Zeile	Kosten	Zahl der Durchläufe
2	c_1	$n-1$
3	c_2	$n-1$
4	c_3	0
5	c_4	$n-1$
6	c_5	$\sum_{j=2}^{n} t_j$
7	c_6	$\sum_{j_2}^{n} (t_j - 1)$
8	c_7	$\sum_{j_2}^{n} (t_j - 1)$
10	c_8	$n-1$

$T(A)$ bezeichnet die Laufzeit des Algorithmus für Eingabefeld A und ist die Summe der Produkte aus Kosten und Zahl der Durchläufe.

$$\begin{aligned} T(A) &= (c_1 + c_2 + c_4 + c_8)(n-1) + c_5 \sum_{j=2}^{n} t_j + (c_6 + c_7) \sum_{j=2}^{n} (t_j - 1) \\ &= (c_1 + c_2 + c_4 + c_8 + c_6 + c_7)(n-1) + (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j \\ &= a \cdot n + b + c \sum_{j=2}^{n} t_j \end{aligned}$$

Dabei sind a, b, c konstant. t_j ist abhänig von der konkreten Eingabe.

Im Best Case ist A schon sortiert und für $j = 2, \ldots, n$ gilt in Zeile 6 immer $A[i] \leq s$, wenn $i =$ Startwert $- 1$. Das heißt $t_j = 1$ für $j = 2, \ldots, n$. Damit ist $T(A) = an + b + c \sum_{j=2}^{1} t_j = a' \cdot n + b'$.

Im Worst Case ist A absteigend sortiert. Damit ist $t_j = j$ und die Laufzeit ist

$$\begin{aligned} T(A) &= a \cdot n + b + c \sum_{j=2}^{n} j \\ &= a \cdot n + b + c \left(\frac{n(n+1)}{2} - 1 \right) \\ &= a' \cdot n + b' + c' \cdot n^2 \\ T(n) &= \max_{\substack{e \in E, \\ |e| = n}} \{T(e)\} = c' \cdot n^2 + a' \cdot n + b \end{aligned}$$

This document is available on studocu

Downloaded by Maja Müller (blackcatgirl@freenet.de)

1.3 Asymptotische Notationen

Definitionen

Beispiel 2

Sortieralgorithmus 1 benötigt $T_1(n) = c_1 \cdot n^2$ Zeit. Der Sortieralgorithmus 2 benötigt $T_2(n) = c_2 \cdot n \cdot \log(n)$ Zeit. Dabei ist $c_1 < c_2$. Konkretes Beispiel:

	Rechner A	**Rechner B**
Anweisungen pro Sekunde	10^{10}	10^7
Algorithmus	1	2
Laufzeit	$2 \cdot n^2$ (Maschinencode, guter Programmierer)	$50 \cdot n \cdot \log(n)$ (höhere Sprache, ineffizienter Compiler)
n	10^7	10^7
Berechnung	$\frac{2 \cdot (10^7)^2 \text{ Anw.}}{10^{10} \text{ Anw. pro Sek.}} \approx 5{,}5 \text{ h}$	$\frac{50 \cdot 10^7 \cdot \log_2(10^7) \text{ Anw.}}{10^7 \text{ Anw. pro Sek.}} \approx 14 \text{ min}$

Aus dem Beispiel sieht man:

- Konstante Faktoren sind weniger wichtig als die Abhängigkeit von der Eingabegröße.
- Für kleine Eingabegrößen ist Algorithmus 1 gewöhnlich schneller.
- Egal, wie klein c_1 und wie groß c_2 ist, gibt es immer einen Übergangspunkt (bei der Eingabegröße), an dem Algorithmus 2 schneller wird.
- Für den Vergleich von Algorithmen nutzen wir die asymptotischen Notationen.

Definition 3

Seien $f, g : \mathbb{N} \to \mathbb{R}$.

$$O(g) = \{f(n) \mid \exists c > 0, n_0 > 0 : \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)\}$$
$$\Omega(g) = \{f(n) \mid \exists c > 0, n_0 > 0 : \forall n \geq n_0 : 0 \leq c \cdot g(n) \leq f(n)\}$$
$$\Theta(g) = \{f(n) \mid \exists c_1, c_2 > 0, n_0 > 0 : \forall n \geq n_0 : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\}$$
$$= O(g) \cap \Omega(g)$$

Definition 4

Seien $f, g : \mathbb{N} \to \mathbb{R}$.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$o(g) = \{f(n) \mid \forall c > 0 : \exists n_0 > 0 : \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)\}$$
$$\omega(g) = \{f(n) \mid \forall c > 0 : \exists n_0 > 0 : \forall n \geq n_0 : 0 \leq c \cdot g(n) \leq f(n)\}$$

Beispiel 3 $f(n) = n^3 + 777n^2 + 900$. Es wird behauptet $f(n) \in \Omega(n^3)$. Gesucht sind $c > 0$ und $n_0 \in \mathbb{N}$, sodass für alle $n \geq n_0$ gilt $f(n) \geq c \cdot n^3$. $f(n) = n^3 + 777n^2 + 900 \geq n^3$. Damit folgt für $c = 1$ und $n_0 = 1$ die Behauptung.

Ferner wird behauptet, $f(n) \in O(n^3)$, also $f(n) \leq c \cdot n^3$. Gesucht sind $c > 0$ und $n_0 \in \mathbb{N}$. Abschätzung: $f(n) \leq n^3 + 777n^3 + 900n^3 = 1678n^3$. Damit folgt die Behauptung für $c = 1678$ und $n_0 = 1$.

Aus beiden Behauptungen folgt, dass $f(n) = n^3 + 777n^2 + 900 \in \Theta(n^3)$.

Eigenschaften

- Transitivität: Alle fünf Relationen zur asymptotischen Laufzeit sind transitiv. Wenn für $f \in O(g)$ und $g \in O(h)$ gilt, dann ist auch $f \in O(h)$.
- Reflexivität: O, Ω, Θ sind reflexiv. Es gilt $f \in O(f), f \in \Omega(f), f \in \Theta(f)$.
- Symmetrie: Nur Θ ist symmetrisch. $f \in \Theta(g) \Leftrightarrow g \in \Theta(f)$.
- Schiefsymmetrie: $f \in O(g) \Leftrightarrow g \in \Omega(f)$.
- Θ ist eine Äquivalenzrelation auf der Menge der schließlich positiven Funktionen $(\exists n_0 \geq 0 : \forall n \geq n_0 : f(n) > 0)$.

Bemerkung 2 Es gibt unvergleichbare Funktionen, z.B. $f(n) = n$ und $g(n) = n^{1+\cos(\pi \cdot n)}$.

Definition 5 Seien $f_1, f_2, \ldots : \mathbb{N} \to \mathbb{R}$.

$$\max\{f_1\}(n) = f_1(n)$$
$$\max\{f_1, \ldots, f_k\}(n) = \max\{\max\{f_1, \ldots, f_{k-1}\}(n), f_k(n)\}$$
$$f_1 + f_2 + \ldots + f_k(n) = f_1(n) + f_2(n) + \ldots + f_k(n)$$

Satz 1 Seien $f_1, f_2, \ldots : \mathbb{N} \to \mathbb{R}$.

$$O(f_1 + f_2 + \ldots + f_k) = O(\max\{f_1, \ldots, f_k\})$$
$$\Omega(f_1 + f_2 + \ldots + f_k) = \Omega(\max\{f_1, \ldots, f_k\})$$

Weiterhin gilt

$$f_1, f_2, \ldots, f_k \in \Theta(\max\{f_1, f_2, \ldots, f_k\})$$

Beispiel 4 $f_1(n) = n^2, f_2(n) = 10n$. Es gilt $\max\{f_1, f_2\} \in \Theta(n^2)$. Außerdem

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$\max\{f_1, f_2\}(n) = \begin{cases} 10n & n \leq 9 \\ n^2 & n > 9 \end{cases} .$$

Beispiel 5 Für untereinanderstehende Funktionen gilt $f(n) \in o(g)$. Dabei ist $0 < \alpha < \beta$ und $0 < a < b$ sowie $1 < A < B$.

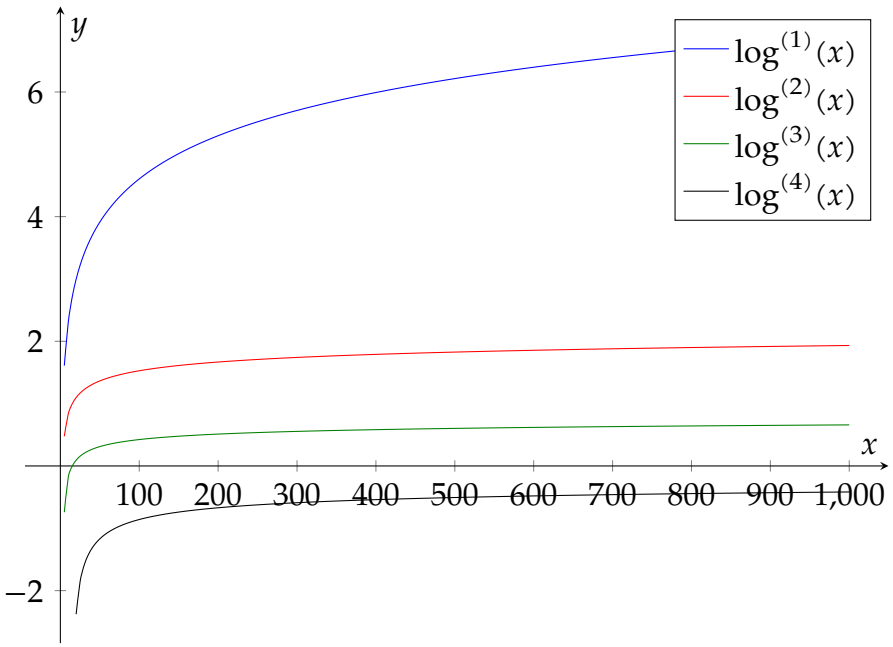
- $\alpha(n)$[1]
- $\log^*(n)$
- $\log(\log(n))$
- $(\log(n))^\alpha$
- $(\log(n))^\beta$
- n^a
- n^b
- A^n
- B^n

Bemerkung 3 Definition der Funktion $\log^*(n)$.

$$\log^*(n) = \min\{i \mid \log^{(i)}(n) \leq 1\}$$
$$\log^{(i)}(n) = \underbrace{\log \log \ldots \log}_{i \text{ Mal}}(n)$$

Verschiedene Grade von $\log^{(i)}(n)$ sind in Abbildung 2 dargestellt.

Abbildung 2 Darstellung verschiedener $\log^{(i)}(n)$



[1] α bezeichnet die Inverse der Ackermannfunktion.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 6 Wir nutzen den Logarithmus dualis. $\log^*(1) = 0, \log^*(2) = 1, \log^*(4) = 2, \log^*(16) = 3, \log^*(2^{16}) = 4$.

Langsame und schnelle Algorithmen

Rechner, der 10^6 Operationen pro Sekunde ausführen kann.

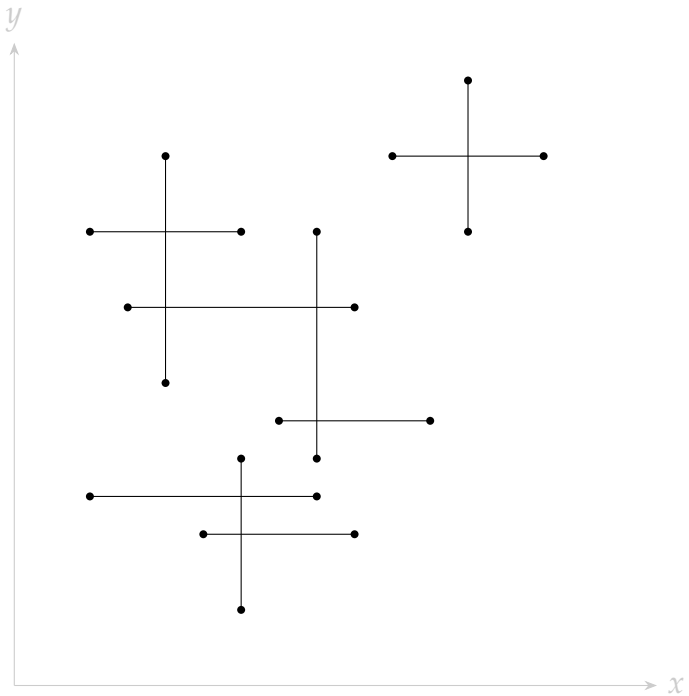
Klasse	**1 sec**	**1 min**	**1 h**	**1 d**	**1 y**
n	10^6	$6 \cdot 10^7$	$3,6 \cdot 10^9$	$8,64 \cdot 10^{10}$	$3,13 \cdot 10^{13}$
$n \cdot \log(n)$	62764	$2,8 \cdot 10^6$	$1,33 \cdot 10^8$	$2,75 \cdot 10^9$	$7.97 \cdot 10^{11}$
n^2	1000	7745	60000	293939	$5,6 \cdot 10^8$
2^n	20	25	31	36	44

1.4 Schnittpunkte von horizontalen und vertikalen Liniensegmenten

Orthogonale Liniensegmente

- Input: Menge H von horizontalen und Menge V von vertikalen Liniensegmenten
- Output: Menge aller Paare, die sich schneiden

Abbildung 3 Liniensegmente im Raum für eine Plane-Sweep-Lösung

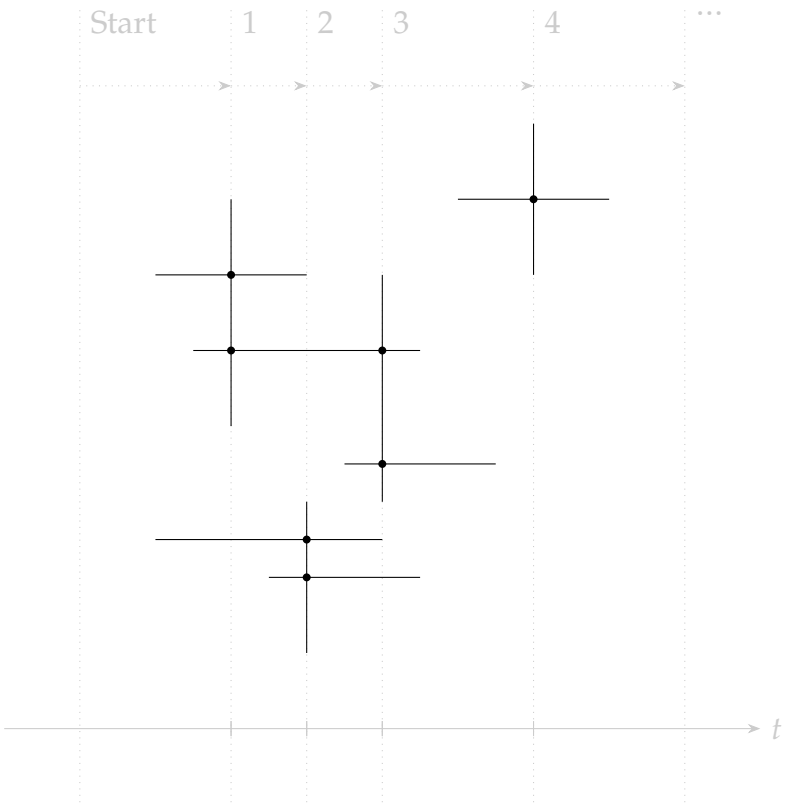


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Lösung mittels Plane Sweep:

- Eine Gerade gleitet von links nach rechts über die Ebene.
- Sie verwandelt eine räumliche Dimension (Abbildung 3) in eine zeitliche (Abbildung 4).
- Im Computer muss aber diskret gearbeitet werden. Deshalb werden Ereignispunkte gewählt.
- Im Beispiel sind unsere Ereignispunkte die x-Koordinaten aller Segmentschnitte.

Abbildung 4 Segmentschnitte im Beispiel (Abbildung 3), welche die Ereignispunkte der gestrichelten Sweepline zum Zeitpunkt t bestimmen



Wir bezeichnen die die zukünftigen Ereignisse verwaltende Ereignisstruktur mit ES. In unserem Beispiel nutzen wir eine sortierte Liste der Ereignispunkte.

Ferner ist SSS die *Sweep-Status-Struktur*. Sie verwaltet alle Segmente, die zum aktuellen Zeitpunkt von der Sweepline geschnitten werden – ihre y-Koordinate.

Sie ist eine dynamische Struktur mit den erforderlichen Operationen Insert, Delete und Search. Wir setzen $S = \{(x_1, (x_1, x_2, y)) \mid (x_1, x_2, y) \in$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

$H\} \cup \{(x_2, (x_1, x_2, y)) \mid (x_1, x_2, y) \in H\} \cup \{(x, (x, y_1, y_2)) \mid (x, y_1, y_2) \in V\}$.

Algorithmus 2 OrthoSegmentSchnitt

1. Sortiere S (nach 1. Koordinate) $\rightarrow$ ES
2. $Y = \emptyset$
   - durchlaufe ES
   - Aktion auf ES falls der neue Endpunkt
     - ⋆ linker Endpunkt eines horizontalen Segments $\rightarrow$ Segment in SSS aufnehmen ($Y = Y \cup (y, (x_1, x_2, y))$)
     - ⋆ rechter Endpunkt eines horizontalen Segments $\rightarrow$ Segment aus SSS löschen ($Y = Y \setminus (y, (x_1, x_2, y))$)
     - ⋆ vertikales Segment $(x, y_1, y_2) \rightarrow$ alle Segmente in der SSS ausgeben, deren y-Koordinate zwischen y_1 und y_2 liegt. $A = \{(w_1, w_2) \in Y \mid w_1 \in [y_1, y_2]\}$. $\Pi_2(A)$ sind die Segmente, die von (x, y_1, y_2) geschnitten werden.
       Out: Menge aller Paare in $(x_1, y_1, y_2) \times \Pi_2(A)$

Laufzeit:

- $O(n \cdot \log(n))$ für das Sortieren der Ereignispunkte
- Die restliche Laufzeit ist abängig davon, wie Y verwaltet wird.
- Insert, Delete, Search sind nötig, möglichst alles in $\log(n) \rightarrow$ dies würde z.B. für einen balancierten Suchbaum funktionieren
- Sei k die Anzahl der Schnittpunkte. $T(n,k) = \sum_{i=1}^{2n} (\log(n) + k_i)$, wobei k_i die Anzahl der Schnittpunkte beim i-tem Halt ist. Damit ist $T(n,k) = 2n \log(n) + \sum k_i \in O(n \log(n) + k)$.
- Der Algorithmus ist output-sensitiv (im besten Fall $\log(n)$, aber restlich vom Output abhängig). Für die Insertions und Deletions braucht er insgesamt $n \cdot \log(n)$ Zeit. Für die Halte mit Suche entsprechend $\log(n) + x$, wobei x die Anzahl der Schnittpunkte und im ungünstigsten Fall n^2 ist.

Verallgemeinerter Fall

Problem: In dieser Konstellation verändern die Schnittpunkte die Datenstruktur.

Definition 6 Ein *abstrakter Datentyp* ist eine Menge von Operationen. Eine *Datenstruktur* ist eine konkrete Methode, Daten abzuspeichern und den Zugriff auf die Daten und deren Modifikation zu organisieren.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 7 Ein Beispiel für einen abstrakten Datentyp ist ein DICTIONARY mit {INSERT, DELETE, SEARCH}. Eine konkrete Datenstruktur ist z.B. ein *binärer Suchbaum*, eine *verkettete Liste* etc.

2 Rekursionsgleichungen

- Viele Algorithmen enthalten rekursive Aufrufe.
- Die Laufzeit solcher Algorithmen kann durch Rekursionsgleichungen beschrieben werden.
- Eine Rekursionsgleichung ist eine Gleichung oder Ungleichung, die eine Funktion durch ihre eigenen Funktionswerte für kleinere Argumente beschreibt.
- Ziel: Bestimmung asymptotischer Schranken (O, Ω, Θ) der Lösungen solcher Gleichungen

Technische Details:

- Vernachlässigung $\lfloor\,\rfloor$ und $\lceil\,\rceil$
- $T(n)$ ist konstant für hinreichend kleine n. Verändern wir $T(1)$ beeinflusst es die Lösung nur um einen konstanten Faktor.

2.1 Iterationsmethode

$$
\begin{aligned}
T(n) &= 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n \\
&= 2 \cdot \left(2 \cdot T\left(\frac{n}{4}\right) + c \cdot \frac{n}{2}\right) + c \cdot n \\
&= 4 \cdot T\left(\frac{n}{4}\right) + 2c \cdot n \\
&= 4 \cdot \left(2 \cdot T\left(\frac{n}{8}\right) + c \cdot \frac{n}{4}\right) + 2c \cdot n \\
&= 8 \cdot T\left(\frac{n}{8}\right) + 3c \cdot n \\
&\vdots \\
&= 2^i \cdot T\left(\frac{n}{2^i}\right) + i \cdot c \cdot n
\end{aligned}
$$

Gesucht ist das kleinste i mit $\frac{n}{2^i} \leq 1$, also $n \leq 2^i$ bzw. $\log(n) \leq i$ oder $i = \lceil \log(n) \rceil$. Damit ist $T(n) = 2^i \cdot T\left(\frac{n}{2^i}\right) + i \cdot c \cdot n \in \Theta(n \log(n))$. Dies ist für obere und für untere Schranken zu nutzen, je nachdem, in welche Richung man abschätzt.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

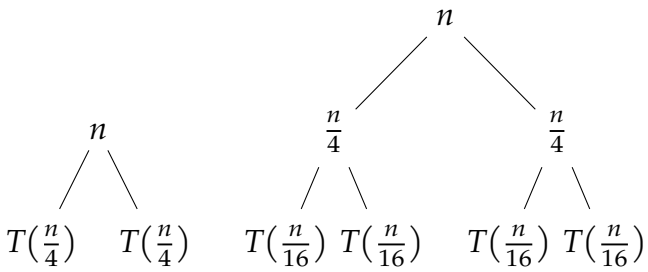
2.2 Rekursionsbaum-Methode

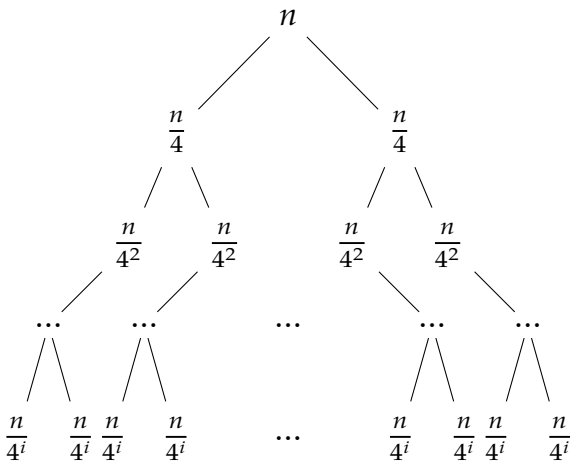
- In jedem Knoten stehen Kosten eines Teilproblems, das irgendwo als Rekursionsaufruf vorkommt.
- Die Gesamtkosten bilden sich aus der Summe der Einträge aller Knoten.

Beispiel 8

$T(n) = 2 \cdot T(\frac{n}{4}) + n$

$T(\frac{n}{4}) = 2 \cdot T(\frac{n}{16}) + \frac{n}{4}$





Abbruch: gesucht ist das kleinste i mit $\frac{n}{4^i} \leq 1 \Leftrightarrow n \leq 4^i \Leftrightarrow \log_4(n) \leq i$, also $i = \lceil \log_4(n) \rceil$.

$$T(n) = \sum_{i=0}^{\lceil \log_4(n) \rceil} 2^i \cdot \frac{n}{4^i} = n \cdot \underbrace{\sum_{i=0}^{\lceil \log_4(n) \rceil} \frac{1}{2^i}}_{\leq 2} \in \Theta(n)$$

Beispiel 9

Die Rekursionsbaum-Methode funktioniert z.B. gut für:

- $T(n) = T(\frac{n}{3}) + T(\frac{n}{5}) + n$
- $T(n) = T(\sqrt{n}) + 1$
- $T(n) = 3 \cdot T(\frac{n}{4}) + n$

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

2.3 Substitutionsmethode

1. Raten einer Lösung (kann schwierig sein)
2. Beweisen durch Induktion

Beispiel 10 $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n$ mit $T(1) = 0$. Wir vermuten $T(n) \in O(n \log(n))$, das heißt $T(n) \leq d \cdot n \log(n)$ für ein geeignetes d.

- Voraussetzung: $T\left(\frac{n}{2}\right) \leq d \cdot \frac{n}{2} \log \frac{n}{2}$
- Behauptung: $T(n) \leq d \cdot n \log(n)$

$$\begin{aligned} T(n) &= 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n \\ &\leq 2 \cdot \left(d \cdot \frac{n}{2} \log \frac{n}{2}\right) + c \cdot n \\ &= d \cdot n \log \frac{n}{2} + c \cdot n \\ &= d \cdot n(\log n - 1) + c \cdot n \\ &= d \cdot n \log n + (c - d) \cdot n \\ &\leq d \cdot n \log n \qquad \text{für } d \geq c \end{aligned}$$

Daraus folgt $T(n) \in O(n \log n)$.

Ein Beispiel für eine schlechte Vermutung wäre $T(n) \geq d \cdot n$. Die Vermutung kann nicht bewiesen werden: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n \leq 2 \cdot d\frac{n}{2} + c \cdot n = d \cdot n + c \cdot n \not\leq d \cdot n$, sie ist auch nicht richtig.

Beispiel 11 $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 1$. Vermutung: $T(n) \leq d \cdot n$.

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 1 \leq 2 \cdot d \cdot \frac{n}{2} + 1 = d \cdot n + 1$$

Die Vermutung konnte nicht bewiesen werden, aber $T(n) \in O(n)$ ist richtig. Mit der Vermutung $T(n) \leq d \cdot n - 1$ funktioniert.

Bemerkung 4

- Diese Methode ist sehr leistungsstark, erfordert aber Kreativität.
- Sie funktioniert für obere und untere Schranken.
- auch für Gleichungen wie z.B. $T(n) = 2 \cdot T\left(\frac{n}{2} + 5\right) + n$ oder $P(n) = \max\{\sqrt{n} \cdot P(\sqrt{n}), c \cdot n\}$

2.4 Variablentransformation

$T(n) = 2 \cdot T(\sqrt{n}) + \log n$.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Wir setzen $m = \log(n)$. Diese Substitution führt zu $T(2^m) = 2 \cdot T\left(2^{\frac{m}{2}}\right) + m$. Wir benennen um und erhalten $S(m) = T(2^m)$, was zu $S(m) = 2 \cdot S\left(\frac{m}{2}\right) + m$ führt, damit $S(m) \in \Theta(m \log m)$.

Wegen $S(m) = T(2^m) = T(n)$ und $m = \log(n)$ folgt $T(n) \in \Theta(\log(n) \cdot \log\log(n))$

2.5 Mastertheorem

Satz 2

Seien $a \geq 1, b > 1$ Konstanten, $f : \mathbb{N} \to \mathbb{R}$ eine Funktion und $T : \mathbb{N} \to \mathbb{R}$ eine durch folgende Rekursionsgleichung definierte Funktion: $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$. In diesem Fall besitzt $T(n)$ folgende asymptotische Schranken:

1. Fall $f \in O\left(n^{\log_b(a)-\epsilon}\right)$ für ein $\epsilon > 0$ gilt

$$T(n) \in \Theta\left(n^{\log_b(a)}\right)$$

2. Fall $f \in \Theta(n^{\log_b(a)})$ gilt

$$T(n) \in \Theta\left(n^{\log_b(a)} \log(n)\right)$$

3. Fall $f \in \Omega\left(n^{\log_b(a)+\epsilon}\right)$ für ein $\epsilon > 0$ und $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ für ein $c < 1$ und genügend große n gilt:

$$T(n) \in \Theta(f(n))$$

Bemerkung 5

- Das Wachstum von f wird mit dem Wachstum von $n^{\log_b(a)}$ verglichen.
- Die schneller wachsende der beiden Funktionen bestimmt das asymptotische Wachstum von $T(n)$.
- zum 1. Fall: $f(n)$ wächst nicht nur asymptotisch langsamer als $n^{\log_b(a)}$, sondern als $n^{\log_b(a)-\epsilon} = \frac{n^{\log_b(a)}}{n^\epsilon}$, d.h. polynomial langsamer.

Beispiel 12

$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + n^2$. $a = 8$ und $b = 2$. $\log_2(8) = 3$. Wir wissen $n^2 \in O(n^{3-\epsilon})$ für $\epsilon = \frac{1}{2}$, also nach 1. Fall des Mastertheorem $T(n) \in \Theta(n^3)$.

Beispiel 13

$T(n) = 8 \cdot T\left(\frac{n}{4}\right) + n \cdot \sqrt{n}$ mit $\log_4(8) = \frac{3}{2}$. $n \cdot \sqrt{n} \in \Theta(n^{\frac{3}{2}})$, also nach 2. Fall des Mastertheorems $T(n) \in \Theta(n^{\frac{3}{2}} \log(n))$.

Beispiel 14

$T(n) = 4 \cdot T\left(\frac{n}{3}\right) + n^2 \log(n)$. Wir schätzen ab $1 < \log_3(4) < 2$. $n^2 \log(n) \in \Omega(n^{\log_3(4)+\epsilon})$ für $\epsilon = 2 - \log_3(4) > 0$. Regularitätsbedingung: $4 \cdot$

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$\left(\frac{n}{3}\right)^2 \log \frac{n}{3} = 4 \cdot \frac{n^2}{9}(\log n - \log 3) \leq 4 \cdot \frac{n^2}{9} \log n = \underbrace{\frac{4}{9}}_{c} \cdot \underbrace{n^2 \log n}_{f(n)} \leq c \cdot n^2 \log n$$

für $c = \frac{4}{9}$.

Bemerkung 6 Es gibt Rekursionsgleichungen, die der geforderten Form $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ genügen, aber nicht mittels Mastertheorem lösbar sind.

Beispiel: $T(n) = 3 \cdot T\left(\frac{n}{3}\right) + n \log n$. $\log_3(3) = 1$. $n \log n \in \Omega(n)$, aber $n \log n \notin \Omega(n^{1+\epsilon})$ für alle $\epsilon > 0$, also ist das Mastertheorem nicht anwendbar.

3 Sortieren und Selektieren

- liegt oft im Wesen der Anwendung
- ist häufige Unterroutine
- es gibt viele verschiedene Sortieralgorithmen
- es können viele Techniken erlernt werden
- wir können eine nicht-triviale untere Schranke beweisen

Problemstellung:

- es werden nicht nur Zahlen sortiert, sondern ganze Datensätze
- jeder Datensatz hat einen Schlüssel, nach dem sortiert werden soll
- Input: Folge $\langle a_1, \ldots, a_n \rangle$ von Elementen einer linear geordneten Menge
- Output: Eine Permutation $\langle a'_1, \ldots, a'_n \rangle$ der Eingabefolge, für die $a'_1 \leq a'_2 \leq \ldots \leq a'_n$

3.1 Mergesort

MERGESORT ist ein Beispiel für Teile-und-Herrsche-Algorithmen.

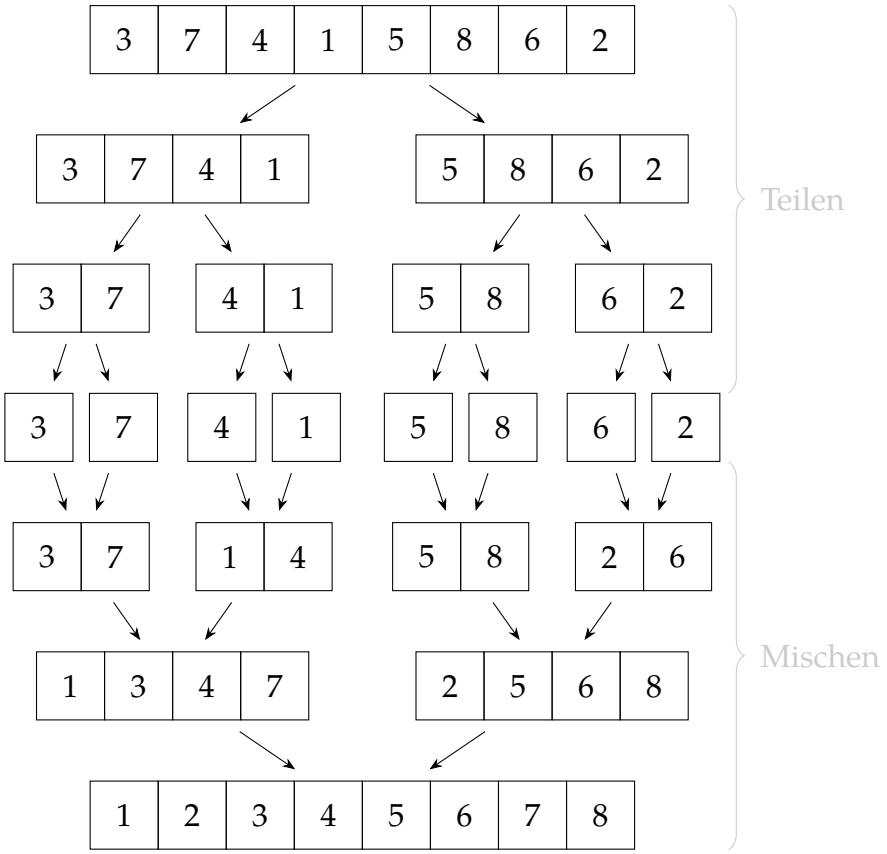
1. Zerlege das Problem in Teilprobleme
2. Löse die Teilprobleme rekursiv und wenn sie hinreichend klein sind auf direktem Wege
3. Setze die Lösung der Teilprobleme zur Lösung des Gesamtproblems zusammen

MERGESORT:

- Teile $\langle a_1, \ldots, a_n \rangle$ in zwei Teilfolgen der Länge $\frac{n}{2}$
- Sortiere beide Teilfolgen rekursiv durch Mergesort
- Mische die beiden sortierten Folgen

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 5 Beispiel zur Funktionsweise von MERGESORT am Beispiel $A = [3,7,4,1,5,8,6,2]$



Algorithmus 3 **algorithm** MERGE(A, p, q, r)

```
1   n_1 = q − p + 1                          ▷ Länge des Teilfeldes A[p...q]
2   n_2 = r − q                              ▷ Länge des Teilfeldes A[q + 1...r]
3   Erzeuge die Felder L[1...n_1 + 1] und R[1...n_2 + 1]
4   ▷ Kopieren von A[p...q] in L[1...n_1]
5   for i = 1 to n_1 do
6       L[i] = A[p + i − 1]
7   end for
8   for j = 1 to n_2 do
9       R[j] = A[q + j]
10  end for
11  ▷ Wächter am Ende von L und R einfügen
12  L[n_1 + 1] = ∞
13  R[n_2 + 1] = ∞
14  i = 1
15  j = 1
16  for k = p to r do
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
17              if L[i] ≤ R[j] then
18                  A[k] = L[i]
19                  i = i + 1
20              else
21                  A[k] = R[j]
22                  j = j + 1
23              end if
24          end for
        end algorithm
```

Laufzeit:

- Zeilen 1–3 und 12–13 in konstanter Zeit
- Zeilen 5–10: $\Theta(n_1 + n_2) = \Theta(n)$
- Zeile 16: n Iterationen dieser For-Schleife, jede Iteration konstante Zeit
- insgesamt: $\Theta(n)$

Algorithmus 4

```
    algorithm MERGESORT(A, p, r)
1       if p < r then
2           q = ⌊(p+r)/2⌋
3           MERGESORT(A, p, q)
4           MERGESORT(A, q + 1, r)
5           MERGE(A, p, q, r)
6       end if
    end algorithm
```

Laufzeit: $T(n) = 2 \cdot T(\frac{n}{2}) + c \cdot n$. Mit dem Mastertheorem $a = b = 2$. $\log_b(a) = \log_2(2) = 1$. $c \cdot n \in \Theta(n^1)$, damit 2. Fall des Mastertheorems, also $T(n) \in \Theta(n \log(n))$.

3.2 Heapsort

HEAPSORT ist ein In-Place-Sortierverfahren. In-Place bedeutet, dass zu jedem Zeitpunkt werden nur konstant viele Werte außerhalb des Feldes zwischengespeichert.

Die Datenstruktur Heap

Die binäre Datenstruktur *Heap* eignet sich als Implementierung für den Abstrakten Datentyp (ADT) PRIORITY QUEUE (PQ, mit {INSERT, EXTRACT

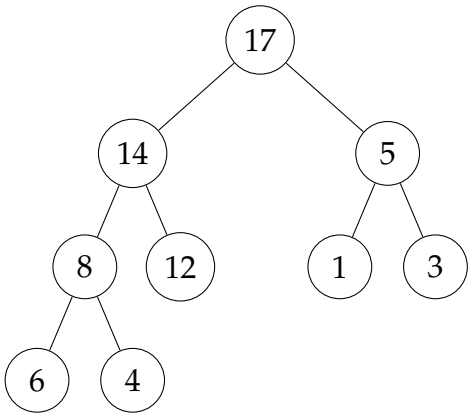
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Max}). Der Heap ist ein fast vollständiger binärer Baum, der als Feld dargestellt wird.

Abbildung 6 Visualisierung der Datenstruktur Heap

17	14	5	8	12	1	3	6	4

Speicherung als Feld



Darstellung als Baum

Das abgebildete Feld ist ein Objekt mit zwei Attributen:

- länge$[A]$ – Anzahl der Elemente
- heap_größe$[A]$ – Länge des Teils von A, der den Heap repräsentiert.
- heap_größe $\leq$ länge (Elemente hinter heap_größe gehören nicht mehr zum Heap)

Heapeigenschaft:

- $A[\text{Vater}(i)] \geq A[i]$
- $\text{Vater}(i) = \lfloor \frac{i}{2} \rfloor$
- $\text{left}(i) = 2i$ $(A[i] \geq A[2i]$ für $1 \leq i \leq \lfloor \frac{n}{2} \rfloor)$
- $\text{right}(i) = 2i + 1$ $(A[i] \geq A[2i+1]$ für $1 \leq i \leq \lfloor \frac{n}{2} \rfloor)$

Die Höhe eines Knotens ist die Anzahl der Kanten auf dem längsten absteigenden Weg zu einem Blatt. Die Höhe des Heaps ist die Höhe der Wurzel.

Grundoperationen für einen Max-Heap: MaxHeapInsert, MaxHeap-ExtractMax.

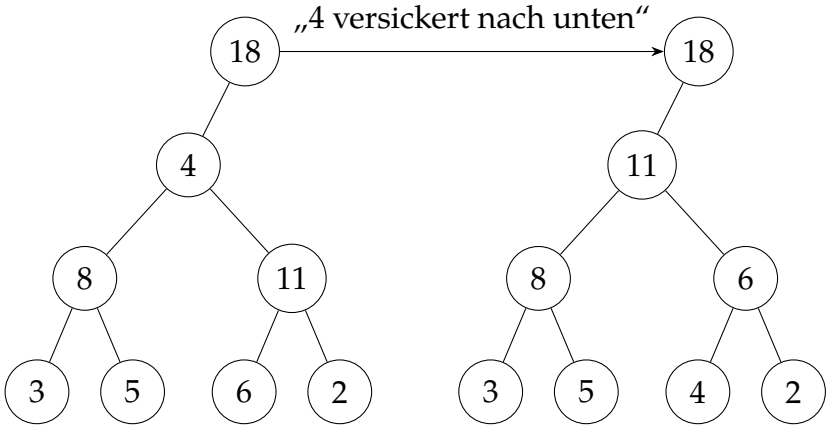
Algorithmus 5 MaxHeapify (Hilfsoperation)

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Input: A, i
- Output: A, sodass ab Index i Heapeigenschaft erfüllt
- Voraussetzung: Teilbäume $\text{left}(i), \text{right}(i)$ sind Heaps, i selbst aber nicht
- Schlüssel an Index i wird versickert bis Heapeigenschaft erfüllt

algorithm MaxHeapify(A, i)
1 $l = \text{left}(i)$
2 $r = \text{right}(i)$
3 **if** $l \leq$ heap_größe$[A]$ **and** $A[l] > A[i]$ **then**
4 $max = l$
5 **else**
6 $max = i$
7 **end if**
8 **if** $r \leq$ heap_größe$[A]$ **and** $A[r] > A[max]$ **then**
9 $max = r$
10 **end if**
11 **if** $max \neq i$ **then**
12 swap$(A[i], A[max])$
13 MaxHeapify(A, max)
14 **end if**
end algorithm

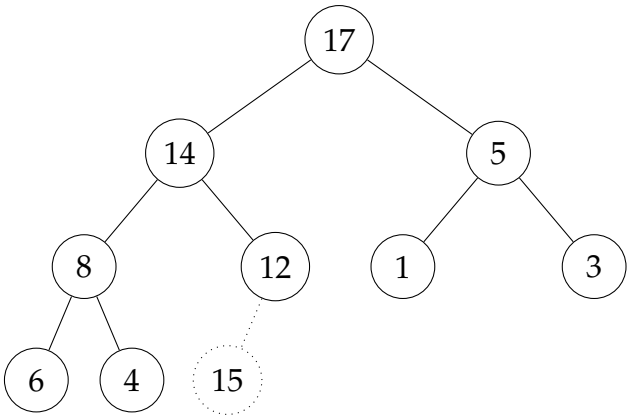
Beispiel 15 Anwenden von MaxHeapify auf einen Nicht-Heap:



Laufzeit: Der linke und der rechte Teilbaum haben höchstens $\frac{2}{3}n$ Knoten. Also $T(n) = T\left(\frac{2}{3}n\right) + c$. Das liegt in der Höhe des Heaps, also $T(n) \in \Theta(\log(n))$.

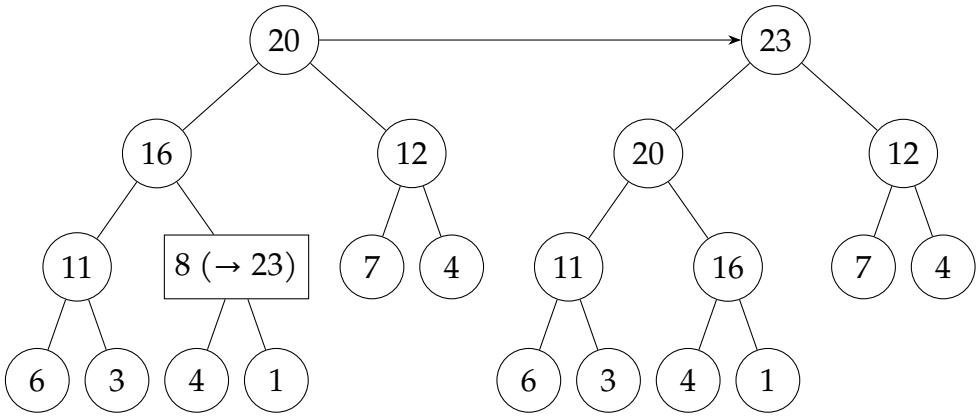
Beispiel 16 MaxHeapInsert$(A, 15)$ fügt neues Element in Heap ein (Anhängen und dann Hochtauschen).

Downloaded by Maja Müller (blackcatgirl@freenet.de)



MaxHeapIncreaseKey$(A, i,$ schlüssel): Schlüssel an Position i wird vergrößert (Priorität erhöht sich auf den Wert „schlüssel“)

Beispiel 17 MaxHeapIncreaseKey$(A, 5, 23)$



Algorithmus 6

```
algorithm MaxHeapIncreaseKey(A, i, key)
1     if key < A[i] then
2         Error
3     end if
4     A[i] = key
5     while i > 1 and A[Vater(i)] < A[i] do
6         swap(A[i], A[Vater(i)])
7         i = Vater(i)
8     end while
end algorithm
```

Algorithmus 7

```
algorithm MaxHeapInsert(A, key)
1     heap_größe[A] = heap_größe[A] + 1
2     A[heap_größe[A]] = −∞
3     MaxHeapIncreaseKey(A, heap_größe[A], key)
end algorithm
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Laufzeit beider oberer Algorithmen: $O(\text{Höhe}) = O(\log(n))$.

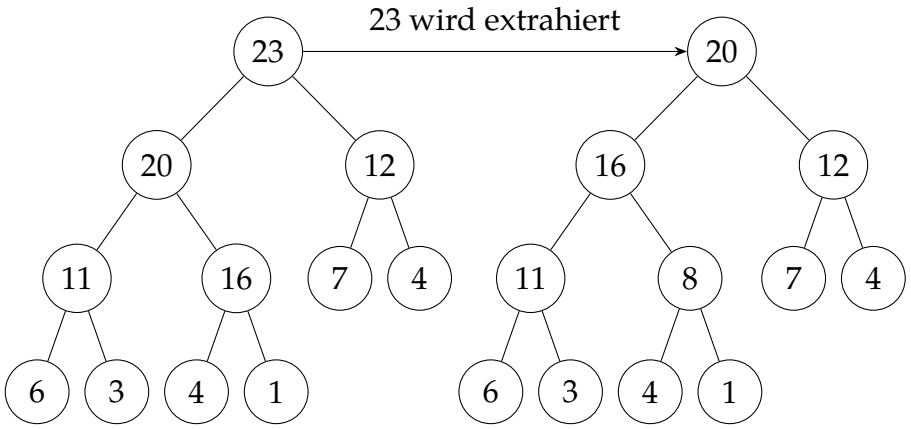
Algorithmus 8 MaxHeapExtractMax

- Input: Heap A
- Output: Maximum von A und Heap A', der alle Elemente außer das Maximum enthält

```
algorithm MaxHeapExtractMax(A)
1       if heap_größe[A] < 1 then
2           Error
3       end if
4       max = A[1]
5       A[1] = A[heap_größe[A]]
6       heap_größe[A] = heap_größe[A] − 1
7       MaxHeapify(A, 1)
8       return max
end algorithm
```

Laufzeit: $O(\text{Höhe}) = O(\log(n))$.

Beispiel 18 MaxHeapExtractMax(A)



Bemerkung 7 Alle Priorityqueue-Operationen benötigen nur $O(\log(n))$.

Build Heap

Baut aus einem ungeordneten Feld einen Max-Heap auf. Idee:

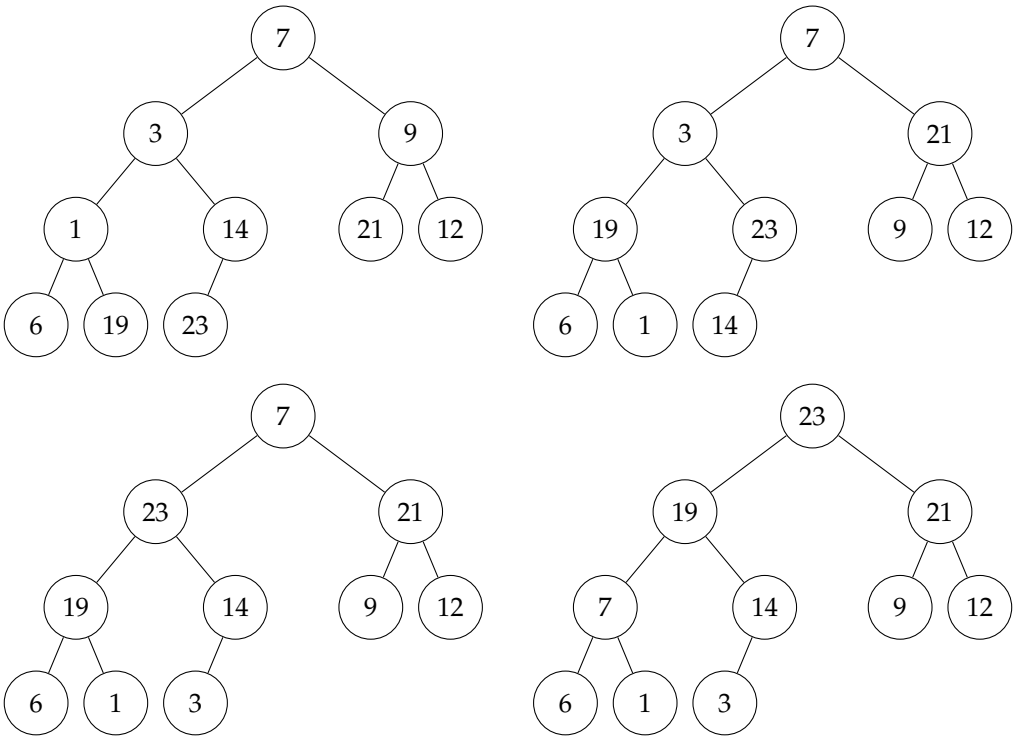
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Jedes Blatt ist zu Beginn ein Heap (enthält nur ein Element).
- Blätter stehen im Teilfeld $A[\lfloor \frac{n}{2} \rfloor + 1 ... n]$
- Aufruf von MaxHeapify für alle inneren Knoten beginnend mit $A[\lfloor \frac{n}{2} \rfloor]$ (dann $A[\lfloor \frac{n}{2} \rfloor - 1]$, bis $A[1]$)

Algorithmus 9

algorithm BuildMaxHeap(A)
1 heap_größe[A] = länge[A]
2 **for** $i = \lfloor \frac{\text{länge}[A]}{2} \rfloor$ **down to** 1 **do**
3 MaxHeapify(A, i)
4 **end for**
end algorithm

Beispiel 19 $A = [7, 3, 9, 1, 14, 21, 12, 6, 19, 23]$



Korrektheit:

- Invariante: Zu Beginn jeder Iteration der For-Schleife ist jeder Knoten $i+1, i+2, ..., n$ die Wurzel eines Heaps.
- Initialisierung: $i = \lfloor \frac{n}{2} \rfloor$. Jeder Knoten $\lfloor \frac{n}{2} \rfloor + 1, \lfloor \frac{n}{2} \rfloor + 2, ...$ ist ein Blatt und damit ein Heap.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

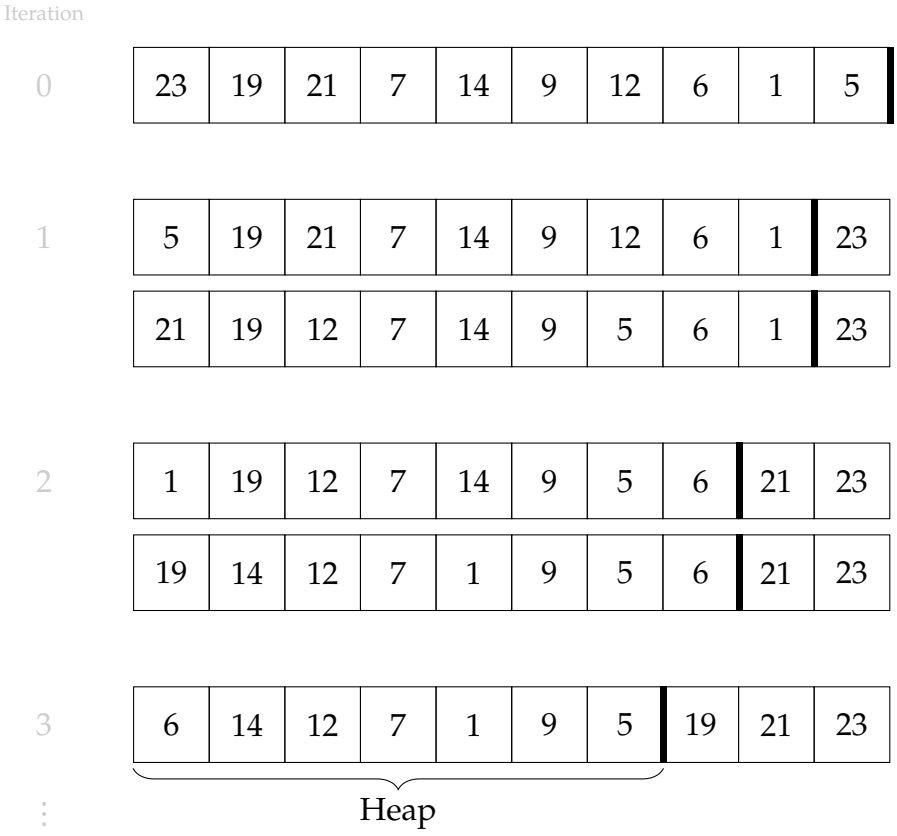
- Fortsetzung: Kinder von i haben höhere Indizes und sind nach Voraussetzung Wurzeln von Heaps. Nach MaxHeapify(A, i) ist auch i Wurzel eines Heaps.
- Terminierung: $i = 0$, dann ist $A[1...n]$ ein Heap.

Laufzeit:

- triviale Schranke: $\frac{n}{2}$ mal Aufruf von MaxHeapify, also $O(n \log(n))$
- Beobachtung: MaxHeapify wird oft für Bäume kleiner Höhe und selten für Bäume großer Höhe aufgerufen.
- Behauptung:
  - n-elementiger Heap hat Höhe $\lfloor \log(n) \rfloor$.
  - Es gibt höchstens $\lceil \frac{n}{2^{h+1}} \rceil$ Knoten der Höhe h.
  - MaxHeapify hat Laufzeit $O(h)$ für Knoten der Höhe h.
- $\sum_{h=0}^{\lfloor \log(n) \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \cdot c \cdot h = \frac{n \cdot c}{2} \sum_{h=0}^{\lfloor \log(n) \rfloor} \frac{h}{2^h}$. Da $\sum_{k=0}^{\infty} k \cdot k^k = \frac{x}{(1-x)^2}$, also ist die Summe konstant und damit in $\Theta(n)$.

Heapsort

Abbildung 7 Beispiel zur Funktionsweise von Heapsort anhand von $A = [23, 19, 21, 7, 14, 9, 12, 6, 1, 5]$



Downloaded by Maja Müller (blackcatgirl@freenet.de)

Algorithmus 10

```
algorithm HeapSort(A)
    BuildMaxHeap(A)
    for i = länge[A] down to 2 do
        swap(A[1], A[i])
        heap_größe[A] = heap_größe[A] − 1
        MaxHeapify(A, 1)
    end for
end algorithm
```

Laufzeit:

- BuildMaxHeap: $O(n)$
- n Mal MaxHeapify: $O(n \log(n))$

3.3 Quicksort

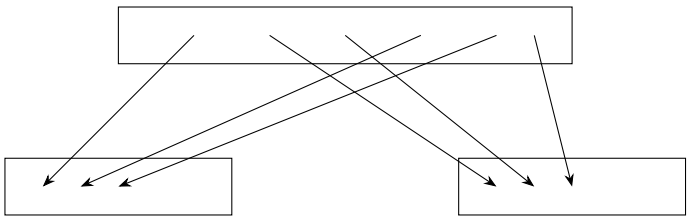
Bei Mergesort wird viel Arbeit beim Mischen aufgewandt, bei Quicksort beim Teilen.

- Worst case $\Theta(n^2)$, average case $\Theta(n \log(n))$
- In-Place-Algorithmus
- Teile-und-Herrsche-Algorithmus

Quicksort:

1. wähle Pivotelement v
2. zerlege Feld $A[l...r]$ in zwei Teilfelder $A[l...q-1]$ und $A[q+1...r]$, wobei alle Elemente im ersten Teilfeld kleiner oder gleich v sind und im zweiten Teilfeld größer v
3. sortiere beide Teilfelder rekursiv mit Quicksort
4. für das Zusammenfügen ist keine Arbeit nötig, da rekursiv in-place sortiert wird

Abbildung 8 Prinzipskizze zu Quicksort



Algorithmus 11

```
algorithm Quicksort(A, l, r)
    if r > l then
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
2           v = A[r]
3           q = Partition(A, l, r, v)          ▷ v ist Schlüssel
4           Quicksort(A, l, q − 1)
5           Quicksort(A, q + 1, r)
6       end if
    end algorithm
```

Bemerkung 8

- Teilfelder haben unterschiedliche Größen (je nach Wahl von v)
- Laufzeit hängt von der Wahl von v ab

Zwei Dinge zu lösen:

1. Wie findet man ein gutes v?
2. Wie funktioniert Partition?

Algorithmus 12

```
    algorithm Partition(A, l, r, v)
1       i = l − 1
2       j = r
3       repeat
4           repeat                    ▷ von links nach rechts bis Element > v
5               i = i + 1
6           until A[i] > v
7           repeat                    ▷ von rechts nach links bis Element ≤ v
8               j = j − 1
9           until A[j] ≤ v
10          if j > i then
11              swap(A[i], A[j])
12          else
13              fertig = true
14          end if
15      until fertig
16      swap(A[i], A[r])
17      return i
    end algorithm
```

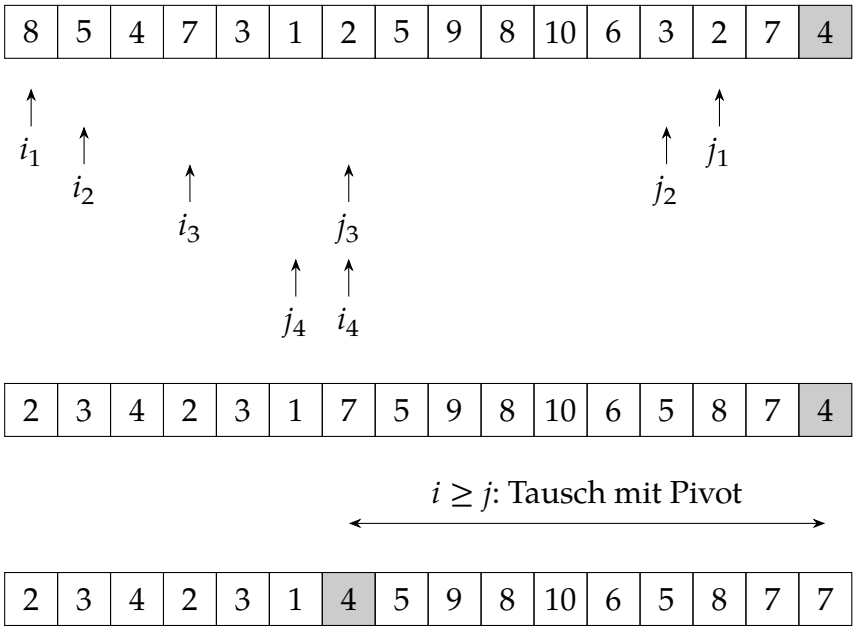
Laufzeit von Partition: $\Theta(r - l) = \Theta(n)$

Laufzeit von Quicksort:

- bester Fall: Das Pivotelement zerlegt Folge in zwei gleich große Teile (Wert ist mittleres Element der sortierten Folge). $T(n) = 2 \cdot T(\frac{n}{2}) + c \cdot n$, also $T(n) \in \Theta(n \log(n))$.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 9 Beispiel zur Sortierung per Quicksort (eine Iteration mit Pivotelement 4)



- schlechtester Fall: Input ist eine bereits sortierte Folge. $T(n) = T(n-1) + c \cdot n$, also $T(n) \in \Theta(n^2)$.

Satz 3 Sind alle Schlüssel im Eingabefeld paarweise verschieden, benötigt Quicksort im Mittel bezogen auf gleichwahrscheinliche Permutationen der Eingabefolgen $O(n \log(n))$ Vergleiche (ungefähr $1,38 \cdot n \log(n)$).

Beweis: $n = 1, T(1) = T(0) = 0$. Für $n \geq 2$ gilt $T(n) = n + 1 + \frac{1}{n} \sum_{k=1}^{n} [T(k-1) + T(n-k)]$. Der erste Summand stammt von Partition, und der zweite von Quicksort. Im Quicksort-Teil gilt, dass jedes Element als Pivotelement gleichwahrscheinlich ist (Pivot Rang i ist Aufteilung in $i-1$ und $n-i$). Damit ist $T(n) = n + 1 + \frac{2}{n} \sum_{k=1}^{n} T(k-1)$.

$$
\begin{aligned}
\frac{T(n)}{n+1} &= 1 + \frac{2}{n(n+1)} \sum_{k=1}^{n} T(k-1) \\
&= 1 + \frac{2}{n(n+1)} \sum_{k=1}^{n-1} T(k-1) + \frac{2}{n(n+1)} T(n-1) \\
&= 1 + \frac{2}{n(n+1)} T(n-1) + \underbrace{\frac{n-1}{n(n+1)} \cdot \frac{2}{n-1} \cdot \sum_{k=1}^{n-1} T(k-1)}_{T(n-1)-n} \\
&= 1 + \frac{2 \cdot T(n-1)}{n(n+1)} + \frac{(n-1)(T(n-1)-n)}{n(n+1)} \\
&= 1 + \frac{n-1}{n+1} + \frac{(n+1)}{n(n+1)} \cdot T(n-1)
\end{aligned}
$$

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$
\begin{aligned}
&= \frac{2}{n+1} + \frac{T(n-1)}{n} \text{ für } n \geq 3 \\
T(2) &= 3 \\
\frac{T(n)}{n+1} &= \frac{T(n-1)}{n} + \frac{2}{n+1} = \frac{T(n-2)}{n-1} + \frac{2}{n+1} + \frac{2}{n} = \ldots \\
&= \frac{T(2)}{3} + \frac{2}{4} + \frac{2}{5} + \ldots + \frac{2}{n} + \frac{2}{n+1} \\
&= \frac{2}{1} + \frac{2}{2} + \frac{2}{3} + \ldots + \frac{2}{n+1} - \frac{8}{3} \\
&= 2 \sum_{k=1}^{n+1} \frac{1}{k} - \frac{8}{3} \approx 2(1 + \ln(n)) \\
T(n) &\approx (n+1)(2 + 2\ln(n)) \approx 1,38 \cdot n \log(n)
\end{aligned}
$$

□

Bemerkung 9 „Alle Permutationen sind gleichwahrscheinlich“ gilt in der Praixs meist nicht. Variante „Randomized Quicksort“ verwendet statt Zeile 2 den folgenden Pseudocode:

02 i = RANDOM(p,r)
2a vertausche(A[i], A[r])
2b v = A[r]

Die erwartete Laufzeit von RandomizedQuicksort ist ebenfalls $O(n \log(n))$.

3.4 Selection

- Quicksort braucht im schlechtesten Fall $\Theta(n^2)$ Zeit.
- Wir versuchen diesen Fall auszuschließen, indem wir einen Algorithmus suchen, der in der Mitte teilt (Pivotelement).
- Der Median einer Folge $a_1, \ldots, a_n$ ist das Element, das in der sortierten Folge an Position $\lceil \frac{n}{2} \rceil$ steht.
- Der Median als Pivot für Quicksort liefert eine Laufzeit von $O(n \log(n))$, falls er in $O(n)$ gefunden wird.

Definition 7 Gegeben sei eine Folge $A = \langle a_1, \ldots, a_n \rangle$, wobei die a_i Elemente eine linear geordnete Menge sind[2]. Dann ist der *Rang* von a_i in A definiert als $\text{Rang}(a_i : A) = |\{x \mid x \in A, x \leq a_i\}|$. Wir schreiben $a_{(k)}$ für das Element vom Rang k.

Beispiel 20 Sei $A = (7, 3, 1, -4, 8, -6, 2)$. Dann ist $\text{Rang}(1 : A) = 3$.

[2] Speziell bedeutet dies, dass die Elemente paarweise verschieden sind, da jedes Element in einer Menge nur genau einmal vorkommt.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Definition 8 Der *Median* einer Folge $A = \langle a_1, ..., a_n \rangle$ ist das Element vom Rang $\lceil \frac{n}{2} \rceil$, also $a_{(\lceil \frac{n}{2} \rceil)}$.

Algorithmus 13 SELECTION

- Input: $A = \langle a_1, ..., a_n \rangle, k, 1 \leq k \leq n$
- Output: $a_{(k)}$ (Element vom Rang k)
- Ziel: Algorithmus für Selection mit Zeitkomplexität $O(n)$

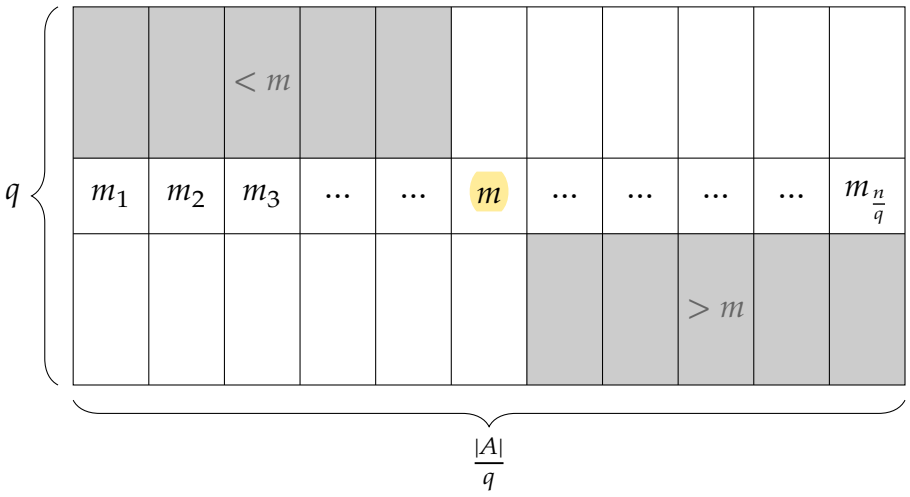
```
algorithm SELECTION(A, k)          ▷ Sei q ∈ ℕ beliebig und konstant.
1     ▷ Schritt 1
2     if |A| ≤ q then
3         Sortiere A.
4         return a_(k)
5     else
6         Zerlege A in ⌊|A|/q⌋ Teilfolgen mit q Elementen.
7     end if
8     ▷ Schritt 2
9     Sortiere jede Teilfolge und bestimme jeweils den Median
                                   ▷ pro Folge O(1), also O(n)
10    ▷ Schritt 3: Median der Medianе m berechnen
11    SELECTION([m_1, m_2, ..., m_⌊|A|/q⌋], ⌈|A|/2q⌉)        ▷ T(n/q)
12    ▷ Schritt 4                                              O(n)
13    A_1 = {x ∈ A | x < m}
14    A_2 = {x ∈ A | x = m}
15    A_3 = {x ∈ A | x > m}
16    ▷ Schritt 5
17    if |A_1| ≥ k then
18        return SELECTION(A_1, k)
19    else if |A_1| + |A_2| ≥ k then
20        return m
21    else
22        return SELECTION(A_3, k − |A_1| − |A_2|)
23    end if
end algorithm
```

Funktionsweise von Selection für Medianfindung:

- m ist der Median der Mediane.
- Für die q = konst. langen Teilfolgen wird der Median durch Sortieren bestimmt.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 10 Visualisierung von SELECTION für Mediane



- Durch den rekursiven Aufruf wird der Median der Mediane bestimmt.
- Mit m als Pivot-Element wird A in A_1, A_2, A_3 zerlegt.
- Durch den zweiten rekursiven Aufruf wird $a_{(k)}$ in A_1 oder $a_{(k-|A_1|-|A_2|)}$ in A_3 bestimmt.

Es stellen sich die Fragen: Wie gut teilt m? Wie groß sind A_1 und A_3 höchstens?

- $\frac{|A|}{2q}$ Mediane der Teilfolgen sind kleiner oder gleich als m.
- Für jeden dieser Mediane sind $\frac{q}{2}$ Elemente „seiner" Teilfolge kleiner oder gleich m.
- Daraus folgt, dass mindestens $\frac{|A|}{2q} \cdot \frac{q}{2} = \frac{|A|}{4}$ Elemente kleiner oder gleich m.
- Analog sind mindestens $\frac{|A|}{4}$ Elemente größer oder gleich m.
- Das heißt, $|A_1| \leq \frac{3}{4}|A|, |A_3| \leq \frac{3}{4}|A|$.

$$T(n) = T\left(\frac{n}{q}\right) + T\left(\frac{3}{4}n\right) + c \cdot n$$

gilt für $\frac{n}{q} + \frac{3}{4}n < n$, woraus folgt, dass $T(n) \in \Theta(n)$. Wir wählen dazu $q = 5$ oder größer, damit hat Selection eine Zeitkomplexität von $\Theta(n)$.

3.5 Eine untere Schranke für das Sortieren

- Frage: Geht Sortieren schneller als in $\Omega(n \log(n))$ Zeit?
- Sortieren auf der Basis von Schlüsselvergleichen: Alle Informationen, die zum Erstellen der sortierten Folge führen, werden aus Ver-

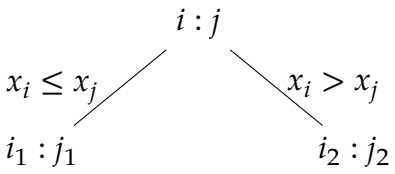
Downloaded by Maja Müller (blackcatgirl@freenet.de)

gleichen zwischen den zu sortierenden Schlüsseln genommen ($x > y?, x < y?, x \geq y?, x \leq y?$).

- Jeder beliebige Sortieralgorithmus, der auf Schlüsselvergleichen beruht, kann als Entscheidungsbaum dargestellt werden.

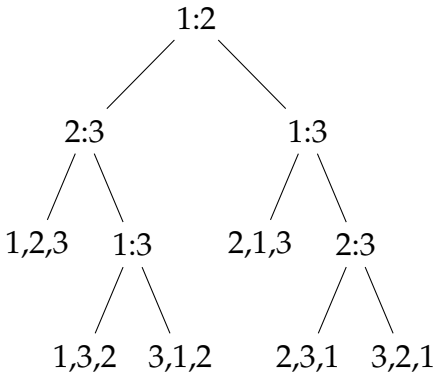
Sei SA ein Sortieralgorithmus und T der zu SA gehörige Entscheidungsbaum.

- Der Knoten $i : j$ entspricht dem Vergleich $x_i \leq x_j$.
- In der Wurzel steht der erste Vergleich des Algorithmus.



- $x_{i_1} \overset{?}{\leq} x_{j_1}$ ist der erste Vergleich nach $x_i \overset{?}{\leq} x_j$, der im Falle $x_i \leq x_j$ ausgeführt wird.
- $x_{i_2} \overset{?}{\leq} x_{j_2}$ ist der erste Vergleich nach $x_i \overset{?}{\leq} x_j$, der im Falle $x_i > x_j$ ausgeführt wird.
- Jedes Blatt enthält eine Permutation π der Indizes 1 bis n.
- Die Ausführung des Algorithmus entspricht dem Ablaufen eines Pfades von der Wurzel zu einem Blatt.

Beispiel 21 Entscheidungsbaum für InsertionSort auf einem Feld von 3 Elementen $A = [x_1, x_2, x_3]$.



Ergebnis: Indizes (die Feldelemente an diesen Indizes bilden die Schlüssel)

Bemerkung 10 Ein Algorithmus auf Basis von Schlüsselvergleichen kann nur korrekt sein, wenn jede der $n!$ Permutationen in einem Blatt steht (jede Permutation kann für einen konkreten Input die richtige sein).

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Die Länge des längsten Pfades ist die Worst-Case-Anzahl von Vergleichen. Also ist die Höhe des Baumes die Worst-Case-Anzahl von Vergleichen. Damit ist die Mindesthöhe eines binären Baumes mit $n!$ Blättern die untere Schranke für die Anzahl der Vergleiche im Worst-Case (für alle denkbaren Sortieralgorithmen auf Basis von Schlüsselvergleichen).

Satz 4 Jeder auf Schlüsselvergleichen basierende Sortieralgorithmus führt im Worst-Case $\Omega(n \log(n))$ Vergleiche aus.

Beweis: Wir bestimmen die minimale Höhe eines Binärbaumes, der jede der $n!$ Permutationen ein Blatt hat. Ein binärer Baum der Höhe h hat höchstens 2^h Blätter. Also soll gelten $n! \leq 2^h$ bzw. $\log(n!) \leq h$. Da $\left(\frac{n}{2}\right)^{\frac{n}{2}} \leq n!$ und damit $\frac{n}{2} \log\left(\frac{n}{2}\right) \leq \log(n!)$, folgt aus $\log(n!) \leq h$ direkt $h \in \Omega(n \log(n))$. □

3.6 Sortieren in linearer Zeit

Sortieren in linearer Zeit ist nur möglich, wenn der Algorithmus Informationen nicht ausschließlich aus Vergleichen erhält (zusätzliche Informationen über zu sortierende Elemente nötig, z.B. $\in \mathbb{N}, a \leq x \leq b$).

Countingsort

Sei $k \in \mathbb{N}, a_1, \ldots, a_n \in \mathbb{N}, 0 \leq a_i \leq k$. Alle Inputelemente sind natürliche Zahlen zwischen 0 und k.

Idee:

- Wir bestimmen für jedes Objekt den Rang, also die Anzahl der Elemente, die kleiner oder gleich sind.
- Mit dieser Information bestimmen wir die Position im Ausgabefeld.
- Auf Elemente mit gleichem Wert muss geachtet werden.

Algorithmus 14 COUNTINGSORT: Sei $A[1...n]$ der Input, $B[1...n]$ das sortierte Feld und $C[0...k]$ ein Hilfsfeld, welches zu jedem i zwischen $0 \leq i \leq k$ die Anzahl der Schlüssel, die gleich i sind. $0, \ldots, k$ sind die möglichen Schlüssel.

```
algorithm CountingSort(A, B, k)
1     for i = 0 to k do
2         C[i] = 0
3     end for
```


Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
4        for j = 1 to |A| do
5            C[A[j]] = C[A[j]] + 1
6        end for
7        ▷ C[i] enthält die Anzahl der Elemente, die gleich i sind.
8        for i = 1 to k do
9            C[i] = C[i] + C[i − 1]
10       end for
11       ▷ C[i] enthält die Anzahl der Elemente, die ≤ i sind.
12       for j = |A| down to 1 do
13           B[C[A[j]]] = A[j]
14           C[A[j]] = C[A[j]] − 1
15       end for
     end algorithm
```



Beispiel 22 Sei $A = [2,5,3,0,2,3,0,3]$. Dann ist $k = 5$ und $C = [2,0,2,3,0,1]$. Nach der Bildung der Partialsummen ist $C = [2,2,4,7,7,8]$. Anhand dessen wird B nun aufgefüllt:

B = [, , , , , , ,]
B = [, , , , , , 3,]
B = [0, , , , , , 3,]
B = [0, , , , , 3, 3,]
B = [0, , , 2, , 3, 3,]
B = [0, 0, , 2, , 3, 3,]
B = [0, 0, , 2, 3, 3, 3,]
B = [0, 0, , 2, 3, 3, 3, 5]
B = [0, 0, 2, 2, 3, 3, 3, 5]

Laufzeit:

- Die Zeilen 1–3 benötigen $\Theta(k)$, Zeilen 4–6 $\Theta(n)$, Zeilen 8–10 $\Theta(k)$ und Zeilen 12–15 $\Theta(n)$.
- Damit ist die Gesamtlaufzeit $T(n) = \Theta(n+k)$.

Bemerkung 11 Wenn k groß ist, wird das C-Feld groß. Falls $k \in O(n)$, hat Countingsort eine Laufzeit von $O(n)$. In diesem Fall wird es in der Praxis angewendet.

Definition 9 Ein Sortieralgorithmus heißt *stabil*, wenn gleiche Schlüssel im Ausgabefeld in der gleichen Reihenfolge wie im Eingabefeld stehen.

Bemerkung 12 CountingSort ist stabil.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Radixsort

- ursprünglich Sortierverfahren für Lochkarten
- heute z.B. für Geburtsdaten (Tag.Monat.Jahr) oder d-stellige Dezimalzahlen
- intuitiv: zuerst nach höchstwertigem Bit sortieren (Jahr vor Monat vor Tag) → viele Zwischenstapel
- RADIXSORT beginnt mit dem niederwertigstem Bit.

Beispiel:

Input: 318, 425, 928, 535, 575, 219, 374, 883, 392
Bit 0: 392, 883, 374, 425, 535, 575, 318, 928, 219
Bit 1: 318, 219, 425, 928, 535, 374, 575, 883, 392
Bit 2: 219, 318, 374, 392, 425, 535, 575, 883, 928

Algorithmus 15

algorithm RADIXSORT(A, d)
1 **for** $i = 1$ **to** d **do**
2 Verwende ein stabiles Verfahren, um A nach Bitposition i zu sortieren.
3 **end for**
end algorithm

Laufzeitanalyse:

- n ist d-stellige Dezimalzahl
- k ist die Zahl der möglichen Werte pro Bitposition (Anzahl der Ziffern, $k = 10$ für Dezimalsystem)
- Wenn das in Zeile 2 verwendete Verfahren $O(n + k)$ Zeit benötigt, sortiert RADIXSORT in $\Theta(d \cdot (n + k))$ Zeit.

Bemerkung 13 Falls k nicht zu groß ist, ist COUNTINGSORT eine gute Wahl. Für $d = \text{const.}$ und $k \in O(n)$ gilt $T(n) \in \Theta(n)$. Man hat eine gewisse Flexibilität, die Schlüssel in Stellen aufzulösen (dezimal, binär, andere Basis).

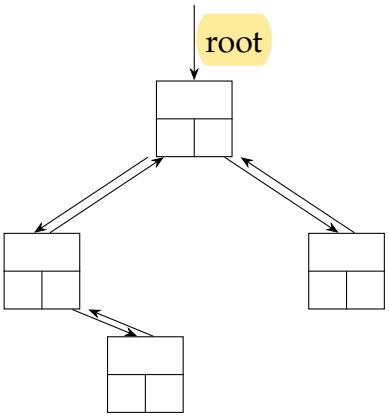
4 Suchbäume

- Der *Suchbaum* ist eine Datenstruktur.
- Operationen auf Suchbäumen: SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT, DELETE
- wünschenswert $O(\log(n))$ Zeit für all diese Operationen

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Anwendung als Implementierung der abstrakten Datentypen Dictionary und Priority Queue
- Dictionary = {Insert, Delete, Search}
- Der Suchbaum ist eine Datenstruktur, die als binärer Wurzelbaum[3] organisiert ist (verkettete Datenstruktur).

Abbildung 11 Beispiel für einen Suchbaum



Jeder Knoten enthält:

- Schlüssel (und weitere Daten)
- Zeiger auf
  - left (linkes Kind)
  - right (rechtes Kind)
  - parent (Vaterknoten)

Definition 10 Ein binärer Suchbaum ist eine verkettete, als binärer (Wurzel-)Baum organisierte Datenstruktur, in der für jeden Knoten x gilt

- $\text{key}[x] \leq \text{key}[y]$ für alle y aus dem rechten Teilbaum und
- $\text{key}[x] \geq \text{key}[y]$ für alle y aus dem linken Teilbaum.

4.1 Grundlegende Operationen

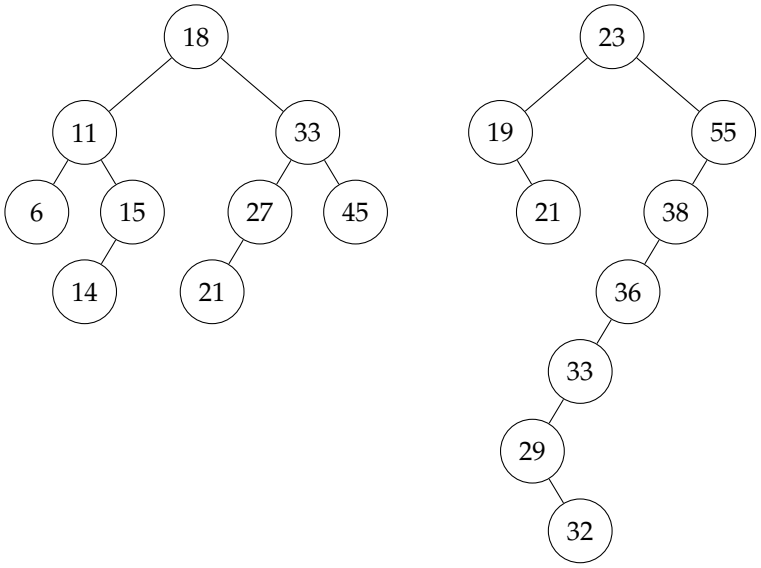
Traversierung von binären Bäumen

Ausgeben aller Schlüssel in festgesetzter Reihenfolge. Rekursive Definition der Reihenfolgen:

[3] Ein Baum ist ein Wurzelbaum, wenn ein Knoten als Wurzel ausgezeichnet ist.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 12 Beispiele für binäre Suchbäume



- Inorder-Tree-Walk: linker Teilbaum, Wurzel, rechter Teilbaum
- Preorder-Tree-Walk: Wurzel, linker Teilbaum, rechter Teilbaum
- Postorder-Tree-Walk: linker Teilbaum, rechter Teilbaum, Wurzel

Algorithmus 16

```
algorithm InorderTreeWalk(x)
1     if x ≠ NIL then
2         InorderTreeWalk(left[x])
3         print(key[x])
4         InorderTreeWalk(right[x])
5     end if
end algorithm
```

Algorithmus 17

```
algorithm PreorderTreeWalk(x)
1     if x ≠ NIL then
2         print(key[x])
3         PreorderTreeWalk(left[x])
4         PreorderTreeWalk(right[x])
5     end if
end algorithm
```

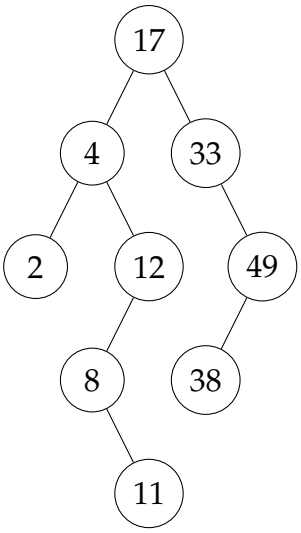
Algorithmus 18

```
algorithm PostorderTreeWalk(x)
1     if x ≠ NIL then
2         PostorderTreeWalk(left[x])
3         PostorderTreeWalk(right[x])
4         print(key[x])
5     end if
```


Downloaded by Maja Müller (blackcatgirl@freenet.de)

end algorithm

Beispiel 23 Wir traversieren den folgenden binären Baum:



- Inorder: 2, 4, 8, 11, 12, 17, 33, 38, 49
- Preorder: 17, 4, 2, 12, 8, 11, 33, 49, 38
- Postorder: 2, 11, 8, 12, 4, 38, 49, 33, 17

Bemerkung 14 InorderTreeWalk(x) liefert die sortierte Folge aller Schlüssel aus dem Teilbaum mit der Wurzel x. Beweis per Induktion mit der Suchbaumeigenschaft.

Bemerkung 15 Laufzeit:

$$T(0) = c \qquad (c \text{ konstant})$$
$$T(n) = T(k) + T(n-k-1) + d \qquad (d \text{ konstant})$$

Substitutionsmethode mit der Annahme $T(n) \leq (c+d)n + c$. Für $n = 0$ ist $(c+d)0 + c = T(0)$ und

$$\begin{aligned} T(n) &= T(k) + T(n-k-1) + d \\ &\leq (c+d)k + c + (c+d)(n-k-1) + c + d \\ &= (c+d)n + 2c - (c+d) + d \\ &= (c+d)n + c \end{aligned}$$

Satz 5 Für einen binären Suchbaum mit n Knoten und Wurzel x liefert InorderTreeWalk die sortierte Folge aller Schlüssel im Baum in $\Theta(n)$ Zeit.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Suchen

Algorithmus 19 TreeSearch

- Input: Schlüssel k, Zeiger auf Wurzel eines Suchbaumes
- Output: Zeiger auf einen Knoten mit Schlüssel k

```
algorithm TreeSearch(x, k)
1     if x = NIL or k = key[x] then
2         return x
3     else if k < key[x] then
4         return TreeSearch(left[x], k)
5     else
6         return TreeSearch(right[x], k)
7     end if
end algorithm
```

Algorithmus 20 IterativeTreeSearch (gleicher Algorithmus wie TreeSearch ohne rekursive Aufrufe)

```
algorithm IterativeTreeSearch(x, k)
1     while x ≠ NIL and k ≠ key[x] do
2         if k < key[x] then
3             x = left[x]
4         else
5             x = right[x]
6         end if
7     end while
8     return x
9     ▷ alternative Rückgabe key[x] bei Suche nach Schlüssel
end algorithm
```

Laufzeit: $O(h)$ mit der Höhe des Baumes h.

Minimum und Maximum

Algorithmus 21 TreeMinimum

- Input: Zeiger auf Wurzel eines Suchbaumes
- Output: Zeiger auf einen Knoten mit kleinstem Schlüssel (alternativ Schlüssel selbst)

Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
algorithm TreeMinimum(x)
1       while left[x] ≠ NIL do
2           x = left[x]
3       end while
4       return x
end algorithm
```

Algorithmus 22 TreeMaximum

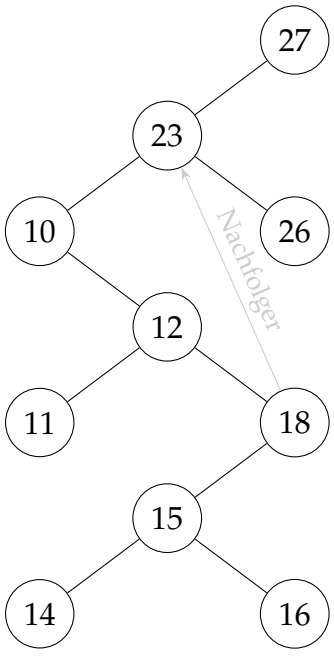
- Input: Zeiger auf Wurzel eines Suchbaumes
- Output: Zeiger auf einen Knoten mit größtem Schlüssel (alternativ Schlüssel selbst)

```
algorithm TreeMaximum(x)
1       while right[x] ≠ NIL do
2           x = right[x]
3       end while
4       return x
end algorithm
```

Laufzeit: $O(h)$ mit der Höhe des Baumes h.

Vorgänger und Nachfolger

Abbildung 13 Beispiel zur Bestimmung des Nachfolgers in einem Suchbaum



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Algorithmus 23 TreeSuccessor

- Input: Knoten x
- Output: Knoten mit kleinstem Schlüssel, der größer als key$[x]$

```
algorithm TreeSuccessor(x)
1     if right[x] ≠ NIL then
2         return TreeMinimum(right[x])
3     end if
4     y = parent[x]
5     while y ≠ NIL and x = right[y] do
6         x = y
7         y = parent[y]
8     end while
9     return y
end algorithm
```

Für paarweise verschiedene Schlüssel:

- Fall 1: Nachfolger ist kleinstes Element im rechten Teilbaum
- Fall 2: x hat kein rechtes Kind, also ist der Nachfolger auf dem Weg von x zur Wurzel und zwar der erste Knoten, der von links betreten wird

Bemerkung 16 Falls die Schlüssel nicht paarweise verschieden sind, ist der Nachfolger dasjenige Element, das TreeSuccessor liefert.

Satz 6 Die Operationen Search, Minimum, Maximum, Predecessor, Successor können in binären Suchbäumen in $O(h)$ Zeit ausgeführt werden mit h als der Höhe des Baumes.

Einfügen

- Der Suchbaum muss modifiziert werden.
- Die Suchbaumeigenschaft muss erhalten bleiben.

Algorithmus 24 TreeInsert

- Input:
  - Suchbaum T (Zeiger auf die Wurzel von T)
  - Output: Knoten z mit key$[z] = v$, wobei left$[z]$ = NIL und right$[z]$ = NIL.

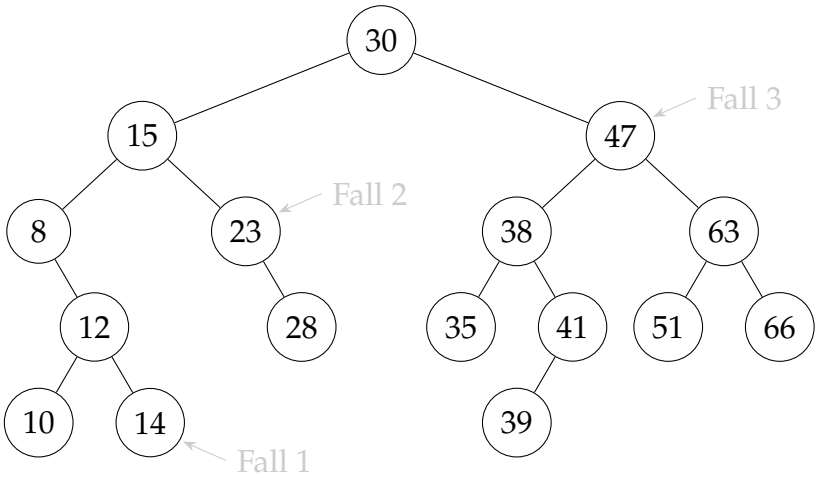
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Output: Suchbaum T', der alle Knoten aus T und zusätzlich z enthält.

```
algorithm TreeInsert(T, z)
1     y = NIL
2     x = root[T]
3     while x ≠ NIL do
4         y = x
5         if key[z] < key[x] then
6             x = left[x]
7         else
8             x = right[x]
9         end if
10    end while
11    parent[z] = y
12    if y = NIL then                          ▷ T war leer
13        root[T] = z
14    else if key[z] < key[y] then
15        left[y] = z
16    else
17        right[y] = z
18    end if
end algorithm
```

Löschen

Beispiel 24 Löschen aus Suchbaum:



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Fall 1: keine Kinder, dann entferne den Knoten und aktualisiere den Elternknoten
- Fall 2: ein Kind, dann das Kind des zu entfernden Knoten an den Elternknoten des zu entfernenden Knoten hängen
- Fall 3: Knoten z hat zwei Kinder: Nachfolger von z aus dem Baum entfernen und an z's Stelle setzen

Algorithmus 25 TreeDelete

- Input: Suchbaum T, Knoten z aus T
- T', der genau die Knoten aus T bis auf z enthält

```
algorithm TreeDelete(T, z)
1     if left[z] = NIL or right[z] = NIL then
2         y = z
3     else
4         y = TreeSuccessor(z)
5     end if
6     if left[y] ≠ NIL then
7         x = left[y]
8     else
9         x = right[y]
10    end if
11    if x ≠ NIL then
12        parent[x] = parent[y]
13    end if
14    if parent[y] = NIL then
15        root[T] = x
16    else
17        if y = left[parent[y]] then
18            left[parent[y]] = x
19        else
20            right[parent[y] = x
21        end if
22    end if
23    if y ≠ z then
24        key[z] = key[y]
25        Kopiere die Satellitendaten von y in z.
26    end if
27    return y
end algorithm
```


Downloaded by Maja Müller (blackcatgirl@freenet.de)

Bemerkung 17 Kommentare zum Algorithmus:

- Zeilen 1–5: Bestimme Knoten y, der herausgelöst werden kann, also $y = z$, falls z höchstens ein Kind
- Zeilen 6–10: Setze x auf das Nicht-Nil-Kind von y bzw. auf Nil, falls y kinderlos ist
- Zeilen 11–13: y herauslösen
- Zeilen 14–22: Zeiger umsetzen
- Zeilen 23–26: Daten umschreiben

Satz 7 INSERT und DELETE im binären Suchbaum benötigen $O(h)$ Zeit für h als die Höhe des Baumes.

4.2 Rot-Schwarz-Bäume

Definition und grundlegende Eigenschaften

- Ziel: Suchbäume, deren Höhe immer $O(\log(n))$ ist.
- Jeder Knoten enthält Attribute Color, Key, Left, Right, Parent.

Definition 11 Ein binärer Suchbaum ist ein *Rot-Schwarz-Baum*, wenn er folgende Eigenschaften erfüllt:

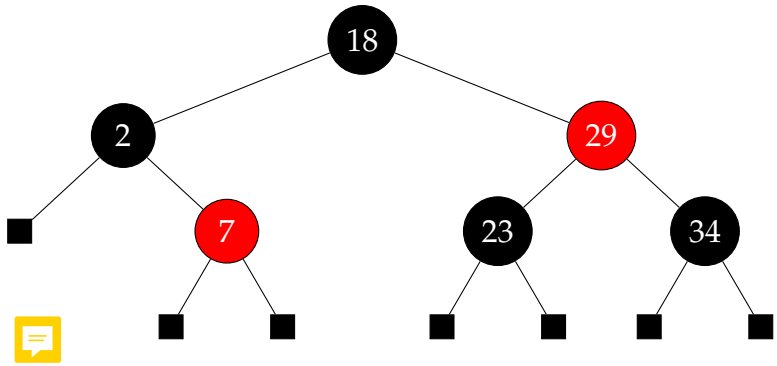
1. Jeder Knoten ist entweder rot oder schwarz.
2. Die Wurzel ist schwarz.
3. Alle Nil-Blätter sind schwarz.
4. Ist ein Knoten rot, dann sind seine beiden Kinder schwarz.
5. Für jeden Knoten v gilt: Alle Pfade zu einem (von v abstammenden) Blatt enthalten die gleiche Anzahl schwarzer Knoten.

Bemerkung 18 Bemerkung zu Nil-Kindern:

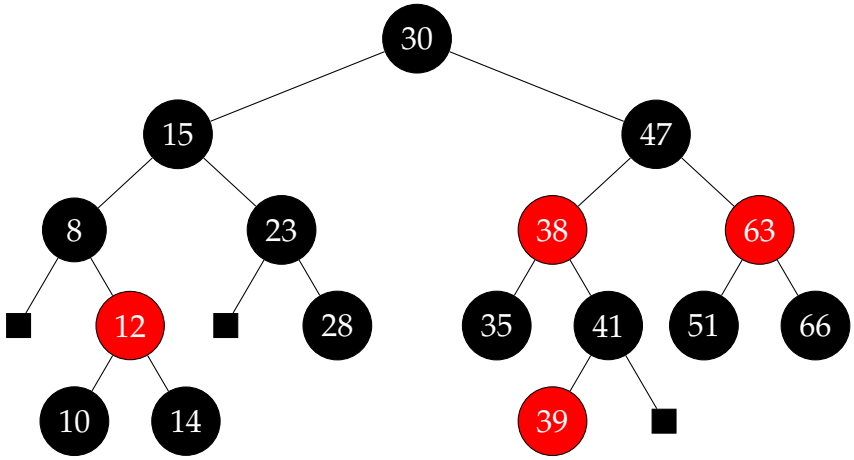
- Jeder Knoten hat genau zwei Kinder, das sind unter Umständen Nil-Knoten.
- Knoten mit Schlüssel heißen innere Knoten.

Beispiel 25 Beispiel für einen Rot-Schwarz-Baum:

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Beispiel 26 Nicht-rot-schwarz-färbbarer Suchbaum (einige Nil-Knoten vernachlässigt):



Bemerkung 19 nil[T] ist ein Objekt mit Attributen wie gewöhnliche Knoten mit color[nil[T]] = schwarz. Parent, Left, Right und Key können beliebige Werte annehmen.

Definition 12 Sei T ein Rot-Schwarz-Baum und x ein innerer Knoten von T. Die *Schwarzhöhe* von x, $\mathrm{bh}(x)$, ist die Anzahl schwarzer Knoten auf einem Pfad von x zu einem (von x abstammenden) Blatt. Dabei wird x selbst nicht mitgezählt, das erreichte Blatt aber schon.

Satz 8 Die Höhe eines Rot-Schwarz-Baumes mit n inneren Knoten ist höchstens $2\log(n+1)$.

Bemerkung 20 Hilfsaussage: Für jeden Knoten x gilt: Die Zahl der inneren Knoten im Teilbaum mit Wurzel x ist mindestens $2^{\mathrm{bh}(x)} - 1$.

Beweis: Induktion über Höhe des Knotens x. Dabei ist $n(x)$ die Zahl der inneren Knoten im Teilbaum mit Wurzel x.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Induktionsanfang mit $h = 0$: x ist ein Nil-Blatt, also ist $n(x) = 0$ und $\text{bh}(x) = 0 = 2^0 - 1$.
- Induktionsvoraussetzung: Für alle Knoten x mit $h(x) \leq k$ gilt $n(x) \geq 2^{\text{bh}(x)} - 1$.
- Induktionsbehauptung: Dann gilt auch für alle Knoten x mit $h(x) = k + 1$ $n(x) \geq 2^{\text{bh}(x)} - 1$.

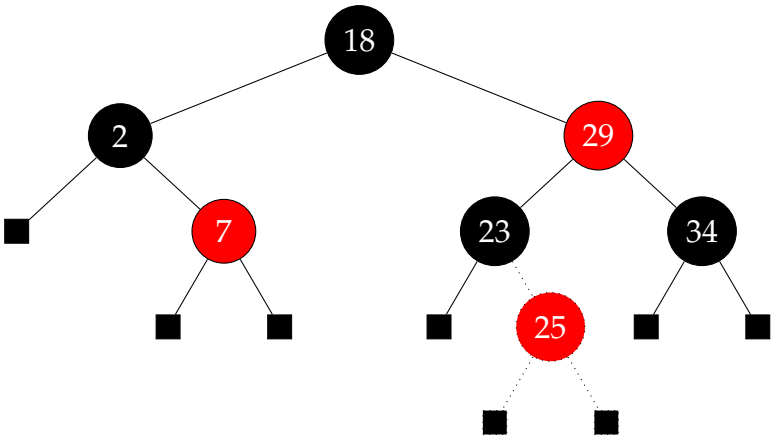
Beweis des Induktionsschrittes: Sei x ein Knoten mit $h(x) = k+1$ (innerer Knoten mit zwei Kindern, die Nil sein können). Wegen $k \geq 0$ gilt $k+1 \geq 1$. Jedes Kind hat Schwarzhöhe $\text{bh}(x)$ (falls Kind rot) oder $\text{bh}(x)-1$ (falls Kind schwarz). Sei x_l das linke Kind und x_r das rechte Kind. Damit ist $h(x_l) < h(x)$ und $h(x_r) < h(x)$ wegen $h(x) = \max\{h(x_l), h(x_r)\}+1$. Mit der Induktionsvoraussetzung:

$$\begin{aligned} n(x_l) &\geq 2^{\text{bh}(x_l)} - 1 && \geq 2^{\text{bh}(x)-1} - 1 \\ n(x_r) &\geq 2^{\text{bh}(x_r)} - 1 && \geq 2^{\text{bh}(x)-1} - 1 \\ n(x) &\geq 1 + 2 \cdot (2^{\text{bh}(x)-1} - 1) = 2^{\text{bh}(x)} - 1 \end{aligned}$$

Beweis von Satz 8: Sei h die Höhe von T. Aus Eigenschaft 4 folgt, dass auf jedem Pfad von Wurzel zu Blatt mindestens die Hälfte der Knoten schwarz sind (da Anfang und Ende schwarz). Daraus folgt $\text{bh}(\text{root}(T)) \geq \frac{h}{2}$. Aus der Hilfsaussage folgt, dass $n = n(\text{root}(T)) \geq 2^{\text{bh}(\text{root}(T))} - 1 \geq 2^{\frac{h}{2}} - 1$, also $2\log(n+1) \geq h$. □

Bemerkung 21 Folgerung: Die Operationen SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR auf Rot-Schwarz-Bäumen benötigen $O(\log(n))$ Zeit. INSERT und DELETE nicht, da sie den Baum modifizieren und nicht sichergestellt ist, dass der entstehende Baum Rot-Schwarz-Baum ist.

Beispiel 27 Einfügen einer 25 in einen RS-Baum:

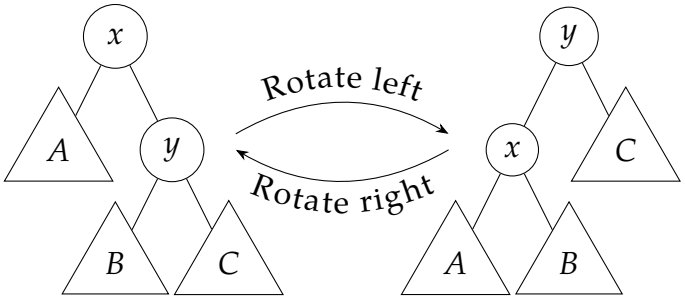


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Einfügen einer 12 ist weniger trivial.

Rotationen

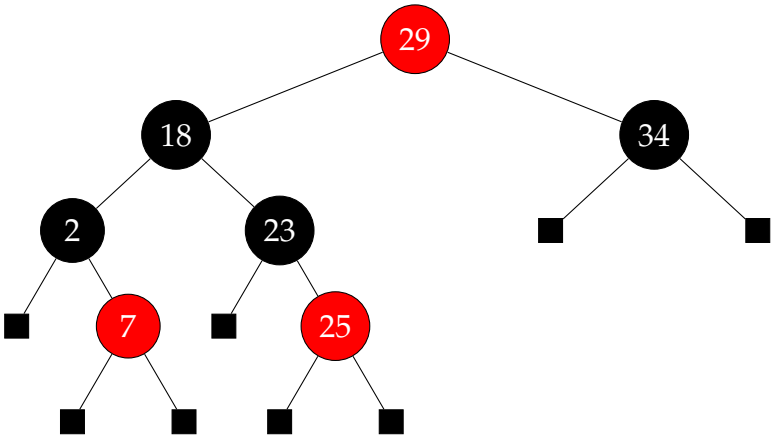
Lokale Rotation, die die Suchbaumeigenschaft erhält.



A, B, C sind Suchbäume, die nicht verändert werden.

- für LeftRotate muss $\text{right}[x] \neq \text{NIL}$ gelten
- für RightRotate muss $\text{left}[x] \neq \text{NIL}$ gelten

Beispiel 28 LeftRotate auf den Baum aus Beispiel 27



Algorithmus 26

```
algorithm LeftRotate(T, x)
1   y = right[x]
2   right[x] = left[y]
3   if left[y] ≠ nil[T] then
4       parent[left[y]] = x
5   end if
6   parent[y] = parent[x]
7   if parent[x] = nil[T] then
8       ▷ Wenn x Wurzel war, wird y Wurzel.
9       root[T] = y
```


Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
10          else
11              if x = left[parent[x]] then
12                  left[parent[x]] = y
13              else
14                  right[parent[x]] = y
15              end if
16          end if
17          left[y] = x
18          parent[x] = y
        end algorithm
```

Laufzeit: $O(1)$.

Einfügen

- zunächst wie in normalen Suchbäumen
- neuen Knoten rot färben
- falls Eigenschaft 4 verletzt, Modifikation des Baumes

Algorithmus 27

```
    algorithm RBInsert(T, z)
1       Zeilen 1–18 wie TreeInsert(T, z), wobei NIL durch nil[T] ersetzt
        wird
19      left[z] = nil[T]
20      right[z] = nil[T]
21      color[z] = rot
22      RBInsertFixup(T, z)
    end algorithm
```

Algorithmus 28

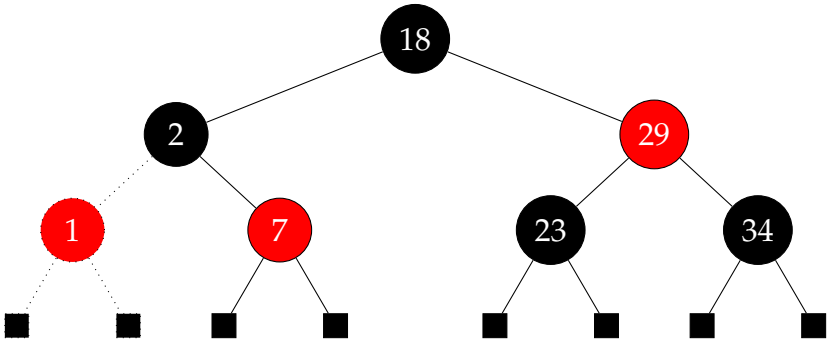
```
    algorithm RBInsertFixup(T, z)
1       while color[parent[z]] = rot do
2           if parent[z] = left[parent[parent[z]] then
3               ▷ Vater ist linkes Kind von Opa
4               y = right[parent[parent[z]]]          ▷ y ist Onkel von z
5               if color[y] = rot then
6                   ▷ 1. Fall: roter Onkel
7                   ▷ Umfärben: Vater und Onkel schwarz, Opa rot
8                   color[parent[z]] = schwarz
9                   color[y] = schwarz
10                  color[parent[parent[z]]] = rot
11                  z = parent[parent[z]]              ▷ weiter mit Opa
12              else
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

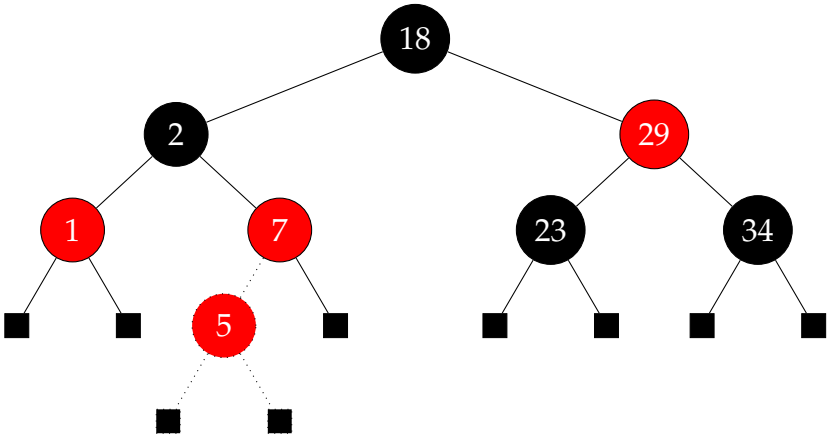
```
13                  if z = right[parent[z]] then
14                      ▷ 2. Fall: schwarzer Onkel, Knick auf Weg zu Opa
15                      z = parent[z]
16                      LeftRotate(T, z)            ▷ Linksrotation um Vater
17                  end if
18                  color[parent[z]] = schwarz
19                  color[parent[parent[z]] = rot
20                  RightRotate(T, parent[parent[z]])
21              end if
22          else
23              ▷ 3. Fall: schwarzer Onkel, gerader Weg zu Opa
24              ▷ wie then-Zwei, wobei left und right vertauscht sind
25          end if
26      end while
27      color[root[T]] = schwarz
    end algorithm
```

Beispiel 29

Insert 1: keine RS-Eigenschaft verletzt

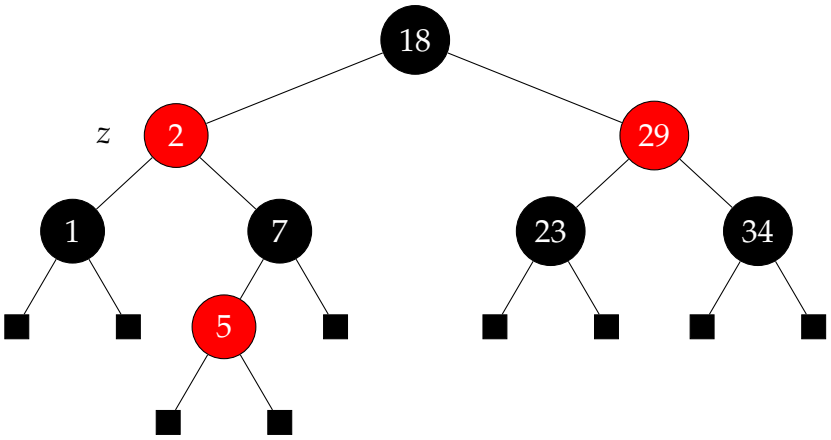


Insert 5 (Fall roter Onkel):



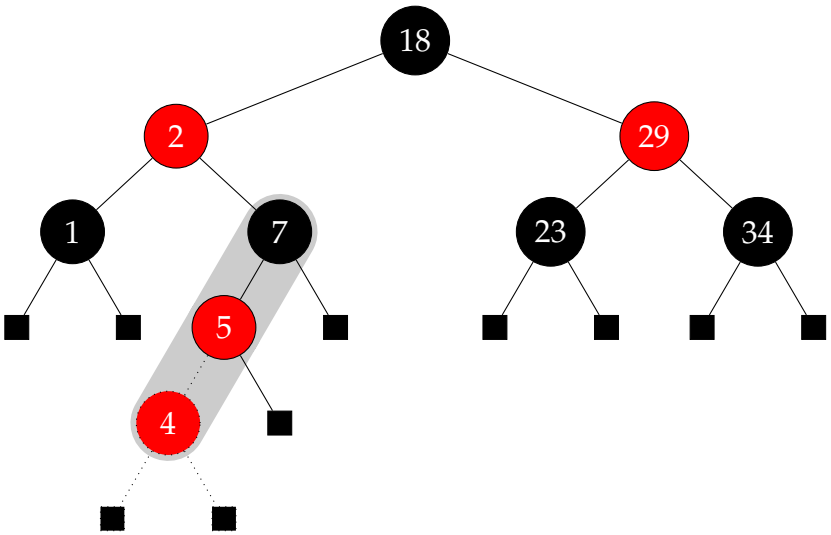
wird zu (Vater schwarz, Opa rot; Rechtsrotation um Opa)

Downloaded by Maja Müller (blackcatgirl@freenet.de)

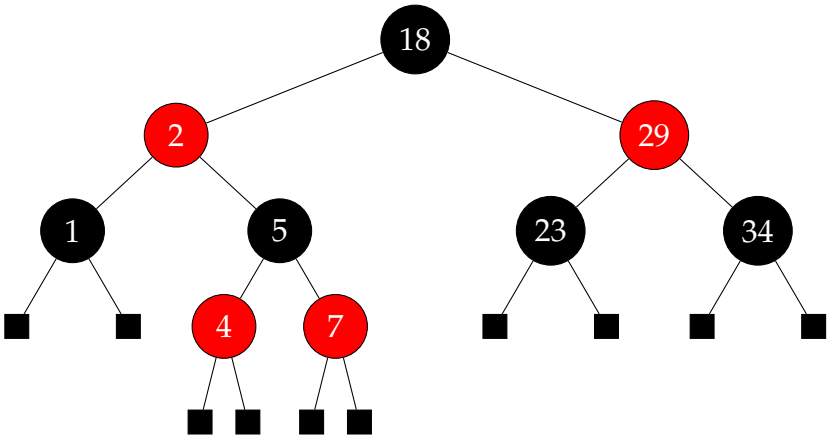


Insert 4:

- schwarzer Onkel (rechtes NIL-Blatt der 7)
- gerader Weg zu Opa → 3. Fall

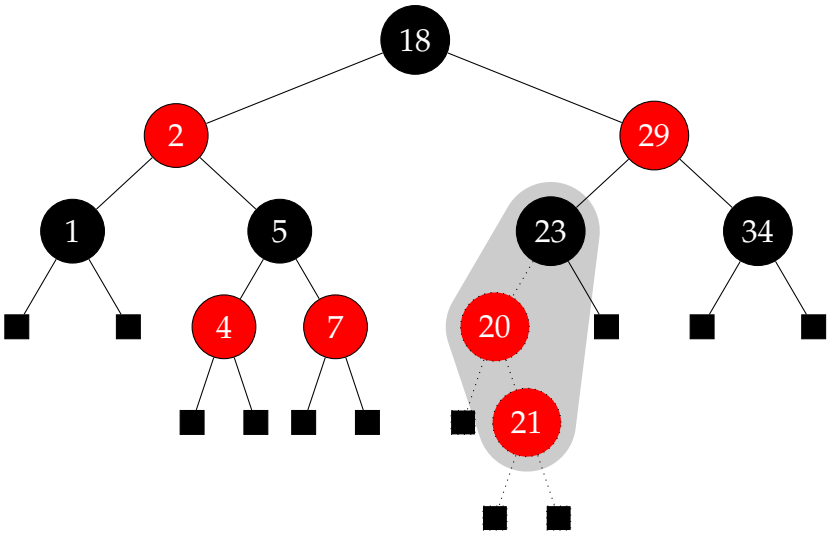


wird zu

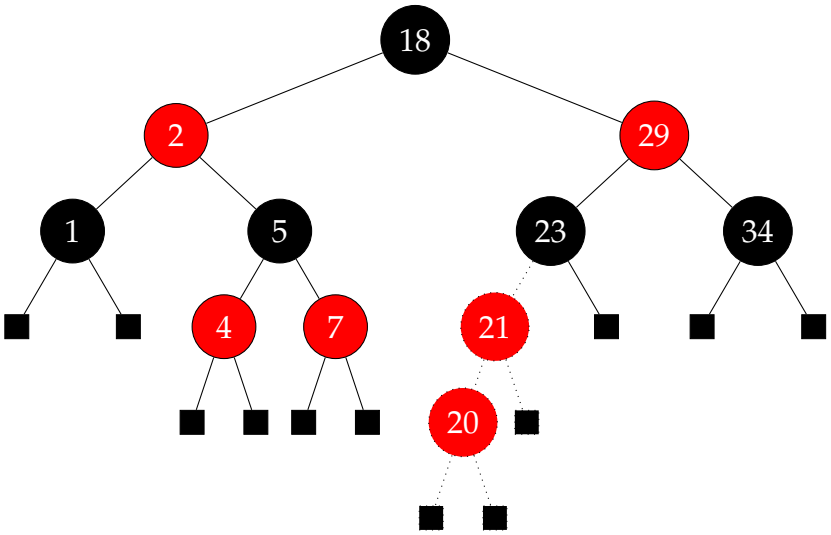


Insert 20, 21 (21: schwarzer Opa, Knick auf Weg zu Opa, 2. Fall)

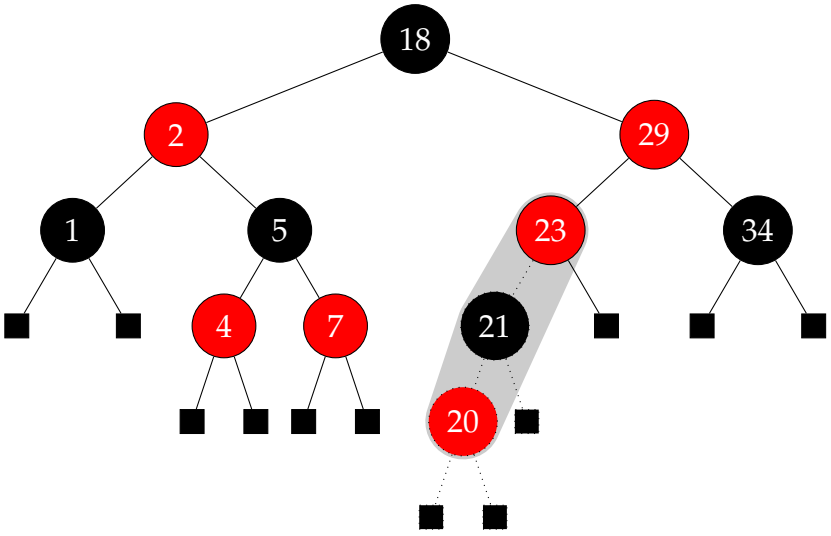
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Links-Rotation um Vater

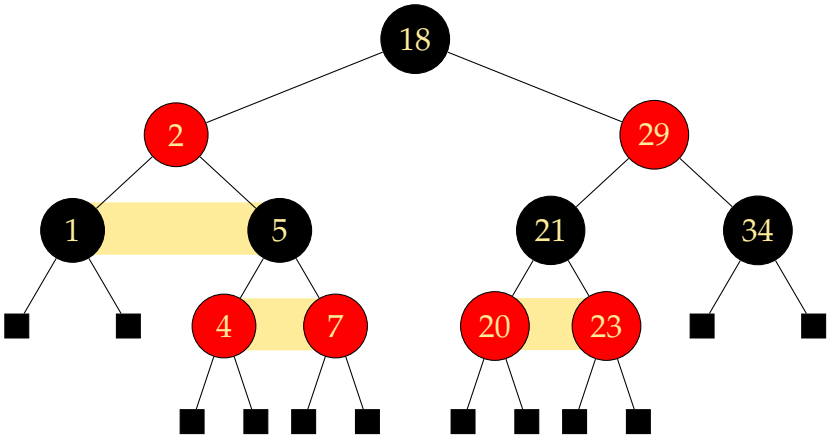


Umfärben mit Vater rot und Opa schwarz



Downloaded by Maja Müller (blackcatgirl@freenet.de)

Rechts-Rotation um Opa



Behauptung: RBInsertFixup ist korrekt, d.h. es stellt die Rot-Schwarz-Baum-Eigenschaften wieder her. Welche Eigenschaften könnten durch RBInsert verletzt sein?

- 2: Wurzel ist rot, in diesem Fall war der Baum leer → Abhilfe in letzter Zeile
- 4: Vater des eingefügten Kindes ist auch rot

Invariante für die While-Schleife (Zeilen 1–26):

i. color[z] = rot
ii. Falls parent[z] Wurzel, ist parent[z] schwarz.
iii. Wenn es Verletzungen der RS-Eigenschaften gibt, dann ist es genau eine und zwar Nr. 2 oder Nr. 4.

Initialisierung: i, ii, iii gelten vor dem ersten Durchlauf der Schleife.

i. z wird als roter Knoten angefügt.
ii. Wenn parent[z] Wurzel ist, war er schwarz und hat seine Farbe nicht geändert.
iii. 1, 3, 5 gelten, wenn RBInsertFixup aufgerufen wird. Wenn 2 verletzt ist, dann war der Baum leer und keine andere Eigenschaft wird verletzt. Wenn 4 verletzt ist, dann ist der Vater von z rot, damit ist p[z] nicht die Wurzel und damit 2 nicht verletzt.

Fortsetzung (i, ii, iii werden während des Durchlaufs der While-Schleife nicht gestört):

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- 6 Fälle: wir betrachten nur die drei „Vater ist linkes Kind von Opa“ Fälle, die anderen sind symmetrisch
- parent[parent[z]] existiert, da nur etwas getan wird, wenn parent[z] rot, dann ist parent[z] aber nicht die Wurzel
- Fall 1: z hat roten Onkel (Umfärben und z auf Opa setzen)
  i. parent[parent[z]] wird rot gefärbt und ist das neue z (bezeichnet als z')
  ii. parent[z'] ist parent[parent[parent[z]]], die Farbe dieses Knotens ändert sich nicht. Falls er Wurzel ist, war er schwarz und bleibt es.
  iii. 1 und 3 werden nicht verletzt.
    - z' zu Beginn der nächsten Iteration die Wurzel: 4 wurde korrigiert, damit ist 2 die einzige mögliche Verletzung, das wird aber auch korrigiert.
    - z' nicht Wurzel: keine Verletzung von 2 passiert, damit ist nur die Verletzung von 4 möglich, die aber lokal korrigiert wird: falls parent[z'] schwarz, dann fertig, falls parent[z'] rot, dann wieder Verletzung von 4
- Fall 2: Onkel schwarz, Knick auf Weg zu Opa: Links-Rotation überführt Fall 2 in Fall 3.
  – Eigenschaft 5 wird durch Rotation nicht beeinflusst, da zwei rote Knoten rotiert werden.
  – 4 bleibt verletzt, 2 ist nicht verletzt.
- Fall 3: Onkel schwarz, gerader Weg zu Opa
  – 5 bleibt durch Umfärben und Rotation erhalten: Nach Umfärben oder Rotation ist die Wurzel des Opa-Teilbaumes schwarz und beide Kinder sind rot.
  – Daher ist 4 nicht mehr verletzt, parent[z] schwarz und die Schleife wird verlassen.

Terminierung (z.z.: nach Terminierung der While-Schleife sichern i, ii, iii die Korrektheit von RBInsertFixup, d.h. alle Eigenschaften sind erfüllt): Die Schleife terminiert, wenn der parent[z] schwarz ist. Damit ist Eigenschaft 4 nicht verletzt. Also kann nur 2 verletzt sein; sollte dem so sein, wird es in der letzten Zeile in Ordnung gebracht. Also gelten alle Eigenschaften bei Terminierung.

Laufzeit:

- RBInsert 1–21 ist $O(\log(n))$
- RBInsertFixup: While kann $\log(n)$ Mal durchlaufen werden und jeder Durchlauf ist konstant
- $O(\log(n))$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Löschen

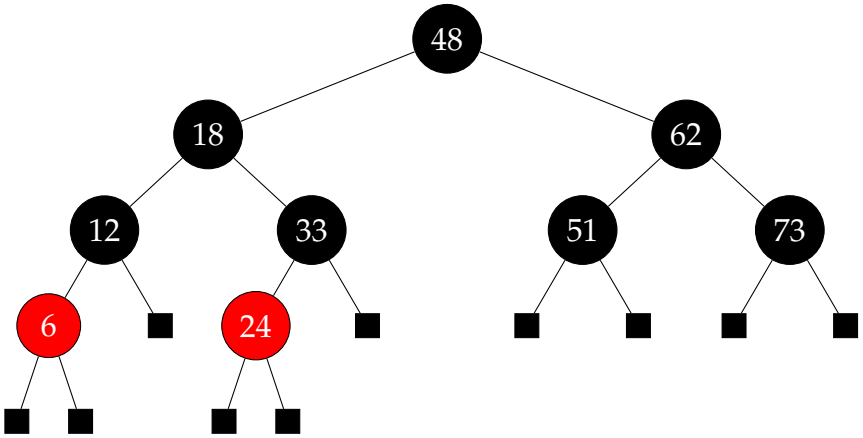
Algorithmus 29

```
    algorithm RBDelete(T, z)
1       Zeilen 1–10 wie TreeDelete (NIL durch nil[T] ersetzen)
2       Zeile 11: parent[x] = parent[y]
3       Zeilen 14–26 von TreeDelete
4       if color[y] = schwarz then
5           RBDeleteFixup(T, x)
6       end if
7       return y
    end algorithm
```

Bemerkung 22 Bemerkungen zu RBDelete:

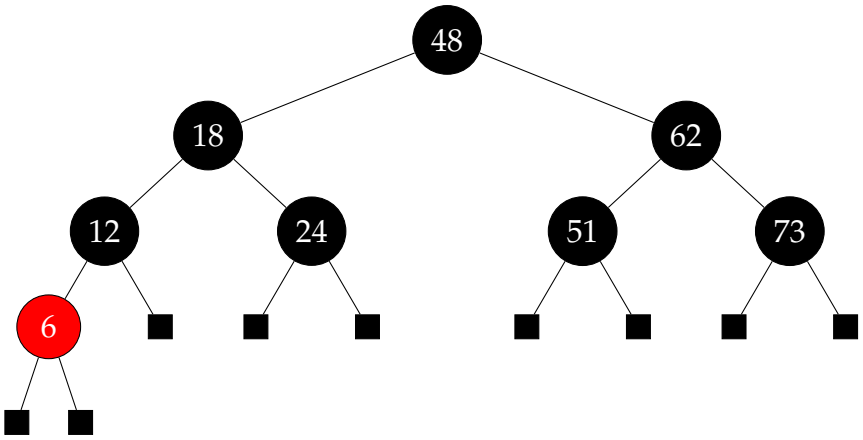
- Zeile 11: Zuweisung ohne IF x != NIL
- Erinnerung an TreeDelete:
  - z zu löschender Knoten
  - y Knoten, der tatsächlich entfernt wird (hat nur ein Kind oder nil[T] als Kind)
  - x ist das eine Kind
  - y nimmt Platz von z ein
  - x nimmt Platz von y ein
- Falls der gelöschte Knoten rot ist, gelten alle RS-Eigenschaften.
- Der gelöschte Knoten y ist schwarz und hat höchstens ein Kind: y wird herausgetrennt und sein Kind an RBDeleteFixup übergeben.

Beispiel 30 Wir betrachten den folgenden Rot-Schwarz-Baum:

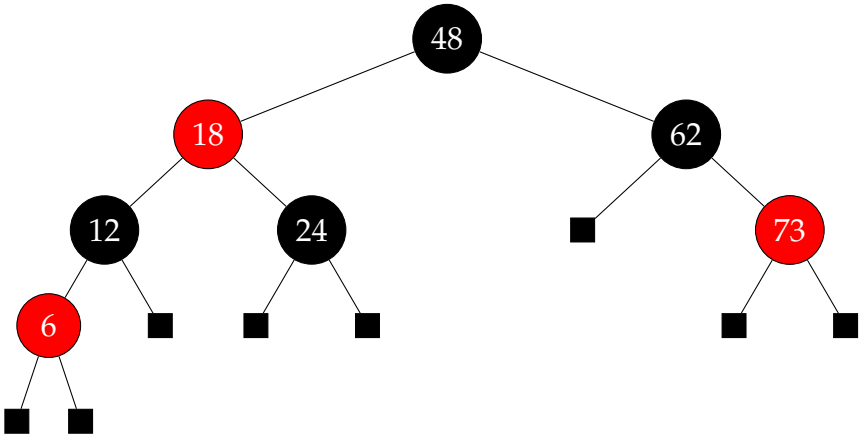


Löschen von 33: Schwarz der 33 wird an rote 24 übergeben

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Löschen von 51: Schwarz der 51 wird an das schwarze nil$[T]$ übergeben



Bemerkung 23 RS-Eigenschaften:

- 1, 3 nicht beeinflusst
- 2: y könnte Wurzel sein, also könnte x rot sein und neue Wurzel
- 4: Problem falls x und parent$[y]$ beide rot
- 5: Jeder Pfad, der y enthält, hat nach dem Löschen einen schwzaren Knoten weniger

Lösungsvorschlag: Wenn x rot ist, übernimmt es die Farbe des schwarzen y, und wenn x ist schwarz ist, übernimmt es auch die Farbe des schwarzen y (zählt jetzt doppelt schwarz). Dieses überschüssige Schwarz wird nach oben gereicht.

Algorithmus 30

algorithm RBDeleteFixup(T, x)
1 **while** $x \neq$ root$[T]$ **and** color$[x]$ = schwarz **do**
2 **if** x = left[parent$[x]$] **then**
3 w = right[parent$[x]$] ▷ *w ist Bruder von x*
4 **if** color$[w]$ = rot **then** ▷ *1. Fall: roter Bruder*

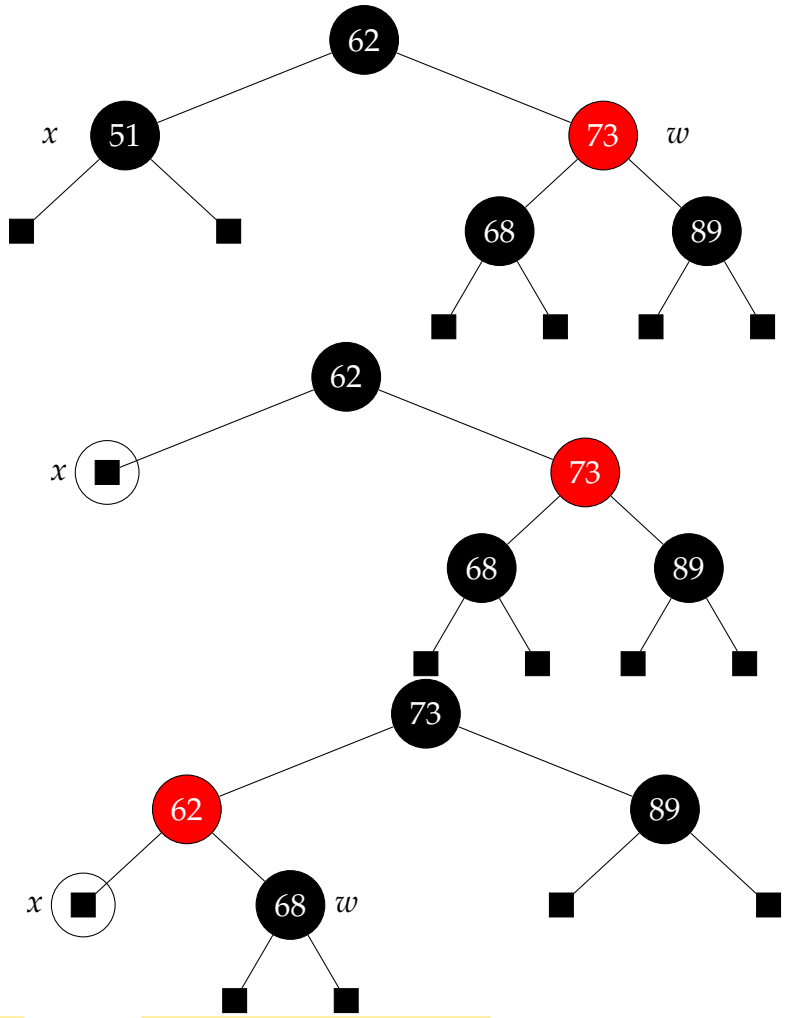
Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
5                   color[w] = schwarz
6                   color[parent[x]] = rot
7                   LeftRotate(T, parent[x])
8                   w = right[parent[x]]
9               end if
10              if color[left[w]] = schwarz and color[right[w]] = schwarz
                then                                              ▷ 2. Fall
11                  color[w] = rot
12                  x = parent[x]
13              else
14                  if color[right[w]] = schwarz then             ▷ 3. Fall
15                      color[left[w]] = schwarz
16                      color[w] = rot
17                      RightRotate(T, w)
18                      w = right[parent[x]]
19                  end if
20                  ▷ 4. Fall
21                  color[w] = color[parent[x]]
22                  color[parent[x]] = schwarz
23                  color[right[w]] = schwarz
24                  LeftRotate(T, parent[x])
25                  x = root[T]
26              end if
27          else
28              ▷ wie oberer Verzweigungsteil nur mit left und right vertauscht
29          end if
30      end while
31      color[x] = schwarz
    end algorithm
```

Bemerkung 24 Zu RBDeleteFixup

- x rot: While wird nicht ausgeführt, Zeile 31 ausgeführt, x wird schwarz, fertig
- ab jetzt x schwarz, Vorstellung schwarz schwarz, damit 5 nicht verletzt, w ist der Bruder
  - Fall 1: w ist rot. w und parent$[x]$ umfärben und Left-Rotation um parent$[x]$, neues w

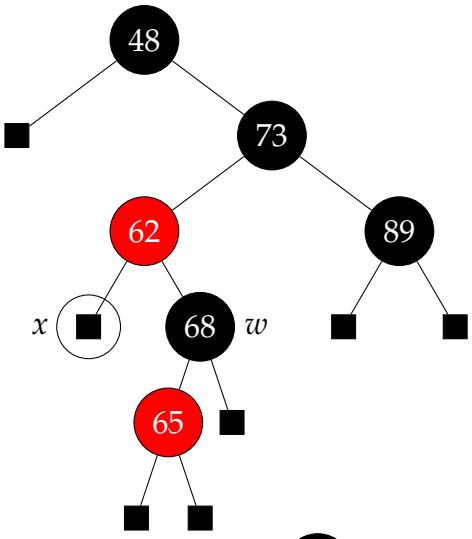
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

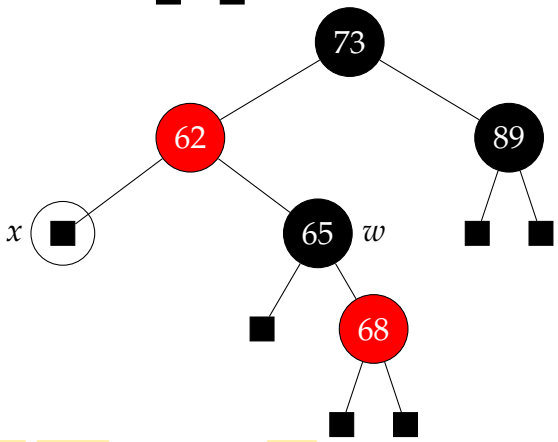


Fall 1 wird in Fall 2, 3 oder 4 überführt.

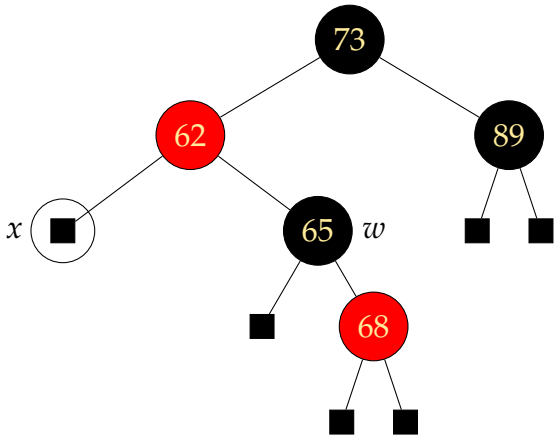
- Fall 2: beide Kinder von w sind schwarz: w rot färben, $x = \text{parent}[x]$, x könnte auch schwarz sein, neues x wieder schwarz schwarz, weiter
- Fall 3: rechtes Kind von w schwarz, linkes rot: Farbe von w und left$[w]$ tauschen, Rechtsrotation um w, w neu setzen, Transformation zu Fall 4 (Eigenschaft 5 bleibt erfüllt)

Downloaded by Maja Müller (blackcatgirl@freenet.de)

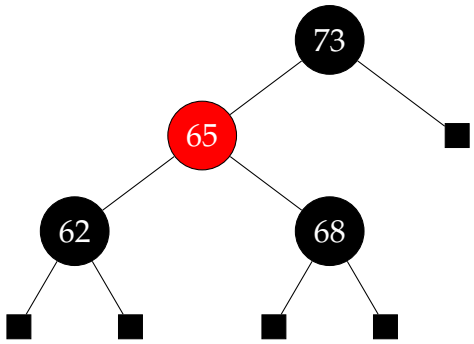




– Fall 4: rechtes Kind von w ist rot, linkes egal: w erhält Farbe von parent$[x]$, parent$[x]$ schwarz, right$[w]$ schwarz, Linksrotation um parent$[x]$, zusätzliches schwarz wurde durch Umfärben und Rotation versorgt, durch $x = \text{root}[T]$ terminiert die Schleife



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Laufzeit:

- RBDELETE: $O(\log(n))$
- RBDELETEFIXUP:
  - außer Fall 2 nur Farbänderung und höchstens 3 Rotationen, also konstant
  - Fall 2: keine Rotation, pro Durchlauf x eine Etage nach oben, also $O(\log(n))$

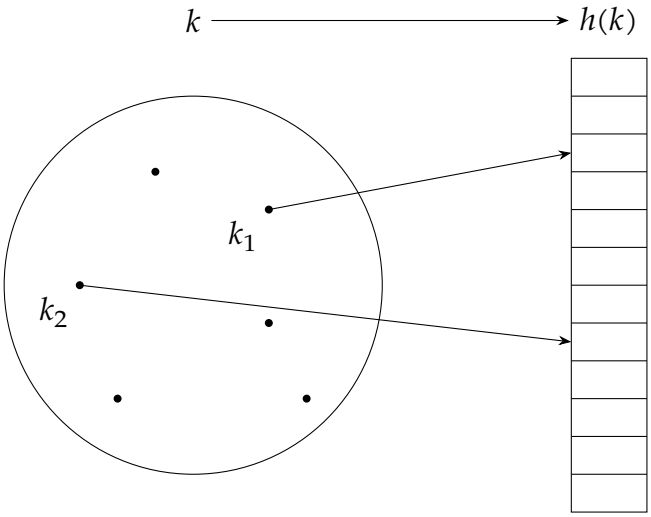
Satz 9 RBINSERT und RBDELETE benötigen $O(\log(n))$ Zeit.

5 Hashing

- Gesucht ist eine effiziente Datenstruktur zur Implementierung des abstrakten Datentyps DICTIONARY ({INSERT, DELETE, SEARCH}).
- Verallgemeinerung von Arrays für folgende Situation:
  - Es gibt sehr viele mögliche Schlüssel.
  - Zu jedem Zeitpunkt werden nur wenige davon verwaltet.
- direkte Adressierung: Jeder Index entspricht einem möglichem Schlüssel. $T[0...l-1]$ mit $T[k]$ zeigt auf das Element mit Schlüssel k.
- Hashing: $k \to h(k)$. Die Tabellenlänge ist proportional zur Anzahl der gespeicherten Schlüssel.
- $\mathcal{U} = \{0, 1, ..., l-1\}$ ist das Universum der Schlüssel.
- $h : \mathcal{U} \to \{0, ..., m-1\}$. Mithilfe einer Hashfunktion wird der Platz für Schlüssel k berechnet.
- Problem: Wenn $m \ll |\mathcal{U}|$ ist h nicht injektiv, d.h. es gibt (viele) Paare $k_1, k_2 \in \mathcal{U}$ mit $k_1 \neq k_2$, wobei $h(k_1) = h(k_2)$ (Kollision). Zwei Lösungen:
  - Hashing mit Verkettung
  - offene Adressierung

Downloaded by Maja Müller (blackcatgirl@freenet.de)

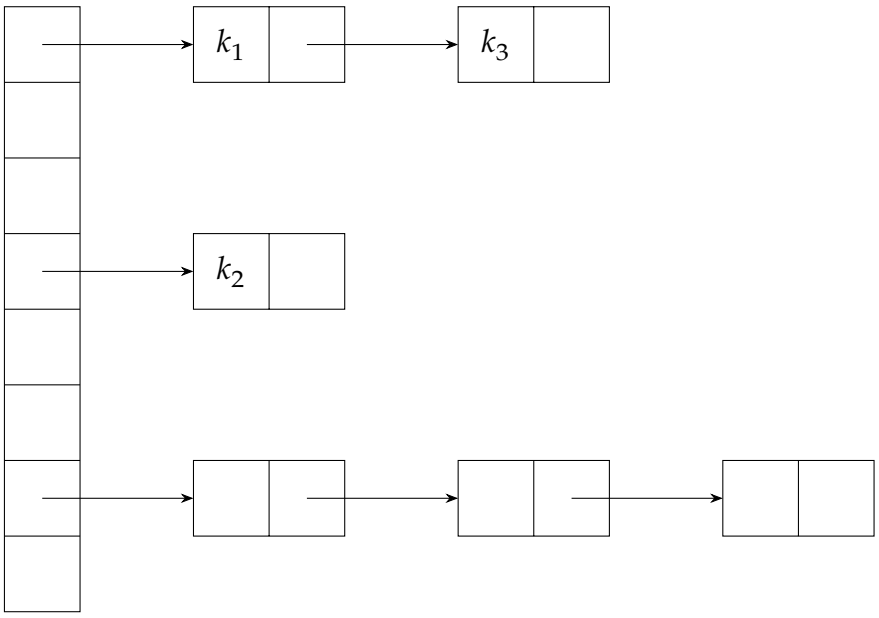
Abbildung 14 Grundprinzip des Hashings



5.1 Hashing mit Verkettung

In jeder Feldposition wird eine verkettete Liste angelegt, die alle Schlüssel speichert, die auf diesen Platz abgebildet werden.

Abbildung 15 Verkettete Hash-Tabelle



Algorithmus 31

algorithm ChainedHashInsert(T, x)
1 Füge x an den Kopf der Liste $T[h(\text{key}[x])]$ ein.
end algorithm

Algorithmus 32

algorithm ChainedHashSearch(T, k)

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

1 Suche in der Liset $T[h(k)]$ nach einem Element mit dem Schlüssel k.
end algorithm

Algorithmus 33

algorithm ChainedHashDelete(T, x)
1 Entferne x aus der Liste $T[h(\text{key}[x])]$.
end algorithm

Analyse:

- Insert: $O(1)$
- Search, Delete:
  - Worst-Case: $O(n)$ mit $n = |K|$
  - Average mit der Tabellengröße m und der Zahl der abgespeicherten Elemente n. Der Auslastungsfaktor ist $\alpha = \frac{n}{m}$.
  - Laufzeit hängt davon ab, wie gut die Hashfunktion die Schlüssel auf die m Plätze verteilt.

Annahme des einfachen uniformen Hashings: Jedes Element wird unabhängig von den bisher einsortierten Elementen mit gleicher Wahrscheinlichkeit auf jeden der m Plätze abgebildet.

Satz 10 Unter der Annahme des einfachen uniformen Hashings benötigt ChainedHashSearch im Erwartungswert $O(1 + \alpha)$ Schritte.

Bemerkung 25 Falls $n \in O(m)$, ist $\alpha \in O(1)$. Dann dauert Search wie Delete $O(1)$.

5.2 Hashfunktionen

- Hashfunktionen erfüllen möglichst die Annahme des einfachen uniformen Hashings.
- Beispiel: $h(k) = \lfloor k \cdot m \rfloor$ (falls Schlüssel in $(0, 1)$ gleichverteilt.
- Für die meisten Hashfunktionen ist $\mathcal{U} \subseteq \mathbb{N}$ nötig (Schlüssel müssen als natürliche Zahlen interpretierbar sein, Zeichenkette entspricht einer natürlichen Zahl zu einer beliebigen Basis).

Verschiedene Hashfunktionen

Divisionsmethode:

$$h(k) = k \bmod m$$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- schnell, sicher zu berechnen
- m keine 2er-Potenz
- günstig: m Primzahl, die nicht zu dicht an einer 2er-Potenz liegt

Multiplikationsmethode mit $a \in (0,1)$ als Konstante:

$$h(k) = \lfloor m \cdot (k \cdot a - \lfloor k \cdot a \rfloor) \rfloor$$

- Die Wahl von m ist nun unkritisch.
- Meistens wählt man $m = 2^p$.
- Man erhält einen Wert zwischen 0 und $m - 1$.

Universelles Hashing

Ziel: Hashfunktionen zufällig wählen (aus sorgfältig konstruierter Klasse). Das ist unabhängig von den Schlüsseln, die tatsächlich gespeichert werden.

Die Menge der Hashfunktionen ergibt sich als

$$\mathcal{H} \subseteq \{h \mid \mathcal{U} \to \{0, ..., m-1\}\}.$$

Beispiel: Sei p eine Primzahl:

- $h_{a,b}(k) = ((a \cdot k + b) \bmod p) \bmod m$ für $a \in \{1, 2, ..., p-1\}$ und $b \in \{0, 1, ..., p-1\}$.
- $\mathcal{H}_p = \{h_{a,b} \mid a \in \{1, 2, ..., p-1\}, b \in \{0, 1, ..., p-1\}\}$

Definition 13 Sei $\mathcal{H} = \{h \mid \mathcal{U} \to \{0, ..., m-1\}\}$ eine endliche Menge von Hashfunktionen. $\mathcal{H}$ heißt *universell*, wenn für alle $k, l \in \mathcal{U}$ mit $k \neq l$ gilt: Die Zahl der Funktionen h, mit $h(k) = h(l)$ ist höchstens $\frac{|\mathcal{H}|}{m}$.

Bemerkung 26 Die Wahrscheinlichkeit einer Kollision zwischen den Schlüsseln k und l ist nicht höher als $\frac{1}{m}$.

Satz 11 Es werde h aus einer universellen Menge von Hashfunktionen ausgewählt, um n Schlüssel in einer Tabelle der Länge n zu speichern. Dabei werden Kollisionen mit Verkettung aufgelöst. Dann ist die erwartete Länge der Liste, in die Schlüssel k abgebildet wird, höchstens α, falls k nicht in der Tabelle enthalten ist und höchstens $1 + \alpha$, falls k in der Tabelle enthalten ist.

5.3 Offene Adressierung

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Alle Elemente werden in der Hashtabelle gespeichert.
- Es wird in der Tabelle sukzessive nach einem freien Platz für das einzufügende Element gesucht (Sondieren).
- $h : \underbrace{\mathcal{U}}_{\text{Schlüssel}} \times \underbrace{\{0, 1, ..., m-1\}}_{\text{Versuchsnummer}} \to \{0, 1, ..., m-1\}$ mit dem Universum $\mathcal{U}$ und der Größe der Tabelle m
- Forderung: Für alle $k \in \mathcal{U}$ ist die Folge $h(k, 0), h(k, 1), ..., h(k, m-1)$ eine Permutation von $0, 1, ..., m-1$. Diese Forderung impliziert: Wenn es einen freien Platz gibt, wird er auch gefunden.

Algorithmus 34

```
algorithm HashInsert(T, k)
1     i = 0
2     repeat
3         j = h(k, i)
4         if T[j] = NIL then
5             T[j] = k
6             return j
7         else
8             i = i + 1
9         end if
10    until i = m
11    Error „Überlauf“
end algorithm
```

Algorithmus 35 HashSearch: Sondiert die gleiche Reihenfolge wie HashInsert

```
algorithm HashSearch(T, k)
1     i = 0
2     repeat
3         j = h(k, i)
4         if T[j] = k then
5             return j
6         end if
7         i = i + 1
8     until i = m or T[j] = NIL
end algorithm
```

Algorithmus 36 HashDelete:

- Nach dem Löschen kann nicht einfach NIL geschrieben werden, sondern es muss „DELETED“ eingefügt werden, damit Suchen bzw. Einfügen ohne Probleme weiterlaufen.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- HashInsert Zeile 4 ersetzen durch

  IF T[j] = NIL OR T[j] = DELETED THEN T[j] = k

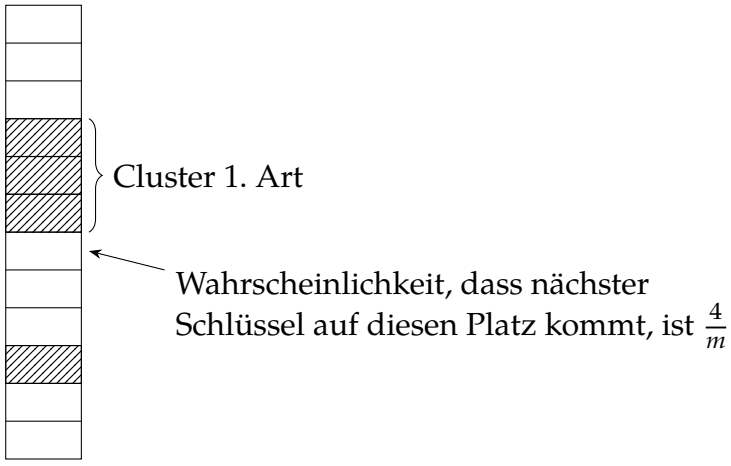
Fragestellung: Wie wählt man eine Hashfunktion, deren Sondierungsfolgen schnell freie Plätze finden?

Lineares Sondieren

Sei $h' : \mathcal{U} \to \{0, 1, ..., m-1\}$ eine Hashfunktion. Wir wählen $h(k, i) = (h'(k) + i) \bmod m$ für $i = 0, 1, ..., m-1$.

- Die Plätze werden in der Reihenfolge $T(h'(k)), T(h'(k)+1), ..., T(h'(k)+m-1)$ überprüft.
- Die Funktion ist leicht zu implementieren.
- Es bilden sich langsam Folgen belegter Tabellenplätze, Cluster 1. Art.

Abbildung 16 Lineare Sondierung



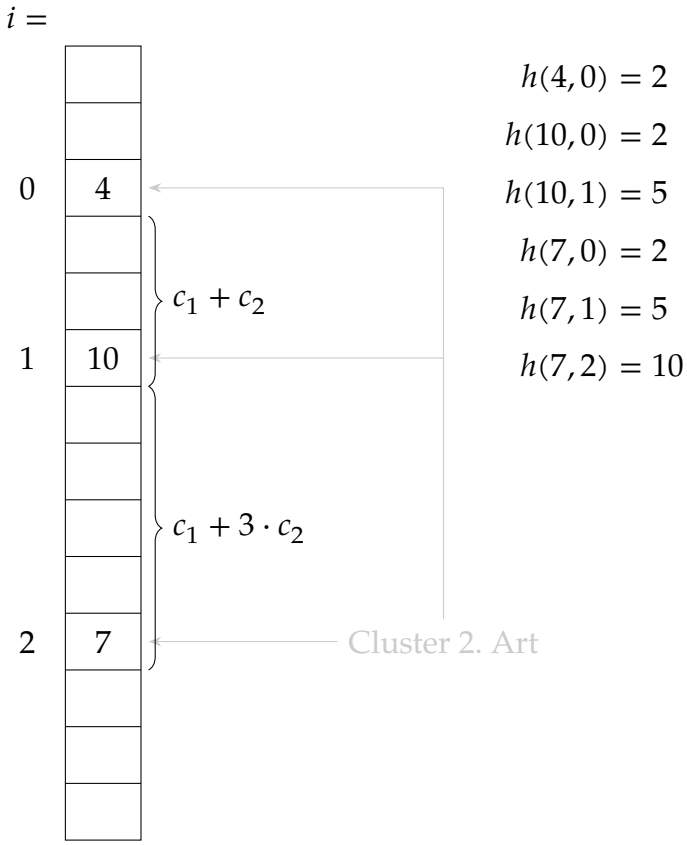
Quadratisches Sondieren

Sei $h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$ eine Hashfunktion mit c_1, c_2 konstant und $c_2 \neq 0$.

- Diese Methode ist einfach zu implementieren.
- Diese Methode ist besser als lineares Sondieren.
- c_1, c_2 müssen sorgfältig gewählt werden, um die ganze Tabelle zu durchsuchen.
- Für $k_1, k_2 \in \mathcal{U}$ mit $h'(k_1) = h'(k_2)$ sind die Sondierungsfolgen auch gleich, Cluster 2. Art.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 17 Quadratische Sondierung



Doppeltes Hashing

Seien h_1, h_2 einstellige Hashfunktionen mit $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Die Sondierungsfolge hängt vom Schlüssel k ab, wobei $h_1(k)$ die Startposition ist und $h_2(k)$ die Schrittweite.

- $h_2(k)$ und m sollten stets teilerfremd sein (nur dann ist die Sondierungsfolge eine Permutation von $0, 1, ..., m-1$)
- einfache Möglichkeit: $m = 2^t$ und $h_2(k)$ immer ungerade; alternativ m prim und $0 < h_2(k) < m$. Beispiel:
  - $h_1(k) = k \bmod m$ mit m als Primzahl
  - $h_2(k) = 1 + k \bmod m'$ mit $m' = m - 1$

Um die Qualität eines Hashverfahrens zu beurteilen, kann man die Zahl verschiedener Sondierungsfolgen bestimmen (siehe Tabelle 1).

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Tabelle 1 Sondierungsfolgen verschiedener Hashverfahren

lineares Sondieren	**quadratisches Sondieren**	**doppeltes Hashing**
$h'(k) + i$	$h'(k)$ (Startwert)	$h_1(k)$ (Startwert)
$0, 1, 2, \ldots$ $1, 2, 3, \ldots$ $\vdots$ $m-1, 0, 1, \ldots$	definiert die Folge vollständig	für jeden Startwert m verschiedene Folgen
m Folgen	m Folgen	m^2 Folgen

Satz 12 Für eine Hashtabelle basierend auf doppeltem Hashing mit Auslastungsfaktor $\alpha = \frac{n}{m} < 1$ ist die erwartete Anzahl der inspizierten Zellen höchstens $\frac{1}{1-\alpha}$ bei erfolgloser Suche und höchstens $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ bei erfolgreicher Suche, wenn man uniformes Hashing voraussetzt.

6 Binomialheaps

Implementierung einer Priority Queue mit folgenden Operationen:

- Make-Heap: erzeugt einen Heap, der kein Element enthält, und gibt ihn zurück
- Insert(H, x): fügt x in H ein
- Minimum(H): gibt den Zeiger auf den Knoten zurück, der das Minimum enthält
- Extract-Min(H): gibt das Minimum zurück und stellt die Heapeigenschaft für den Heap ohne das Minimum wieder her
- Union(H_1, H_2): erzeugt einen neuen Heap, der alle Knoten aus H_1 und H_2 enthält
- Decrease-Key(H, x, k): verringert den Schlüssel von x auf k
- Delete(H, x): entfernt x aus H

6.1 Binomialbaum

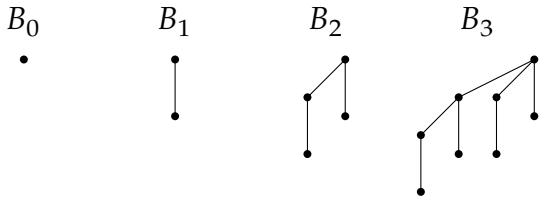
Definition 14 Ein Baum B heißt *geordnet*, wenn für die Kinder eines jeden Knotens eine Reihenfolge festgelegt ist.

Bemerkung 27 Binäre Bäume sind geordnet.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Definition 15 *Binomialbäume* sind rekursiv definierte geordnete Bäume. Dabei ist B_0 ein einzelner Knoten. B_k besteht aus zwei Bäumen B_{k-1}, die in folgender Weise verkettet werden: Die Wurzel des einen wird das linke Kind der Wurzel des anderen.

Abbildung 18 Binomialbäume für $k = 0, \ldots, 3$



Satz 13 Für den Binomialbaum B_k gilt:

i. B_k hat 2^k Knoten.
ii. Die Höhe von B_k ist k.
iii. B_k enthält genau $\binom{k}{i}$ Knoten der Tiefe i $(i = 0, \ldots, k)$.
iv. Der Grad der Wurzel von B_k ist k. Es seien $x_{k-1}, \ldots, x_0$ die Kinder der Wurzel (von links nach rechts). Dann ist der Teilbaum mit Wurzel x_i ein B_i.

Beweis per vollständiger Induktion. Induktionsanfang: Alle Eigenschaften gelten für B_0. Induktionsschritt für k nach $k+1$:

i. $n(B_{k+1}) = 2 \cdot n(B_k) \overset{\text{IV}}{=} 2 \cdot 2^k = 2^{k+1}$
ii. $h(B_{k+1}) = h(B_k) + 1 \overset{\text{IV}}{=} k + 1$
iii. Sei n_i die Anzahl der Knoten in Tiefe i. $n_i(B_{k+1}) = n_i(B_k) + n_{i-1}(B_k) \overset{\text{IV}}{=} \binom{k}{i} + \binom{k}{i-1} = \binom{k+1}{i}$
iv. $\deg(\text{root}(B_{k+1})) = 1 + \deg(\text{root}(B_k)) \overset{\text{IV}}{=} 1 + k$. Der linkeste Teilbaum (der 1. Teilbaum) des B_{k+1} ist B_k (gerade angehängt, die anderen nach IV $B_{k-1}, B_{k-1}, \ldots, B_0$). □

Bemerkung 28 Folgerung: Der maximale Grad jedes Knotens in einem Binomialbaum ist $\lfloor \log(n) \rfloor$.

6.2 Binomialheap

Definition 16 Ein *Binomialheap* ist eine Menge von Binomialbäumen mit folgenden Eigenschaften:

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Jeder Binomialbaum ist min-heap-geordnet, d.h. für jeden Knoten gilt, dass ein Schlüssel größer gleich dem Schlüssel eines Vaters ist: $\text{key}(x) \geq \text{key}(\text{parent}(x))$.
- Für jedes $k \in \mathbb{N}$ gibt es höchstens einen B_k.

Bemerkung 29 Der Knoten mit minimalem Schlüssel steht in der Wurzel eines Binomialbaums.

Bemerkung 30 Sei $n = \sum_{i=0}^{\lfloor \log(n) \rfloor} b_i 2^i$, d.h. $\langle b_{\lfloor \log(n) \rfloor} ... b_0 \rangle$ ist Binärdarstellung von n. B_k kommt im Heap vor genau dann, wenn $b_k = 1$.

Darstellung von Binomialheaps

- Wurzeln sind als verkette Liste organisiert (Wurzelliste).
- Die Wurzelliste ist nach dem Grad der einzelnen Bäume sortiert.
- head$[H]$ ist der Zeiger auf die erste Wurzel in der Liste.
- Die einzelnen Bäume sind in „linkes-Kind-rechter-Bruder-Darstellung“ gespeichert.

Operationen auf Binomialheaps

Finden des minimalen Elements:

- Das Minimum steht in einer Wurzel, also muss die Wurzelliste durchsucht werden.
- Laufzeit: $O(\log(n))$ (Länge der Liste)

Vereinigung:

Algorithmus 37 Hilfsmittel BinomialLink in $O(1)$

algorithm BinomialLink(y, z)
1 parent$[y] = z$
2 brother$[y]$ = child$[z]$
3 child$[z] = y$
4 deg$[z]$ = deg$[z] + 1$
end algorithm

Algorithmus 38 BinomialHeapUnion: Binomialbäume der Heaps H_1 und H_2 müssen verkettet werden zum Binomialbaum eines Heaps mit $n_1 + n_2$ Knoten.

algorithm BinomialHeapUnion(H_1, H_2)

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

1 Verkette die Wurzellisten von H_1 und H_2 zu einer Wurzelliste H, die aufsteigend nach dem Grad der Bäume sortiert ist.

2 Durchlaufe H und verkette die Bäume von gleichem Grad in folgender Weise:

- ⋆ Fall 1: vom Grad i gibt es genau 1 Baum: erhalte ihn.
- ⋆ Fall 2: vom Grad i gibt es genau 2 Bäume: verlinke beide, wobei der Knoten mit der kleineren Wurzel die neue Wurzel wird.
- ⋆ Fall 3: vom Grad i gibt es genau 3 Bäume: erhalte den ersten, verlinke die beiden anderen, wobei der Knoten mit der kleineren Wurzel die neue Wurzel wird.

end algorithm

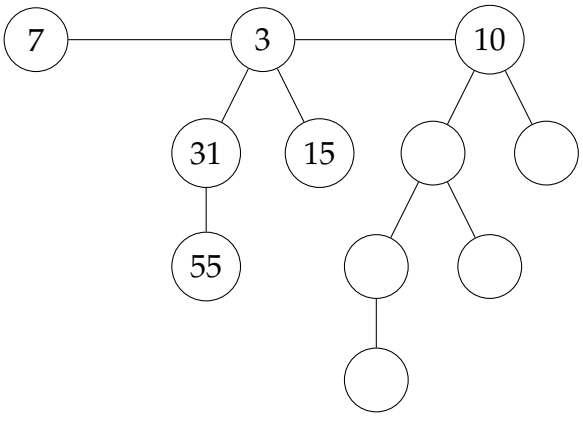
Gesamtlaufzeit: $O(\log(n_1+n_2))$ (Durchlaufen der Wurzelliste mit jeweils konstant vielen Vergleichen und Verkettungen)

Algorithmus 39

algorithm BinomialHeapInsert(H, x)
1 H' = MakeBinomialHeap(H, x)
2 parent[x] = NIL
3 child[x] = NIL
4 brother[x] = NIL
5 deg[x] = 0
6 head[H'] = x
7 H = BinomialHeapUnion(H, H')
end algorithm

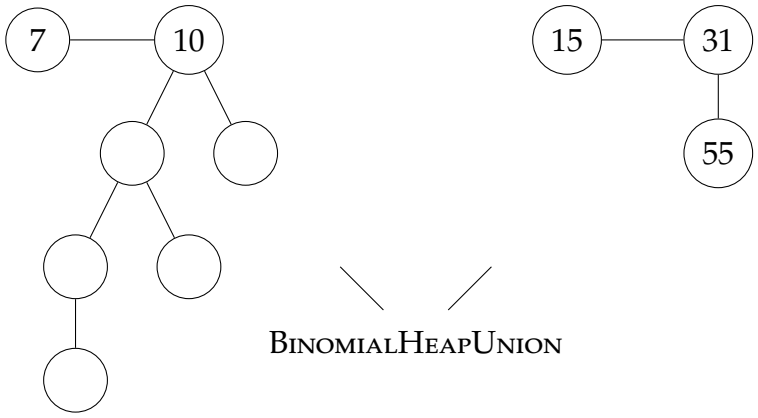
Laufzeit: $O(\log(n))$

Minimum löschen:



Idee: Kinder des Wurzelknotens, der das Minimum enthält, zu neuem Heap verketten und diesen Heap mit dem bestehenden Heap (ohne den Baum, der das Minimum enthält) vereinigen.

Downloaded by Maja Müller (blackcatgirl@freenet.de)



Algorithmus 40 **algorithm** BinomialHeapExtractMin(H)

1 Finde Wurzel x mit minimalem Schlüssel in der Wurzelliste von H und entferne x aus der Wurzelliste.
2 $H' =$ MakeBinomialHeap(x)
3 Verkette die Kinder von x in umgekehrter Reihenfolge. Setze den Vater jedes Kindes auf NIL. Setze head[H'] auf den Kopf der resultierenden Liste.
4 $H =$ BinomialHeapUnion(H, H')
5 **return** x

end algorithm

Schlüssel herabsetzen:

- wie in binären Heaps: neuen Schlüssel l einfach nach oben tauschen
- $O(\log(n))$ Zeit, da maximale Höhe eines Binomialheaps $O(\log(n))$

Algorithmus 41 **algorithm** BinomialHeapDecreaseKey(H, x, k)

1 **if** $k >$ key[x] **then**
2 Error
3 **end if**
4 key[x] $= k$
5 $y = x$
6 $z =$ parent[y]
7 **while** $z \neq$ NIL **and** key[y] $<$ key[z] **do**
8 swap(key[y], key[z])
9 ▷ *auch Satellitendaten tauschen*
10 $y = z$
11 $z =$ parent[y]
12 **end while**

end algorithm

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Löschen eines Knotens auch wie im binären Heap

Algorithmus 42

```
algorithm BinomialHeapDelete(H, x)
    BinomialHeapDecreaseKey(H, x, −∞)
    BinomialHeapExtractMin(H)
end algorithm
```

Laufzeit: $O(\log(n))$

7 Amortisierte Kostenanalyse

- Wir analysieren eine ganze Folge von Datenstrukturoperationen.
- Einige dieser Operationen sind teuer, andere nicht.
- Es handelt sich stets um eine Worst-Case-Analyse.
- Es wird keine Wahrscheinlichkeit einbezogen.

7.1 Einführende Beispiele

Stapeloperationen

- Push(S, x) legt x auf S.
- Pop(S) entnimmt das oberste Element und gibt es zurück.

Wir führen ferner ein:

Algorithmus 43

```
algorithm StackMultiPop(S, k)
    while not StackEmpty(S) do
        StackPop(S)
        k = k − 1
    end while
end algorithm
```

Laufzeit von Multipop auf Stapel mit s Elementen:

$$c \cdot \min\{s, k\} \rightarrow \text{im Wort-Case gleich Stapelgröße } s, \text{ also } O(n)$$

Eine beliebige Folge von n Push-, Pop-, MultiPop-Operationen benötigt im Worst-Case $O(n^2)$ Zeit. Es kann aber nicht jede Operation ein MultiPop mit $O(n)$ Laufzeit sein.

Idee:

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Jedes Push erhält 2 Geldstücke.
- Von 1 Geldstück wird die aktuelle Push-Operation bezahlt.
- Das andere Geldstück erhält das im Stapel befindliche Objekt als Guthaben.
- Wird Pop aufgerufen, wird es mit dem hinterlegten Geldstück gezahlt.
- Auch ein MultiPop kann von dem hinterlegten Geld gezahlt werden.

Daraus ergibt sich, dass eine Folge von n Operationen höchstens $2n$ Kosteneinheiten (Geldstücke) verbraucht.

- Die amortisierten Kosten der Folge sind die Summe der mitgebrachten Kosteneinheiten.
- Die tatsächlichen Kosten übersteigen die amortisierten Kosten nicht, da das Guthaben im Stapel nie negativ wird.

Damit sind die amortisierten Kosten eine obere Schranke für die tatsächlichen. Uns genügt es also, die amortisierten Kosten zu bestimmen.

Im Beispiel sind es $2 \cdot n$ (mit n als Anzahl der Operationen). Es ergibt sich die folgende Aufstellung:

- Push amortisiert 2 KE
- Pop amortisiert 0 KE
- MultiPop amortisiert 0 KE

Binärzähler

- k-Bit-Binärzähler, der von 0 an aufwärts zählt
- Feld $A[0...k-1]$ von Bits
- höchstwertiges Bit in $A[k-1]$: $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$

Algorithmus 44

```
algorithm BinaryIncrement(A)
1     i = 0
2     while i < |A| and A[i] = 1 do
3         A[i] = 0
4         i = i + 1
5     end while
6     if i < |A| then
7         A[i] = 1
8     end if
end algorithm
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

	$A[3]$	$A[2]$	$A[1]$	$A[0]$	
$\cdots$	0	0	0	0	
$\cdots$	0	0	0	1	2 KE
$\cdots$	0	0	1	0	2 KE
$\cdots$	0	0	1	1	2 KE
$\cdots$	0	1	0	0	2 KE
$\cdots$	0	1	0	1	
$\cdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\cdots$	1	1	1	1	

Die Kosten jeder Increment-Operation sind linear in der Anzahl der umgeklappten Bits. Manche gehen schnell, bei anderen müssen viele Bits gekippt werden.

Oberflächliche Analyse:

- Einmaliges Ausführen von Increment $O(k)$, für eine Folge von n Operationen $O(n \cdot k)$
- Diese Schranke ist korrekt, aber nicht scharf, weil nicht alle Operationen so lange dauern können.

Idee:

- Flip von 0 auf 1: 2 Kosteneinheiten (eine für den Flip und eine als Guthaben)
- Jede 1 in der Struktur hat ein Geldstück als Guthaben, davon kann der Flip zu 0 bezahlt werden.
- Bei jedem Increment wird genau 1 Bit auf 1 gesetzt. Also verursacht jedes Increment 2 KE amortisierte Kosten.
- Diese sind eine obere Schranke für die tatsächlichen Kosten, da die Summe über alle Guthaben in der Struktur nie negativ wird.

7.2 Potentialmethode

- Das Guthaben gehört nicht zu den einzelnen Objekten in der Datenstruktur, sondern die Struktur hat als ganzes ein Potential.
- Davon können teure Operationen bezahlt werden.
- Andere Operationen zahlen ein.

Sei Φ eine Abbildung, die einem Zustand der Datenstruktur D eine reelle Zahl zuordnet, und D_i der Zustand nach der i-ten Operation O_i.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

D_0 ist der Zustand vor der ersten Operation. $\hat{c}_i$ bezeichnet die amortiserten Kosten von O_i.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Die amortisierten Kosten der n Operationen betragen

$$\begin{aligned}\sum_{i=1}^{n} \hat{c}_i &= \sum_{i=1}^{n} c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= \Phi(D_n) - \Phi(D_0) + \sum_{i=1}^{n} c_i.\end{aligned}$$

Kann man eine Potentialfunktion Φ finden, sodass $\Phi(D_n) \geq \Phi(D_0)$ gilt, dann sind die amortisierten Kosten eine obere Schranke für die tatsächlichen Kosten.

Dies ist erfüllt, wenn $\forall i \in \{1, ..., n\} : \Phi(D_i) \geq \Phi(D_0)$. Bequem ist $\Phi(D_0) = 0$ und $\forall i : \Phi(D_i) \geq 0$.

Stapeloperationen

Sei s die Anzahl der Elemente im Stack. Φ bildet auf s ab.

- Push: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + s + 1 - s = 2$
- Pop: $\hat{c}_i = 1 + s - 1 - s = 0$
- MultiPop: $\hat{c}_i = \hat{c}_i - k' + s - k' - s = 0$ (wobei $k' = \min\{s, k\}$)

Binärzähler

Sei Φ die Anzahl der Bits, die gleich 1 sind. O_i setzt t_i Bits zu 0 und 1 Bit zu 1.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = t_i + 1 + \overbrace{(-t_i + 1)}^{\Phi(D_i) - \Phi(D_{i-1})} = 2$$

Die Potentialfunktion spielt für den Algorithmus keine Rolle, sie ermöglicht nur die amortisierte Analyse.

8 Fibonacci-Heaps

- gleiche Operationen wie auf Binomialheaps (also auch Union)
- Extract-Min und Delete in $O(\log(n))$ amortisierter Zeit

This document is available on studocu

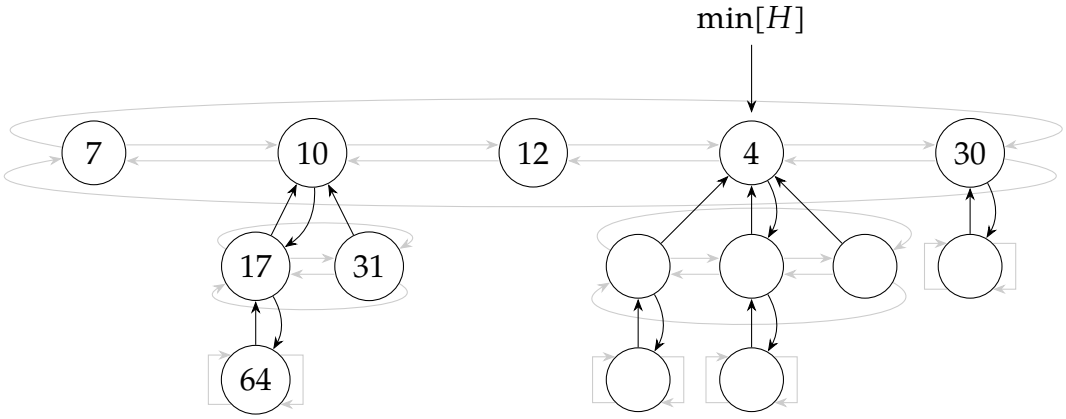
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- alle anderen Opertionen kosten amortisiert $O(1)$
- SEARCH auch hier nicht effizient

Ein Fibonacci-Heap ist

- eine Menge von heapgeordneten Bäumen
- die Bäume sind ungeordnet (unter den Kindern eines Knotens gibt es keine Reihenfolge)
- die Knoten können markiert sein

Abbildung 19 Beispiel für einen Fibonacci-Heap



Darstellung eines Fibonacci-Heaps:

- Die Wurzelliste ist eine doppelt verkettete Ringliste.
- Die Listen der Kinder eines Knotens sind ebenso doppelt verkettete Ringlisten.
- Jeder Knoten hat einen Pointer auf seinen Vater.
- Jeder Knoten hat einen Pointer auf ein beliebiges seiner Kinder.

Für einen Knoten x ist $\deg[x]$ die Anzahl der Kinder von x und $\text{label}[x]$ gibt an, ob x markiert ist, oder nicht.

Für den Fibonacci-Heap H ist $\min[H]$ der Zeiger auf die Wurzel des Baumes, der den minimalen Schlüssel enthält. $n[H]$ ist die Anzahl der aktuell abgespeicherten Knoten.

Potentialfunktion für die amortisierte Analyse der Kosten der einzelnen Operationen: Sei $t(H)$ die Anzahl der Knoten in der Wurzelliste und $m(H)$ die Anzahl markierter Knoten. Die Potentialfunktion ist

$$\Phi(H) = t(H) + 2 \cdot m(H).$$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

8.1 Basis-Operationen auf Fibonacci-Heaps

Heap erstellen und Minimum finden

Algorithmus 45 MakeFHeap

- gibt einen Heap H mit $n[H] = 0$ und $\min[H] = \text{nil}$ zurück
- Es gilt $t(H) = 0, m(H) = 0$ und damit $\Phi(H) = 0$.

$$\hat{c}_i = c_i + \Phi(H') - \Phi(H) = c_i \in O(1)$$

Algorithmus 46 FHeapMinimum

- durch Zeiger $\min[H]$ gegeben
- tatsächliche Kosten $O(1)$ mit $\Delta\Phi = 0$
- amortisierte Kosten $O(1)$

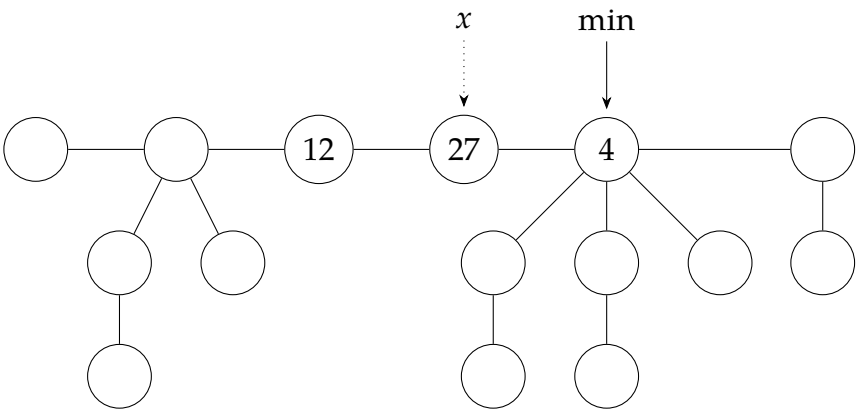
Einfügen

Algorithmus 47

```
algorithm FHeapInsert(H, x)
1    deg[x] = 0
2    parent[x] = NIL
3    child[x] = NIL
4    left[x] = x
5    right[x] = x
6    label[x] = false
7    Verkette die Wurzelliste, die x enthält, mit der Wurzelliste von
     H
8    if min[H] = NIL or key[x] < key[min[H]] then
9        min[H] = x
10   end if
11   n[H] = n[H] + 1
end algorithm
```

Beispiel 31 Links neben $\min[H]$ wird ein neuer Knoten x angefügt.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Bemerkung 31 Nach k Inserts besteht die Wurzelliste aus k isolierten Knoten.

amortisierte Analyse: Sei H der Eingabe-Heap und H' der resultierende Heap. Wir wissen $t(H') = t(H) + 1$ und $m(H') = m(H)$.

$$\begin{aligned}\hat{c_i} &= c_i + \Phi(H') - \Phi(H) \\ &= c_i + t(H') + 2 \cdot m(H') - t(H) - 2 \cdot m(H) \\ &= c_i + 1 + t(H) - t(H) = c_i + 1 \in O(1)\end{aligned}$$

Vereinigen zweier F-Heaps

Algorithmus 48

algorithm FHeapUnion(H_1, H_2)
1 H = MakeFHeap()
2 min[H] = min[H_1]
3 Verkette die Wurzelliste von H_2 mit der Wurzelliste von H
4 **if** min[H_1] = NIL **or** (min[H_2] $\neq$ NIL **and** key[min[H_2]] < key[min[H_1]]) **then**
5 min[H] = min[H_2]
6 **end if**
7 $n[H] = n[H_1] + n[H_2]$
8 Gib die Objekte H_1 und H_2 frei
9 **return** H
end algorithm

Bemerkung 32 Die Vereinigung verkettet die Wurzellisten und bestimmt das neue Minimum. Danach werden H_1 und H_2 zerstört.

amortisierte Analyse: Wir wissen, dass $t(H) = t(H_1) + t(H_2)$ und $m(H) = m(H_1) + m(H_2)$.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$\begin{aligned}\hat{c_i} &= c_i + \Phi(H) - (\Phi(H_1) + \Phi(H_2)) \\ &= c_i + t(H_1) + t(H_2) + 2(m(H_1) + m(H_2)) \\ &\quad - (t(H_1) + 2 \cdot m(H_1) + t(H_2) + 2 \cdot m(H_2)) \\ &= c_i \in O(1)\end{aligned}$$

Entfernen des Minimums

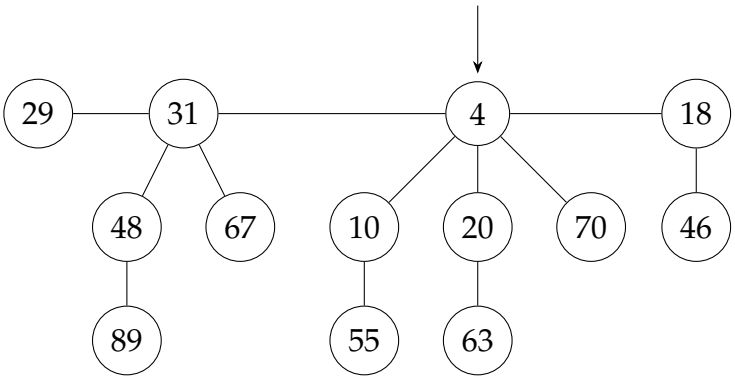
- Bei dieser Operation muss das neue Minimum bestimmt werden.
- Das neue Minimum kann überall in der Wurzelliste stehen, wobei die Wurzelliste lang sein kann.
- Die Wurzelliste muss komplett durchlaufen werden.
- Diese Gelegenheit nutzen wir um „Aufräumen“.
- Gleich große Bäume (bzgl. Knotengrad) werden sukzessive verkettet (ähnlich wie beim Binomialheap).

Algorithmus 49

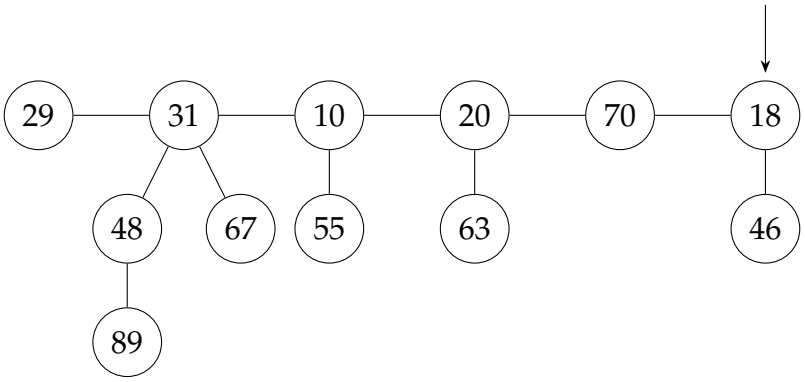
algorithm FHeapExtractMin(H)
1 $z = \min[H]$
2 **if** $z \neq$ NIL **then**
3 **for all** Kind x von z **do**
4 Füge x zur Wurzelliste hinzu
5 parent$[x]$ = NIL
6 ▷ *Markierung bleibt*
7 **end for**
8 Entferne z aus der Wurzelliste
9 **if** $z = \text{right}[z]$ **then** ▷ *z war einziger Knoten im Heap*
10 $\min[H]$ = NIL
11 **else**
12 $\min[H] = \text{right}[z]$
13 FHeapConsolidate(H)
14 **end if**
15 $n[H] = n[H] - 1$
16 **end if**
17 **return** z
end algorithm

Beispiel 32 Wir betrachten den folgenden F-Heap und wollen ExtractMin darauf ausführen.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Die 4 wird extrahiert und es ergibt sich:



Dabei ist 18 nicht notwendigerweise das neue Minimum.

Konsolidierung der Wurzelliste:

- Gleich große Bäume in der Wurzelliste werden verkettet.
- Grundidee: UNION von Binomialheaps, aber folgende Abweichungen:
  - Die Bäume in der Wurzelliste sind nicht nach aufsteigendem Grad sortiert.
  - Auch die Kinder eines Knotens sind nicht nach Grad sortiert.
  - Es gibt unter Umständen sehr viele Bäume vom selben Grad.

Wie findet man schnell Bäume, die verkettet werden können? Wir nutzen ein Hilfsfeld $A[0...D(n[H])]$ mit $D(n[H])$ als dem maximalen Knotengrad des Heaps H mit n Knoten.

Ist $A[i] = y$, so ist y momentan Wurzel mit $\deg[y] = i$. In Feldposition i wird gespeichert, ob bereits ein Baum von Grad i gefunden wurde.

Algorithmus 50 **algorithm** FHEAPLINK(H, y, x)

1 Entferne y aus der Wurzelliste von H

2 Mache y zu einem Kind von x, inkrementiere $\deg[x]$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

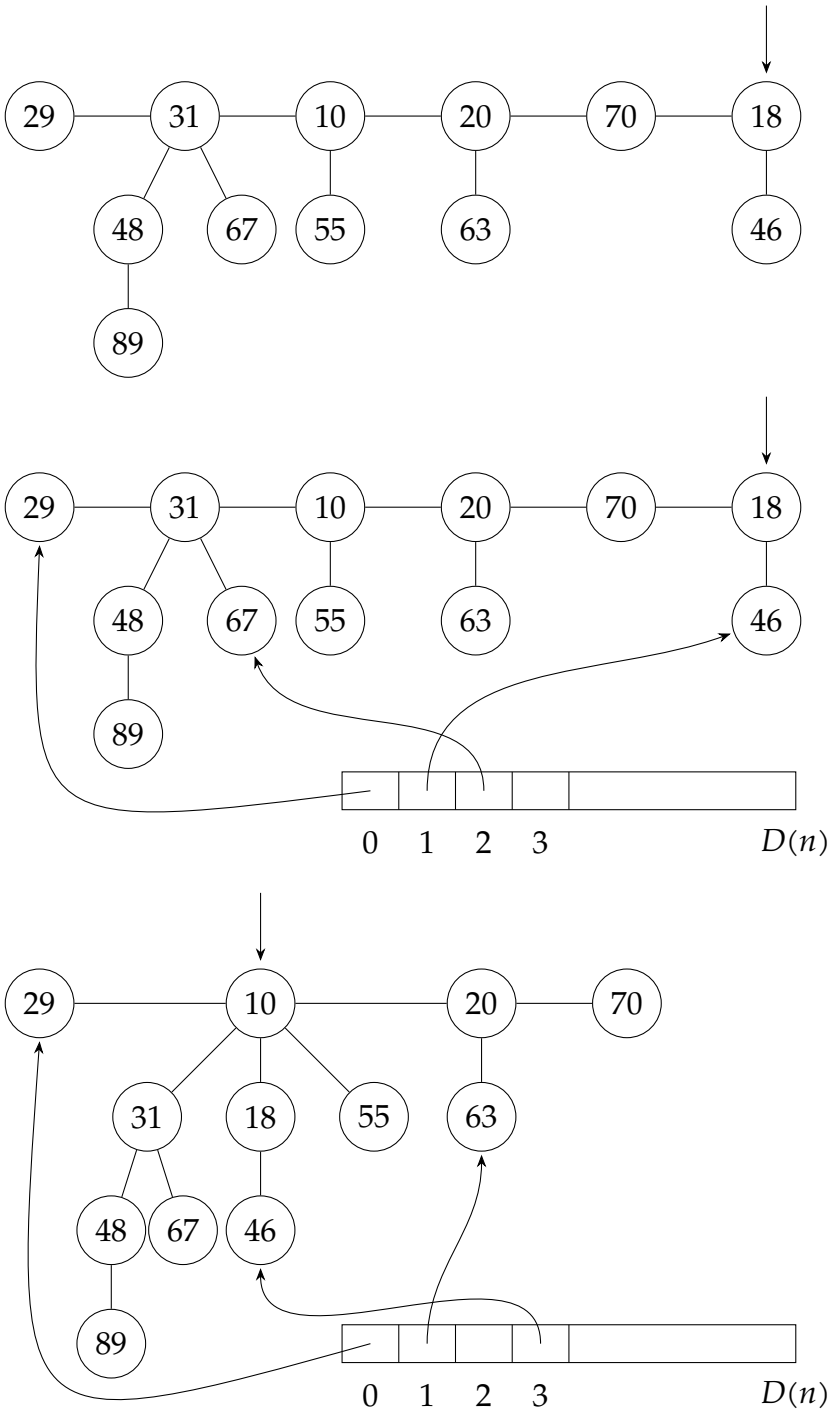
```
3       label[y] = false
4       ▷ Wenn y Kind von x wird, verliert es seine Markierung.
    end algorithm
```

Algorithmus 51

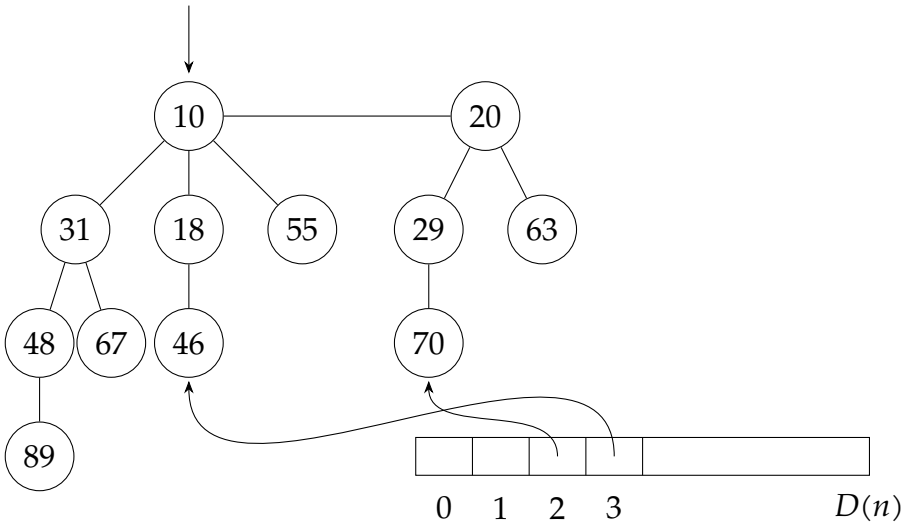
```
    algorithm FHeapConsolidate(H)
1       for i = 0 to D(n[H]) do
2           A[i] = NIL
3       end for
4       for all Knoten w der Wurzelliste von H do
5           x = w
6           d = deg[x]
7           while A[d] ≠ NIL do
8               y = A[d]                ▷ anderer Baum mit selbem Grad wie x
9               if key[x] > key[y] then
10                  swap(x, y)
11              end if
12              FHeapLink(H, y, x)
13              A[d] = NIL
14              d = d + 1
15          end while
16          A[d] = x
17      end for
18      min[H] = NIL
19      for i = 0 to D(n[H]) do
20          if A[i] ≠ NIL then
21              Füge A[i] zur Wurzelliste von H hinzu.
22              if min[H] = NIL or key[A[i]] < key[min[H]] then
23                  min[H] = A[i]
24              end if
25          end if
26      end for
    end algorithm
```

Beispiel 33 Fortsetzung von Beispiel 32:

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Downloaded by Maja Müller (blackcatgirl@freenet.de)



Bemerkung 33

- Wenn die For-Schleife (Zeilen 4–17) terminiert, bleibt von jedem Grad höchstens 1 Baum.
- Wenn vor FHeapConsolidate alle Bäume ungeordnete Binomialbäume sind, sind sie es danach auch.

amortisierte Analyse:

- tatsächliche Kosten c_i:
  - Die For-Schleife (Zeile 4–17) wird höchstens $t(H)-1+D(n)$ Mal durchlaufen ($t(H)$ Knoten in der Wurzelliste minus das Minimum plus die maximale Anzahl an Kindern, also Knotengrad, $D(n)$).
  - Bei jedem Durchlauf der While-Schleife (Zeilen 7–15) werden 2 Wurzeln verkettet, also wird die Anzahl der Bäume in der Wurzelliste um 1 kleiner
  - $c_i \leq d \cdot (t(H) + D(n))$ mit konstantem, aber zunächst irrelevantem, d (also zunächst Analyse mit $c_i \leq t(H) + D(n)$).
- $\Phi(H) = t(H) + 2 \cdot m(H)$

$$\Phi(H') \leq \underbrace{D(n) + 1}_{\text{max. Anzahl der Bäume nach FHeapConsolidate}} + 2 \cdot m(H') \leq D(n) + 1 + m(H)$$

- $\hat{c}_i = c_i + \Phi(H') - \Phi(H)$

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$\begin{aligned}\hat{c_i} &= c_i + \Phi(H') - \Phi(H) \\ &\leq t(H) + D(n) + D(n) + 1 + 2 \cdot m(H) - t(H) - 2 \cdot m(H) \\ &= 2 \cdot D(n) + 1 \in O(D(n))\end{aligned}$$

- In Wirklichkeit gilt $c_i \leq d(D(n)+t(H))$ mit konstantem d. Dies wird mit der Potentialfunktion $\Phi(H) = d(t(H)+2\cdot m(H))$ analysiert. Das führt zum selben Ergebnis.

Satz 14 Sei H ein F-Heap mit n Knoten und $D(n)$ sein maximaler Knotengrad. Auf H kosten INSERT, MINIMUM, UNION amortisiert $O(1)$ Zeit und EXTRACT-MIN amortisiert $O(D(n))$ Zeit.

Bemerkung 34 Sei H ein anfangs leerer F-Heap, auf dem nur die Operationen INSERT, MINIMUM, UNION und EXTRACT-MIN ausgeführt werden. Dann ist jeder Baum ein ungeordneter Binomialbaum. Folgerung: Es gilt in diesem Fall $D(n) = \lfloor \log(n) \rfloor$.

- Die weiteren Operationen DECREASE-KEY und DELETE lösen das (die Aussage über $D(n)$) auf.
- Es ist dann zu zeigen, dass $D(n) \in O(\log(n))$

8.2 Knoten löschen und Schlüssel verringern

Algorithmus 52 FHEAPDECREASEKEY

- x – Knoten
- k – Schlüssel

```
    algorithm FHeapDecreaseKey(H, x, k)
1       if k > key[x] then
2           Error
3       end if
4       key[x] = k
5       y = parent[x]
6       if y ≠ NIL and key[x] < key[y] then
7           Cut(H, x, y)
8           CascadingCut(H, y)
9       end if
10      if key[x] < key[min[H]] then
11          min[H] = x
12      end if
    end algorithm
```


Downloaded by Maja Müller (blackcatgirl@freenet.de)

Algorithmus 53

```
algorithm Cut(H, x, y)
1    Entferne x aus der Kinderliste von y, dekrementiere deg[y].
2    Füge x zur Wurzelliste von H hinzu.
3    parent[x] = NIL
4    label[x] = false
end algorithm
```

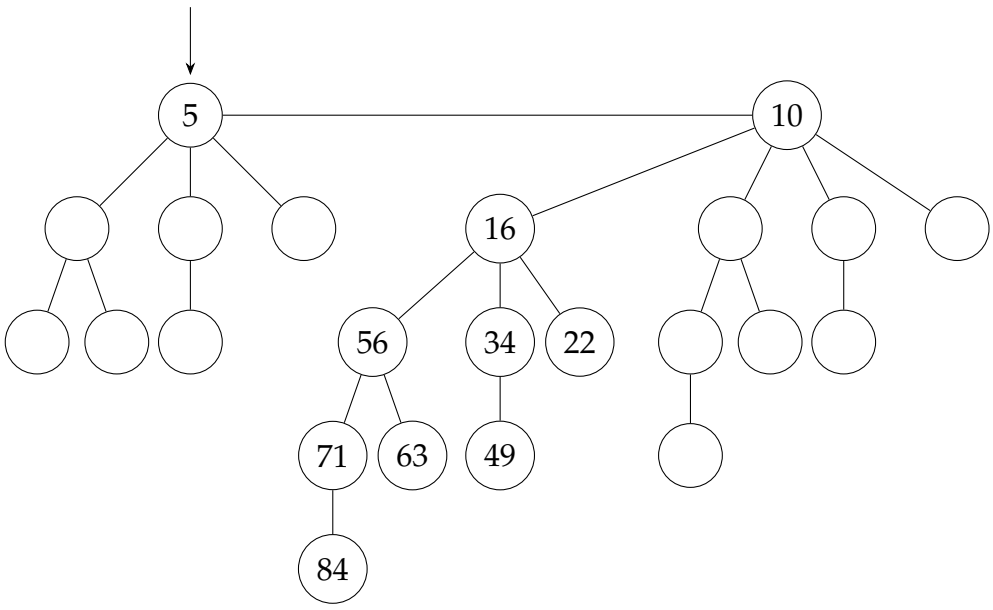
Algorithmus 54

```
algorithm CascadingCut(H, y)
1    z = parent[y]
2    if z ≠ NIL then
3        if label[y] = false then
4            label[y] = true
5        else
6            Cut(H, y, z)
7            CascadingCut(H, z)
8        end if
9    end if
end algorithm
```

Bemerkung 35

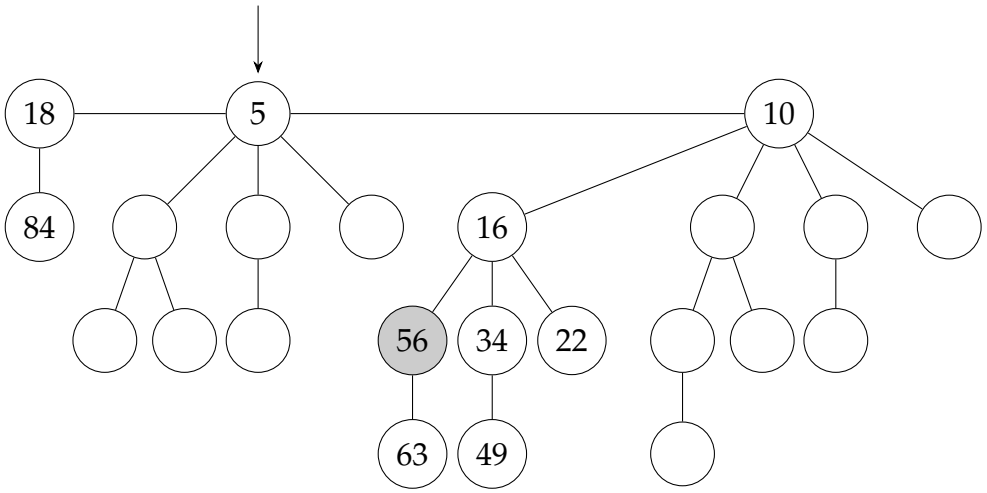
- Verliert ein Knoten das erste Mal ein Kind, wird er markiert.
- Verliert er ein weiteres Kind, wird er selbst vom Vater abgetrennt und unmarkiert in die Wurzelliste gesetzt.

Beispiel 34 Wir betrachten den folgenden F-Heap:

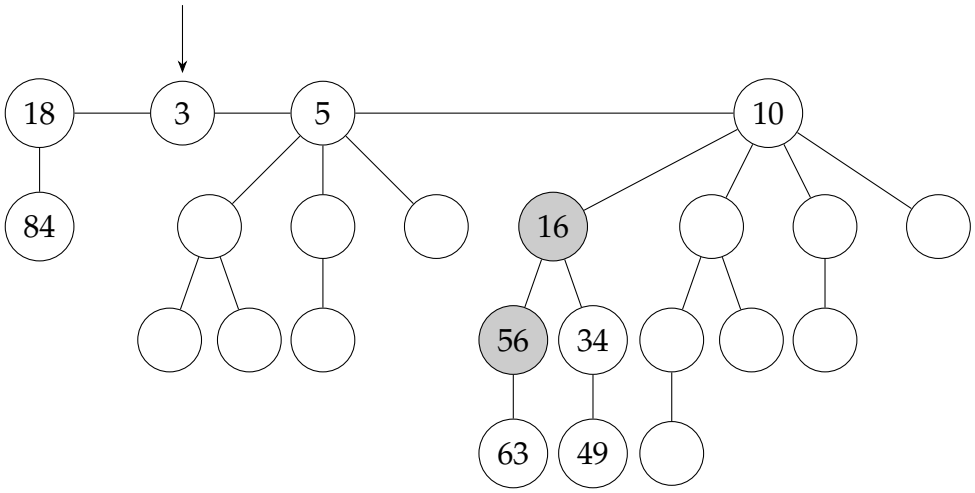


Wir führen FHeapDecreaseKey aus, um den Schlüssel 71 auf 18 zu verkleinern.

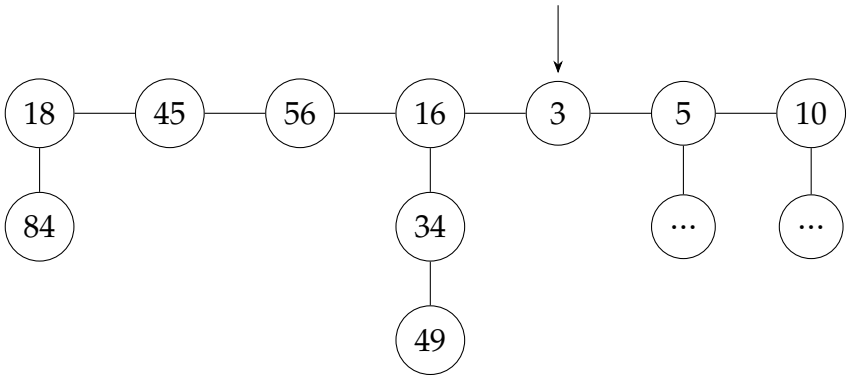
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Im Anschluss führen wir FHeapDecreaseKey aus, um 22 auf 3 zu verringern.



Danach führen wir FHeapDecreaseKey aus, um 63 auf 45 zu reduzieren.



amortisierte Analyse:

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- c ist die Anzahl der Aufrufe von CascadingCut (entspricht der Anzahl abgetrennter Knoten)
- $c_i = k \cdot c$ mit konstantem k
- $t(H') = t(H) + c$
- $m(H') \leq m(H) - (c-1) + 1$ (höchstens ein Knoten wird neu markiert)
- Falls CascadingCut für y aufgerufen wird (also y ein Kind verloren hat), wird getestet, ob y in der Wurzelliste ist.
  - falls ja: keine Markierung
  - falls nein: ist y markiert?
    - ⋆ falls ja: führe CascadingCut auf parent$[y]$ aus
    - ⋆ falls nein: markiere y
- amortisierte Kosten:

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(H') - \Phi(H) \\ &\leq k_c + t(H) + c + 2 \cdot (m(H) - c + 2) - t(H) - 2 \cdot m(H) \\ &= k \cdot c - c + 4\end{aligned}$$

- Da eine Abhängigkeit von k besteht, muss die Potentialfunktion skaliert werden, also $\Phi(H) = k \cdot (t(H) + 2 \cdot m(H))$. Damit ist $\hat{c}_i \leq k \cdot c - k \cdot (c-4) = 4 \cdot k \in O(1)$.

8.3 Wie groß kann $D(n)$ werden?

size(x) bezeichne die Anzahl der Knoten im Teilbaum mit der Wurzel x. Ferner sei deg$[x]$ die genaue Anzahl der Kinder von x.

Lemma 1 Sei x ein beliebiger Knoten in einem F-Heap mit deg$[x] = k$. $y_1, ..., y_k$ bezeichnen die Kinder von x in der Reihenfolge, in der sie mit x verkettet wurden. Dann gilt deg$[y_1] \geq 0$ und deg$[y_i] \geq i - 2$ für $i = 2, ..., k$.

Beweis:

- deg$[y_1] \geq 0$
- $i \geq 2$: bei Verkettung von y_i und x waren $y_1, ..., y_{i-1}$ schon Kinder von x, also ist deg$[x] \geq i - 1$
- bei Verkettung muss deg$[x]$ = deg$[y_i]$ gewesen sein, also zu diesem Zeitpunkt deg$[y_i] \geq i - 1$
- seither hat y_i höchstens 1 Kind verloren (beim 2. wäre er abgetrennt worden, also deg$[y_i] \geq i - 2$. □

Bemerkung 36 Fakten über Fibonacci-Zahlen:

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$f_k = \begin{cases} 0 & k = 0 \\ 1 & k = 1 \\ f_{k-1} + f_{k-2} & k \geq 2 \end{cases}$$

$$f_{k+2} = 1 + \sum_{i=0}^{k} f_i$$

$$f_{k+2} \geq \phi^k \qquad \phi = \frac{1+\sqrt{5}}{2} \text{ (Verhältnis des Goldenen Schnittes)}$$

Lemma 2 Sei x ein beliebiger Knoten in einem F-Heap und sei $k = \deg[x]$. Dann ist $\text{size}(x) \geq f_{k+2} \geq \phi^k$ mit $\phi = 1 + \sqrt{52}$.

Beweis: Sei $s_k = \min_{x \text{ mit } \deg[x]=k} \{\text{size}(x)\}$ die minimale Anzahl von Knoten im Teilbaum mit Grad k. Dabei ist $s_0 = 1$ und $s_1 = 2$. Sei z ein Knoten mit $\deg[z] = k$ und $\text{size}[z] = s_k$.

Da $s_k \leq \text{size}(x)$, ist eine untere Schranke für s_k auch eine untere Schranke für $\text{size}(x)$.

- $y_1, ..., y_k$ sind die Kinder von z in der Reihenfolge der Verkettungen mit z
- zählen der Knoten im Teilbaum mit Wurzel z:

$$\text{size}(z) \geq s_k \geq 2 + \sum_{i=2}^{k} \geq 2 + \sum_{i=2}^{k} s_{i-2}$$

  Die 2 kommt von z selbst und y_1. Das zweite Größer-Gleich gilt aufgrund von Lemma 1.

Behauptung: $s_k \geq f_{k+2}$ für $k \geq 0$.

- Induktionsanfang mit $k = 0$ und $k = 1$
- Induktionsvoraussetzung mit $i = k$: $s_i \geq f_{i+2}$ für $i = 0, ..., k-1$

Schritt:

$$s_k \geq 2 + \sum_{i=2}^{k} s_{i-2} \overset{\text{IV}}{=} 2 + \sum_{i=2}^{k} f_i = 1 + \sum_{i=0}^{k} f_i = f_{k+2}$$

Daraus folgt $\text{size}(x) \geq s_k \geq f_{k+2} \geq \phi^k$. □

Satz 15 Der maximale Knotengrad $D(n)$ in einem F-Heap mit n Knoten ist $O(\log(n))$.

Beweis: Sei x ein Knoten und $k = \deg[x]$. Nach Lemma 2 gilt $n \geq \text{size}(x) \geq \phi^k$. Also ist $k \leq \log_\phi(n)$. Damit ist $D(n) \in O(\log(n))$.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Bemerkung 37 Folgerung: In einem F-Heap erfordern die Operationen EXTRACT-MIN und DELETE $O(\log(n))$ amortisierte Zeit.

9 Union-Find-Strukturen

- abstrakter Datentyp zur Verwaltung dynamischer disjunkter Mengen
- $S = \{S_1, S_2, ..., S_k\}$ mit S_i paarweise disjunkt
- jede der Mengen hat einen Repräsentanten
- Operationen:
  - MAKE-SET(x) erzeugt neue Menge, einziges Element ist x. Damit ist x auch Repräsentant.
  - UNION(x, y) vereinigt die Mengen, deren Repräsentanten x und y sind.
  - FIND-SET(x) gibt Zeiger auf den Repräsentanten derjenigen Menge zurück, die x enthält.

n bezeichne die Anzahl der MAKE-SET-Operationen und m die Gesamtzahl der MAKE-SET-, UNION- und FIND-SET-Operationen.

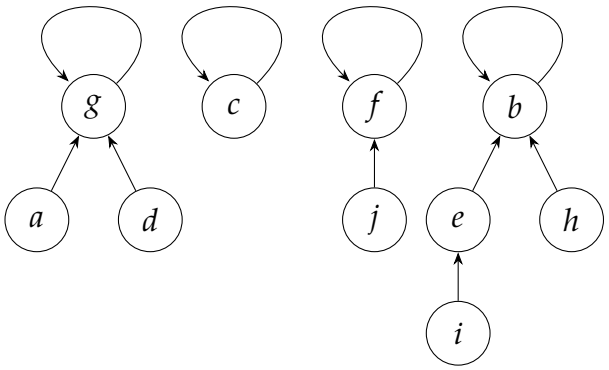
9.1 Disjoint-Set-Forest

Implementierung des Abstrakten Datentyps Union-Find-Struktur

Definition 17 Ein *wurzelgerichteter Baum* ist ein Baum, bei dem jeder Knoten einen Verweis auf seinen Vater hat und die Wurzel auf sich selbst zeigt.

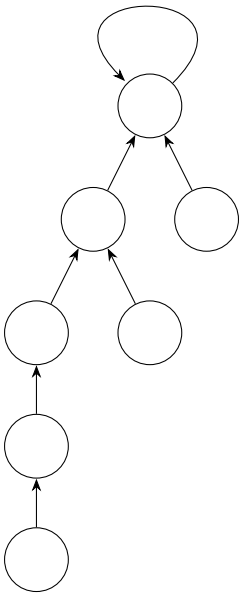
Definition 18 Ein *Disjoint-Set-Forest* für eine Menge $S = \{S_1, ..., S_k\}$ paarweise disjunkter Mengen ist eine Menge von k wurzelgerichteten Bäumen. Jeder Baum enthält die Elemente einer Menge S_i. Der Repräsentant der Menge S_i steht in der Wurzel des zu S_i gehörenden Baumes.

Beispiel 35 $S_1 = \{a, g, d\}, S_2 = \{c\}, S_3 = \{f, j\}, S_4 = \{b, e, h, i\}$



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 20 Wurzelgerichter Baum



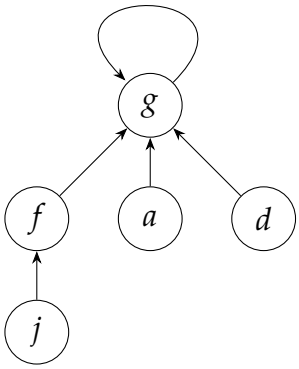
$a, b, c, \ldots$ sind keine Schlüssel und es gibt keine Ordnung.

Algorithmus 55 MakeSet(x) erzeugt einen Baum mit einem Knoten.

Algorithmus 56 FindSet(x) Folgen von x aus den Vaterzeigern bis die Wurzel getroffen wird.

Algorithmus 57 DSFUnion(x, y) Wurzel des einen Baumes zeigt auf die Wurzel des anderen.

Beispiel 36 Beispiel für Union:



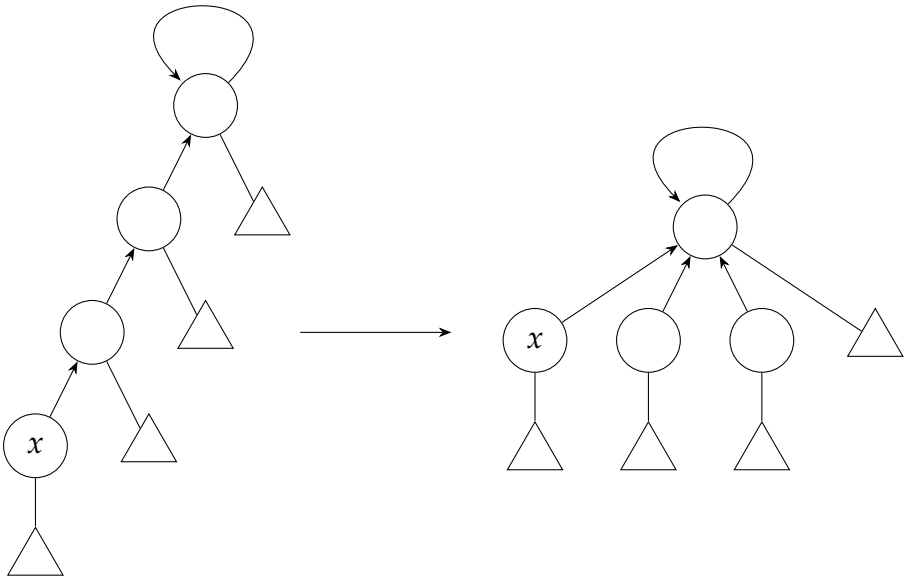
Heuristiken zur Verbesserung der Laufzeit:

- Vereinigung nach Rang:
  - Ziel: Wurzel des Baumes mit kleinerem Rang zeigt auf Wurzel des Baumes mit größerem.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

  - Rang(x)
    - ⋆ definiert sich durch Algorithmus
    - ⋆ obere Schranke für die Höhe eines Baumes
- Pfadverkürzung
  - bei FIND-SET alle auf Pfad zur Wurzel direkt an die Wurzel hängen
  - beim nächsten FIND-SET haben alle diese Knoten einen kurzen Weg.

Abbildung 21 Visualisierung der Pfadverkürzung



Bemerkung 38 In den folgenden Algorithmen werden beide Heuristiken verwandt.

Algorithmus 58

```
algorithm MakeSet(x)
1    parent[x] = x
2    Rang[x] = 0
end algorithm
```

Algorithmus 59

```
algorithm FindSet(x)
1    if x ≠ parent[x] then
2        parent[x] = FindSet(parent[x])
3    end if
4    return parent[x]
end algorithm
```

Algorithmus 60

```
algorithm DSFUnion(x, y)
1    DSFLink(FindSet(x), FindSet(y))
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
end algorithm
```

Algorithmus 61

```
algorithm DSFLink(x, y)
1     if Rang[x] > Rang[y] then
2         parent[y] = x
3     else
4         parent[x] = y
5         if Rang[x] = Rang[y] then
6             Rang[y] = Rang[y] + 1
7         end if
8     end if
end algorithm
```

Bemerkung 39 Anmerkungen zu DSFUnion:

- Union für Bäume mit gleichem Rang (der Wurzel) wählt beliebig einen als Wurzel aus und inkrementiert den Rang der neuen Wurzel.
- Union für Bäume, deren Wurzel verschiedenen Rang haben:
  - Baum mit kleinerem Rang wird an die Wurzel des Baumes mit größerem Rang gehangen.
  - Ränge bleiben unverändert.

Bemerkung 40 Behauptung: Für jeden Knoten x in einem Disjoint-Set-Forest gilt $\text{Rang}[x] \leq \lfloor \log(n) \rfloor$.

Wir zeigen zunächst: Werden nur Make-Set- und Union-Operationen ausgeführt, gilt: Der Teilbaum mit Wurzel x und $\text{Rang}[x] = k$ hat mindestens 2^k Knoten.

- Induktionsanfang mit $k = 0$: $\text{Rang}[x] = 0$ und $2^0 = 1$ Knoten
- Induktionsschritt: $k \to k + 1$:
  - Rang $k + 1$ entsteht durch UNION zweier Bäume vom Rang k. Jeder von diesen hat 2^k Knoten. Ein Baum vom Rang $k + 1$ hat 2^{k+1} Knoten.
  - Find-Set ändert daran nichts, denn es verringert die Knotenzahlen, aber nicht für die Wurzeln der Bäume. Nur für die können die Ränge noch steigen.

Bemerkung 41 height$[x]$ bezeichnet die Anzahl der Knoten auf dem längstem Weg zu einem Blatt.

Bemerkung 42 Behauptung: $\text{Rang}[x]$ ist eine obere Schranke für die Höhe von x.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Rang$[x] = 0$: einzelner Knoten, height$[x] = 0$
- Induktionsvoraussetzung: Für einen Knoten x von Rang k gilt: height$[x] \leq$ Rang$[x]$.
- Induktionsbehaputung: Für einen Knoten x von Rang $k+1$ gilt: height$[x] \leq$ Rang$[x]$.
- Induktionsschritt mit $k \to k+1$:
  - Sei x ein Knoten vom Rang $k+1$.
  - Dieser Baum ist entstanden
    1. durch Link zweier Bäume vom Rang k oder
    2. durch Link von Baum mit Rang $k+1$ und Baum mit Rang $\leq k$.
  - Fall 1: Beide haben nach Induktionsvoraussetzung height $\leq k$. Der entstandene Baum hat höchstens height $= k+1$.
  - Fall 2: height$[x_1] \leq k+1$ und height$[x_2] \leq k$. Das bedeutet x_2 wird Kind von x_1. Damit ist die neue Höhe height$[x_1] \leq k+1$.

Satz 16 Eine beliebige Folge von m MakeSet-, DSFUnion-, und FindSet-Operationen, von denen n MakeSet-Operationen sind, benötigt mit „Vereinigung nach Rang“ (ohne Pfadverkürzung) höchstens $O(m \log(n))$ Zeit.

Beweis:

- MakeSet und DSFUnion kosten $O(1)$
- FindSet liegt in O(Höhe des Baumes)
- Rang ist obere Schranke für die Höhe (Bemerkung 42 und Bemerkung 40)
- also $O(\log(n))$

Satz 17 Eine Folge von m MakeSet-, DSFUnion-, und FindSet-Operationen benötigt im Disjoint-Set-Forest mit Vereinigung nach Rang und Pfadverkürzung höchstens $O(m \cdot \alpha(n))$ Zeit[4].

10 Graphenalgorithmen

Definition 19 Ein *gerichteter Graph* G ist ein Paar (V, E), wobei V eine endliche Menge ist und $E \subseteq V \times V$. Die Elemente von V heißen Knoten, die von E Kanten.

Bemerkung 43 In einem ungerichteten Graph sind die Kanten Zweiermengen von Knoten, d.h. $\{u, v\}$ mit $u, v \in V$ und $u \neq v$.

[4] Dabei ist α die Umkehrung der Ackermannfunktion.

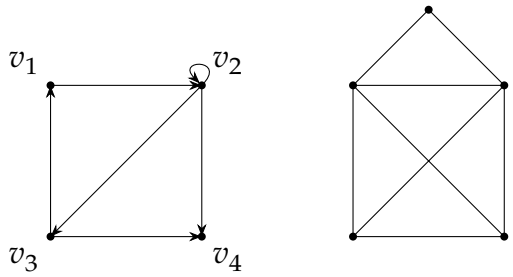
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Bemerkung 44 Die Eingabegrößen für einen Graphenalgorithmus sind $|V|$ und $|E|$.

Definition 20 Sei $G = (V, E)$ ein gerichteter Graph mit $V = \{v_1, ..., v_n\}$. Die *Adjazenzmatrix* $A(a_{ij})_{i,j=1}^{n}$ von G ist definiert durch

$$\forall i, j \in \{1, ..., n\} : a_{ij} = \begin{cases} 1 & (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases} .$$

Beispiel 37 Ein gerichteter und ein ungerichteter Graph.



Beispiel 38 Adjazenzmatrix zum gerichteten Graphen aus Beispiel 37:

Knoten	v_1	v_2	v_3	v_4
v_1	0	1	0	0
v_2	0	1	1	1
v_3	1	0	0	1
v_4	0	0	0	0

Bemerkung 45 Der Speicherbedarf für eine Adjazenzmatrix liegt bei $O(n^2)$, egal, wie groß E ist.

- Für ungerichtete Graphen ist

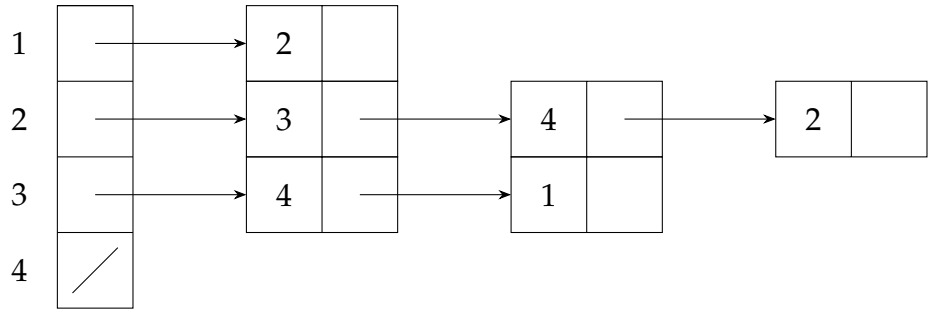
$$a_{ij} = \begin{cases} 1 & \{v_i, v_j\} \in E \\ 0 & \text{sonst} \end{cases}$$

  und die Adjazenzmatrix symmetrisch, hat aber trotzdem $O(n^2)$ Einträge.
- Bei gewichteten Graphen kann in a_{ij} das Gewicht $w(v_i, v_j)$ der Kante (v_i, v_j) abgespeichert werden.
- Diese Speicherung ist ineffizient, wenn der Graph nur wenige Kanten hat, z.B. Kantenzahl m mit $m \in O(n)$.

Definition 21 Sei $G = (V, E)$ ein gerichteter Graph mit $V = \{v_1, ..., v_n\}$. Die *Adjazenzliste* AL von G ist das Feld $AL[1...n]$ bestehend aus n Listen, eine für jeden Knoten von G. Dabei enthält $AL[i]$ alle Knoten v_j mit $(v_i, v_j) \in E$.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 39 Beispiel für eine Adjazenzliste anhand des gerichteten Graphen aus Beispiel 37:



- Die Knoten in $A[i]$ sind in beliebiger Reihenfolge gespeichert.
- Dies ist effizient, denn die Summe der Listen ist $|E|$ (im ungerichteten $2 \cdot |E|$).
- Auch Gewichte können gespeichert werden.
- Nachteil: Für ein gegebenes i, j herauszufinden, ob $(v_i, v_j) \in E$, ist schwieriger ($O(1)$ in der Matrix und $O(n)$ in der Liste).

10.1 Breitensuche

- einer der einfachsten Suchalgorithmen in Graphen
- Grundtyp vieler Graphenalgorithmen
- von ausgewählten Startknoten aus wird der Graph durchsucht, erst in der Breite, dann in der Tiefe
- erzeugt Breitensuchbaum; dieser enthält alle vom Startknoten aus erreichbaren Knoten
- Pfad zwischen zwei Knoten im Baum ist kürzester Pfad zwischen diesen Knoten im Graphen
- Knoten werden währen des Algorithmus gefärbt
  - zu Beginn sind alle weiß
  - grau: Knoten wurde entdeckt
  - schwarz: auch alle Nachbarn des Knoten wurden schon entdeckt

Bemerkung 46 BFS-Baum:

- Wurzel s ist der Startknoten
- wird weißer Knoten v entdeckt, so wird v in den Baum aufgenommen, auch die Kante (u, v) über die v entdeckt wurde, u ist Vater von v
- für jeden Knoten speichern wir folgende Informationen

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- color(u): Farbe von u
- $\pi(u)$: Vorgänger (Vater) von u
- $d(u)$: Entfernung zum Startknoten

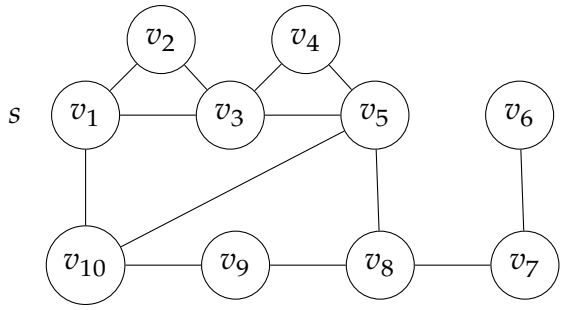
- benutzen Datenstruktur *Warteschlange* (FIFO), um die Knoten zu speichern:
  - head: Kopf
  - enqueue: fügt an
  - dequeue: entfernt
- Input-Graph in Adjazenzlistendarstellung gegeben

Algorithmus 62

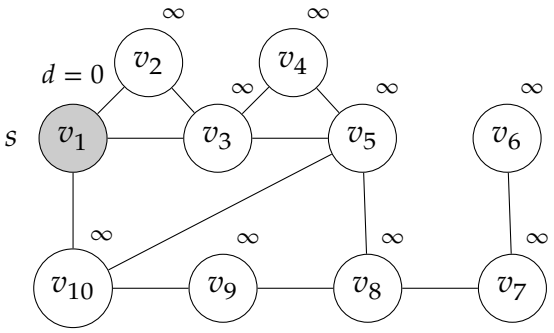
algorithm BFS(G, s)
1 **for all** Knoten $u \in V[G] \setminus s$ **do**
2 ▷ *alle weiß, alle Vaterzeiger* NIL, *alle Abstände unendlich*
3 color$[u]$ = weiß
4 $d[u] = \infty$
5 $\pi[u]$ = NIL
6 **end for**
7 color$[s]$ = grau ▷ *Startknoten entdeckt*
8 $d[s] = 0$
9 $\pi[s]$ = NIL
10 $Q = \emptyset$
11 ENQUEUE(Q, s)
12 **while** $Q \neq \emptyset$ **do**
13 u = DEQUEUE(Q)
14 **for all** $v \in AL[u]$ **do**
15 **if** color$[v]$ = weiß **then**
16 ▷ *alle weißen Nachbarn (unentdeckte Knoten) in den Baum aufnehmen...*
17 color$[v]$ = grau
18 $d[v] = d[u] + 1$
19 $\pi[v] = u$
20 ENQUEUE(Q, v) ▷ *... und in die Schlange*
21 **end if**
22 **end for**
23 color$[u]$ = schwarz
24 **end while**
end algorithm

Beispiel 40 Graph zur Breitensuche:

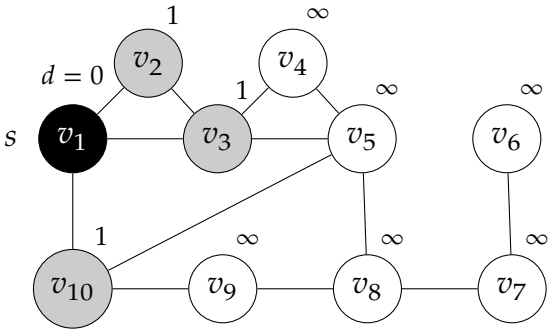
Downloaded by Maja Müller (blackcatgirl@freenet.de)

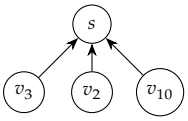


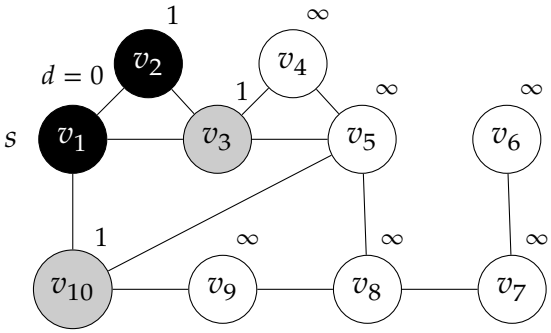
Initialisierung:

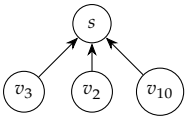






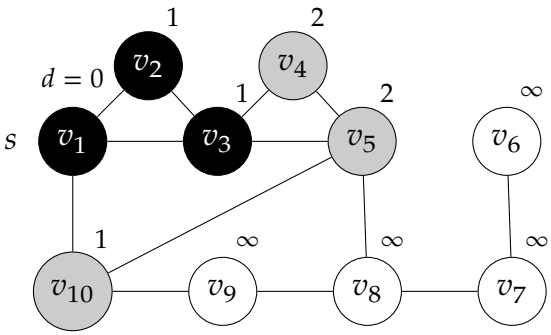


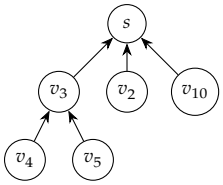


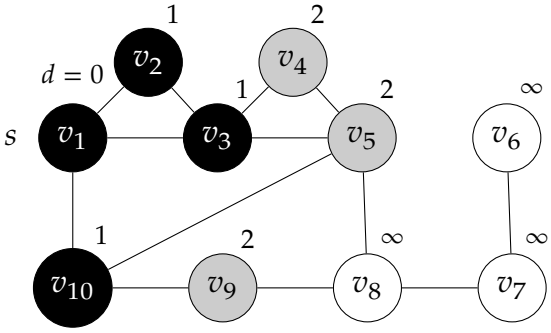


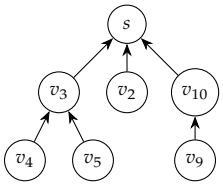
This document is available on studocu

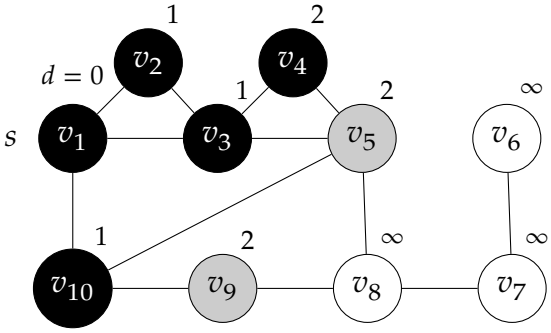
Downloaded by Maja Müller (blackcatgirl@freenet.de)

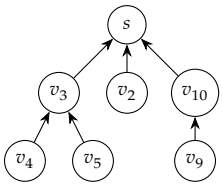


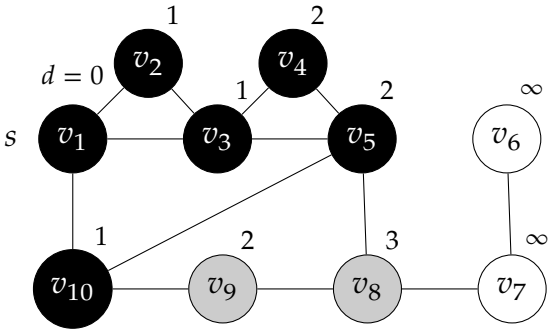


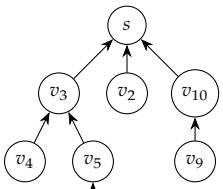




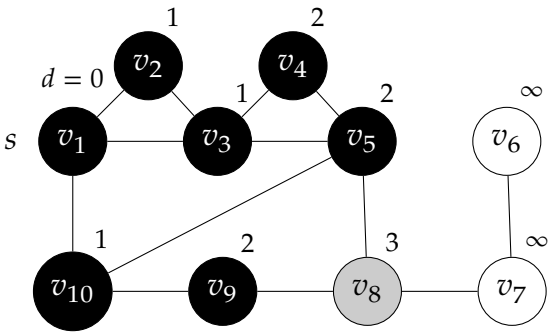


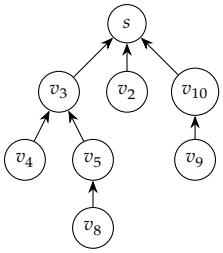


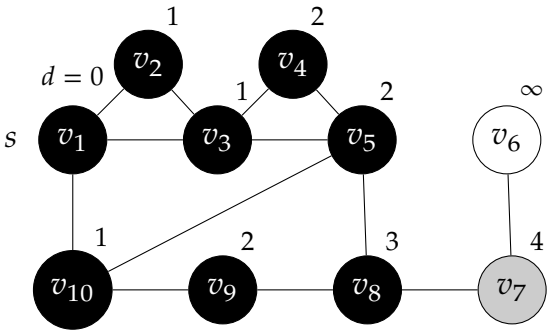


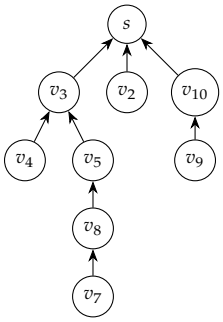


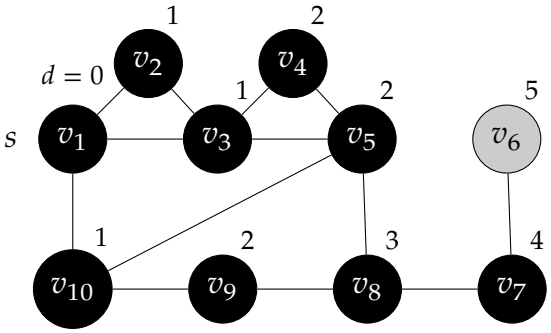
Downloaded by Maja Müller (blackcatgirl@freenet.de)

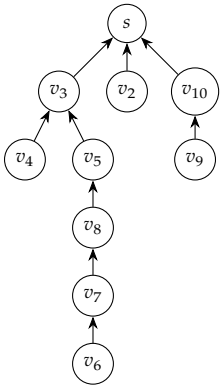




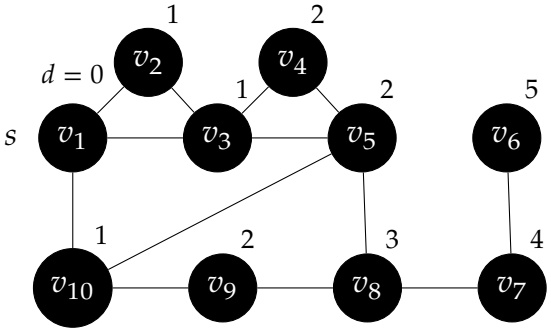


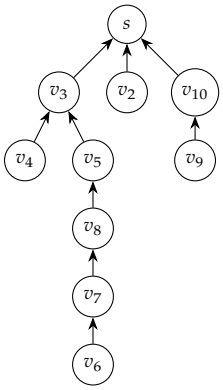






Ergebnis:





This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Laufzeit:

- jeder Knoten wird einmal in die Queue aufgenommen und einmal entfernt
- prüfen aller Adjazenzlisten → O(Summe aller Längen) $= O(|E|)$
- also ist $O(|V| + |E|)$

10.2 Kürzeste Wege

Der kürzeste Pfad zwischen den Knoten u, v ist derjenige mit der kleinsten Anzahl von Kanten.

$$\delta(u,v) = \begin{cases} \infty & \text{fall es keinen Pfad von } u \text{ nach } v \text{ gibt} \\ |P| & \text{sonst mit } |P| = \text{Zahl der Kanten des kürzesten Pfades} \end{cases}$$

Problem Shortest-Paths:

- Input: $G = (V, E), s \in V$ mit ungerichtetem G
- Output: $\delta(s, u)$ für alle $u \in V$

Die in BFS berechneten $d(u)$-Werte sind genau die Länge der kürzesten Wege von s nach u.

Lemma 3 Sei G ein ungerichteter Graph, $s \in V$ und $\{u, v\} \in E$. Dann gilt $\delta(s, v) \leq \delta(s, u) + 1$.

Beweis: Von s nach v gibt es einen Pfad über u. Dieser hat die Länge des kürzesten Pfades von s nach u, $\delta(s, u)$, zusätzlich zur Kante von u nach v, also $\delta(s, u) + 1$. □

Lemma 4 Sei G ein ungerichteter Graph. Für alle $v \in V$ sei $v.d$ der von BFS berechnete d-Wert. Dann gilt $v.d \geq \delta(s, v)$.

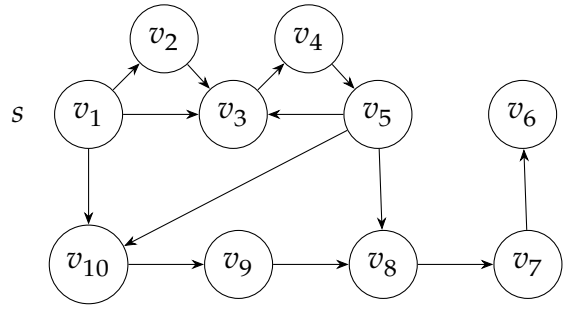
Lemma 5 Sei $Q = \langle v_1, ..., v_k \rangle$ die Warteschlange während der Ausführung von BFS, wobei v_1 der Kopf ist. Dann gilt $v_1.d \leq v_2.d \leq ... \leq v_k.d \leq v_1.d + 1$.

10.3 Tiefensuche

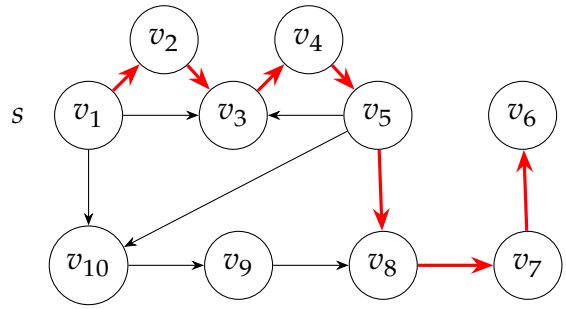
- ist ebenfalls ein Durchsuchungsalgorithmus für Graphen
- geht erst in die Tiefe, dann in die Breite
  - soweit es geht in die Tiefe
  - wenn keine unbesuchten Nachbarn mehr gefunden werden, nur soweit zurück, bis wieder ein unbesuchter Nachbar entdeckt wird
  - von dort aus wieder in die Tiefe

Downloaded by Maja Müller (blackcatgirl@freenet.de)

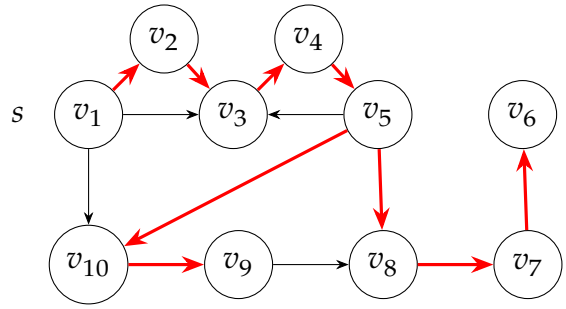
Beispiel 41 Wir betrachten den folgenden gerichteten Graphen.



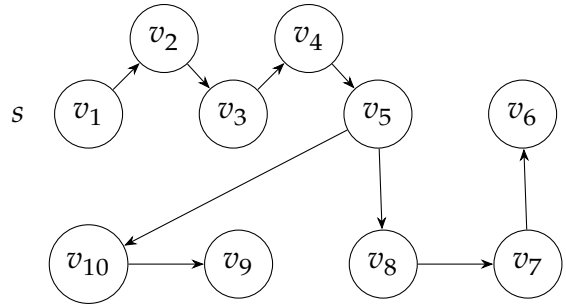
Vom Startknoden s aus gehen wir „immer links“ bis zum Ende.



Wir stellen fest, dass von v_6 nicht weiter in die Tiefe fortgefahren werden kann. Deshalb gehen wir zurück, bis wir wieder in die Tiefe gehen können.



Tiefensuchbaum:



Algorithmus 63 DFS

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

- globale Variable *time*: verteilt Zeitstempel
- $d(u)$: Entdeckungszeitpunkt (Knoten wird grau)
- $f(u)$: Zeitpunkt, zu dem Nachbarschaft von u komplett abgearbeitet (Knoten wird schwarz)
- Wird ein neuer Knoten entdeckt, erhält er das Vorgänger-Attribut $\pi(u)$ (derjenige Knoten, in dessen Nachbarschaft er gefunden wurde).
- Wird kein neuer benachbarter Knoten gefunden, geht es bei einem beliebigen Knoten weiter (ggf. in einer anderen Zusammenhangskomponente). Daher entsteht kein Baum, sondern ein Wald.

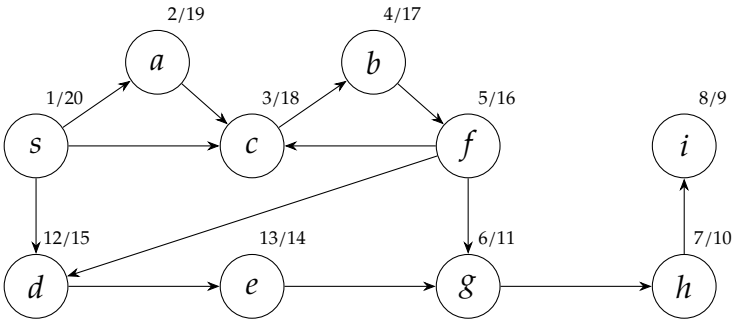
algorithm DFS(G)
1 **for all** Knoten $v \in V[G]$ **do**
2 color$[v]$ = weiß
3 $\pi[v]$ = NIL
4 **end for**
5 $time = 0$
6 **for all** Knoten $v \in V[G]$ **do**
7 **if** color$[v]$ = weiß **then**
8 DFSVisit(v)
9 **end if**
10 **end for**
end algorithm

Algorithmus 64

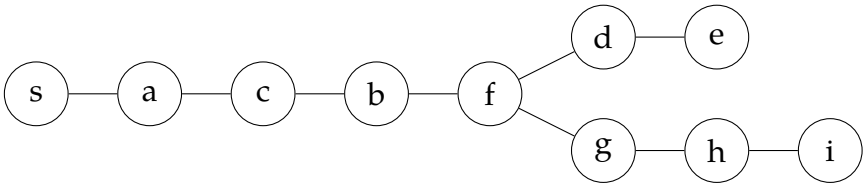
algorithm DFSVisit(v)
1 color$[v]$ = grau
2 $time = time + 1$
3 $d[v] = time$
4 **for all** $u \in AL[v]$ **do**
5 **if** color$[u]$ = weiß **then**
6 $\pi[u] = v$
7 DFSVisit(u)
8 **end if**
9 **end for**
10 color$[v]$ = schwarz
11 $time = time + 1$
12 $f[v] = time$
end algorithm

Beispiel 42 Wir betrachten den folgenden Graphen, auf dem eine Tiefensuche ausgeführt wurde:

Downloaded by Maja Müller (blackcatgirl@freenet.de)



DFS-Tree fertig aufgebaut:



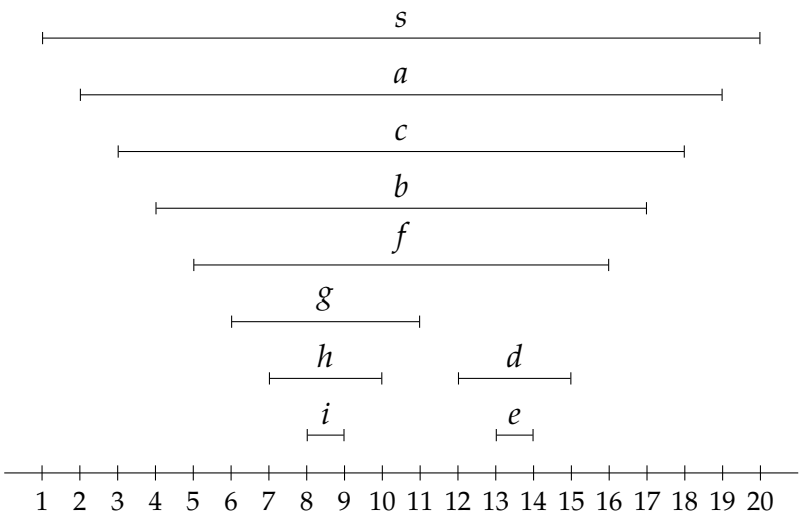
Laufzeit:

- Für jeden Knoten wird genau ein DFSVisit aufgerufen (wenn er grau wird).
- Für jeden Knoten werden alle inzidenten Kanten geprüft:

$$O\left(\sum_{v \in V} \text{länge}[AL[v]]\right).$$

- also $O(|V| + |E|)$

Abbildung 22 Visualisierung des Klammertheorems (Satz 18) für Beispiel 42



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Satz 18 Klammertheorem: Nach jedem Durchlauf von DFS gilt für beliebige Knoten u und v genau eine der folgenden Bedingungen:

1. Die Intervalle $[d(u),f(u)]$ und $[d(v),f(v)]$ sind disjunkt und weder u noch v ist Abkömmling des jeweils anderen im DFS-Wald.
2. $[d(u),f(u)]$ ist vollständig in $[d(v),f(v)]$ enthalten und u ist Abkömmling von v.
3. $[d(v),f(v)]$ ist vollständig in $[d(u),f(u)]$ enthalten und v ist Abkömmling von u.

Beweis:

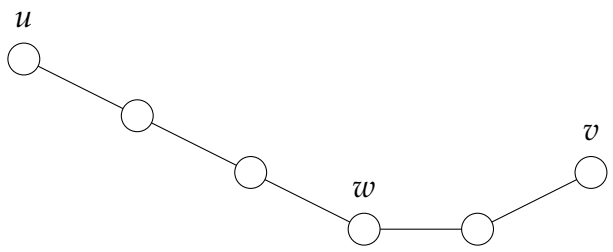
- Fall 1: $d(v) < d(u)$:
  - Fall 1.1: $d(u) < f(v)$:
    - u wird entdeckt, wenn v noch grau ist
    - u ist Abkömmling von v
    - da u später entdeckt wurde als v, wird die Nachbarschaft von u eher abgearbeitet als die von v
    - $f(u) < f(v)$
    - $d(u),f(u)]$ vollständig in $[d(v),f(v)]$ enthalten
  - Fall 1.2: $f(v) < d(u)$:
    - $d(v) < f(v) < d(u) < f(u)$
    - $[d(v),f(v)]$ und $[d(u),f(u)]$ disjunkt
    - keiner ist Abkömmling des anderen
- Fall 2: $d(u) < d(v)$: analog mit u, v vertauscht

Bemerkung 47 Folgerung: v ist Abkömmling von u genau dann, wenn $d(u) < d(v) < f(v) < f(u)$.

Satz 19 Weißer-Pfad-Theorem: v ist Abkömmling von u im DFS-Wald genau dann, wenn zum Zeitpunkt $d(u)$ v von u aus auf einem Pfad, dessen Knoten alle weiß sind, erreichbar ist.

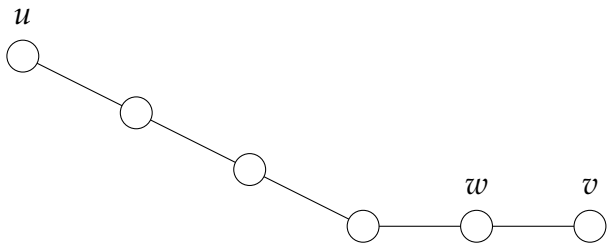
Beweis:

- $\Rightarrow$: Sei v Abkömmling von u und w ein beliebiger Knoten auf dem Pfad von u nach v im DFS-Tree, wobei w Abkömmling von u ist. Nach Bemerkung 47 gilt $d(u) < d(v)$. Also ist w zur Zeit $d(u)$ weiß.



Downloaded by Maja Müller (blackcatgirl@freenet.de)

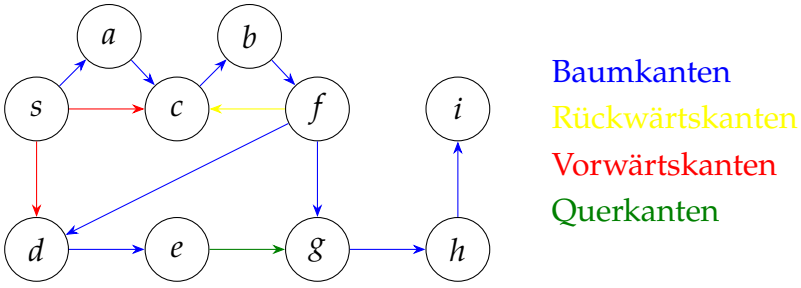
- $\Leftarrow$: Zeitpunkt $d(u)$: v ist von u aus auf einem weißen Pfad erreichbar und v wird kein Abkömmling von u und v ist der zu u am nächsten liegende Knoten, für den das gilt. Sei w der Vorgänger von v auf diesem Pfad. Dann ist w Abkömmling von u.
  Nach Satz 18 gilt $f(w) \leq f(u)$. v wird nach u entdeckt $(d(u) < d(v)$, aber bevor w abgearbeitet ist $(f(v) < f(w))$. Also gilt $d(u) < d(v) < f(v) < f(w) < f(u)$. Aus Satz 18 folgt, dass v Abkömmling von u ist, was zum Widerspruch führt.



Bemerkung 48 Klassifizierung der Kanten in gerichteten Graphen:

a. *Baumkanten* sind die Kanten im DFS-Wald.
b. *Rückwärtskanten* sind Kanten, die von einem Knoten u zu einem Vorfahren v von u verlaufen.
c. *Vorwärtskanten* sind Kanten, die keine Baumkanten sind, aber von einem Knoten u zu einem seiner Abkömmlinge verlaufen.
d. *Querkanten* sind alle übrigen Kanten.

Beispiel 43 Anhand von Bemerkung 48 werden die Kanten im Eingangsbeispiel zum Kapitel klassifiziert.



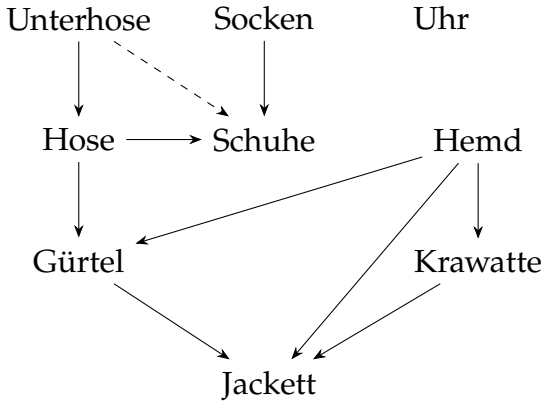
Bemerkung 49 In ungerichteten Graphen gibt es nur Baum- und Rückwärtskanten.

10.4 Topologisches Sortieren

- Input: gerichteter, kreisfreier Graph $G = (V, E)$ mit $V = \{v_1, ..., v_n\}$
- Output: Permutation: $v_{i_1}, ..., v_{i_n}$ der Knoten $v_1, ..., v_n$, sodass für alle j, j' gilt $(v_{i_j}, v_{i_{j'}}) \in E \Rightarrow j < j'$

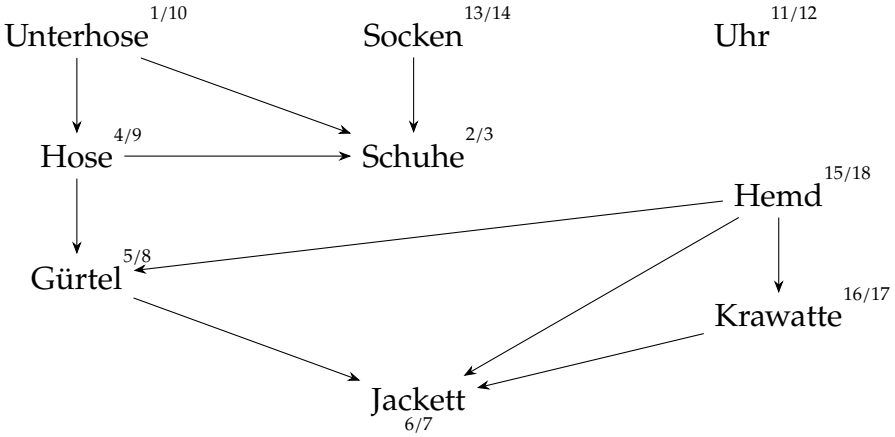
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 44 Input für eine topologische Sortierung:



- Reihenfolge finden, sodass alle Kanten von kleinerem Index zu größerem verlaufen
- Falls der Graph einen Kreis enthält, gibt es keine topologische Sortierung.

Beispiel 45 Tiefensuche in der topologischen Sortierung:



Beispielausgabe der Sortierung: (1.) Hemd (2.) Krawatte (3.) Socken (4.) Uhr (5.) Unterhose (6.) Hose (7.) Gürtel (8.) Jackett (9.) Schuhe

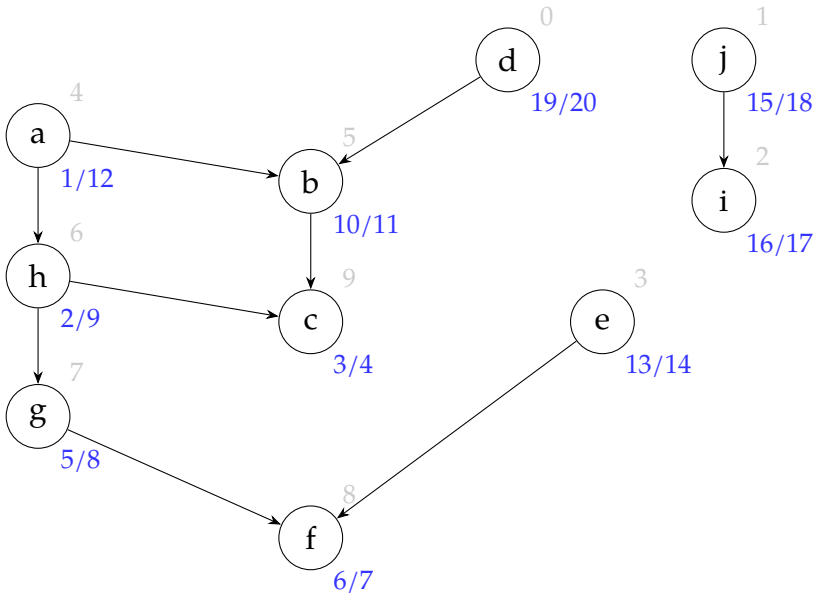
Algorithmus 65 TopologicalSort(G)

- Rufe DFS(G) zur Berechnung der Endzeiten $f[v]$ für alle v auf.
- Füge jeden abgearbeiteten Knoten an den Kopf einer verketteten Liste ein.
- Gib die verkettete Liste der Knoten aus.

Laufzeit: $O(|V| + |E|)$, da die Tiefensuche diese Größenordnung benötigt.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 46 Wir führen auf dem folgenden Graphen eine topologische Sortierung durch:

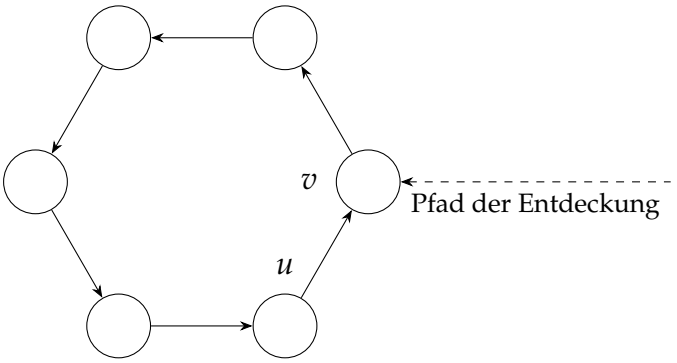


Ergebnisreihenfolge: d → j → i → e → a → b → h → g → f → c

Lemma 6 Ein gerichteter Graph ist genau dann kreisfrei, wenn DFS(G) keine Rückwärtskanten liefert.

Beweis:

- ⇒: Wir nehmen an, es gibt eine Rückwärtskante (u, v). Dann ist v Vorfahre von u im DFS-Forest. Daher gibt es einen Pfad von v nach u. Also schließt (u, v) einen Kreis.
- ⇐: Wir nehmen an, G enthält einen Kreis. v sei der erste auf c entdeckte Knoten, (u, v) die eingehende Kante. Daher bilden zum Zeitpunkt $d(v)$ die Knoten von c einen weißen Pfad von v nach u.



Nach dem Weißen-Pfad-Theorem (Satz 19) wird u Nachfahre von v im DFS-Tree. Damit ist (u, v) Rückwärtskante. □

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Satz 20 TopologicalSort erzeugt eine topologische Sortierung für einen DAG (gerichteten kreisfreien Graph).

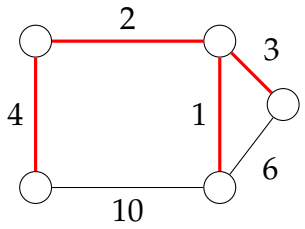
Beweis: Es ist zu zeigen, dass für alle Paare u, v mit $u \neq v$ und $(u, v) \in E$ gilt $f(u) < f(v)$. Sei $(u, v) \in E$ diejenige Kante, die DFS(G) gerade bearbeitet. Fallunterscheidung:

- Fall 1: v ist grau. Dann ist v Vorfahre von u. Daraus folgt (u, v) ist Rückwärtskante. Widerspruch.
- Fall 2: v ist weiß. v ist Abkömmling von u. Daraus folgt $f(v) < f(u)$.
- Fall 3: v ist schwarz. Dann wurde v bereits vollständig abgearbeitet. $f(v)$ wurde bereits definiert. (u, v) wird gerade bearbeitet, also ist $f(u)$ noch nicht definiert. Daraus folgt $f(v) < f(u)$.

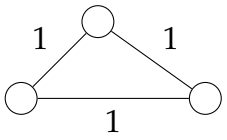
10.5 Minimale Spannbäume

Definition 22 Sei $G = (V, E)$ ein ungerichteter zusammenhängender Graph und $w : E \to \mathbb{R}$ eine Funktion, die jeder Kante ein Gewicht zuordnet. Ein Teilgraph $T = (V', E')$ heißt *Spannbaum* von G genau dann, wenn $V = V'$ und T ein Baum ist. Ein *minimaler Spannbaum* (MST, minimum spanning tree) ist ein Spannbaum mit minimalem Gewicht.

Beispiel 47 Suchen eines MST:

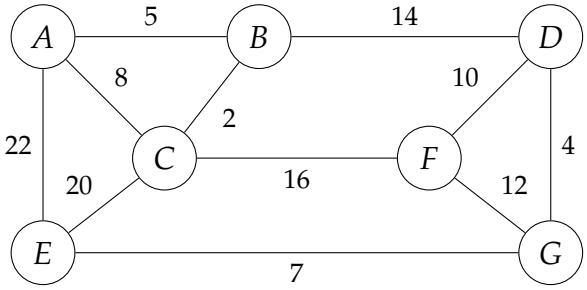


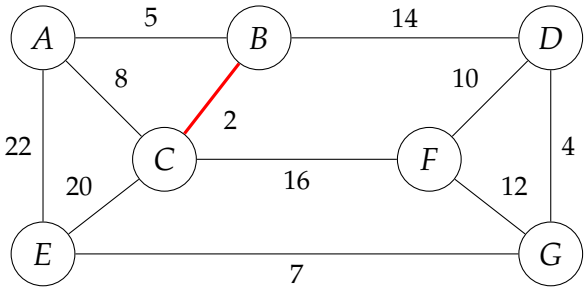
Beispiel 48 Der MST ist nicht eindeutig. Für den folgenden Graphen finden wir 3 MSTs:

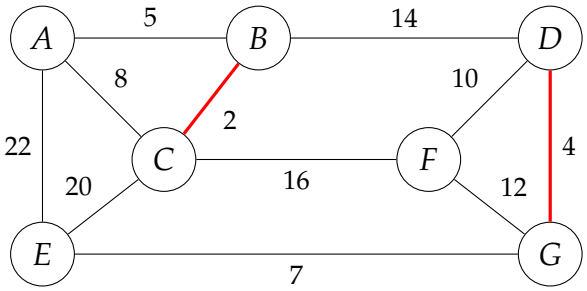


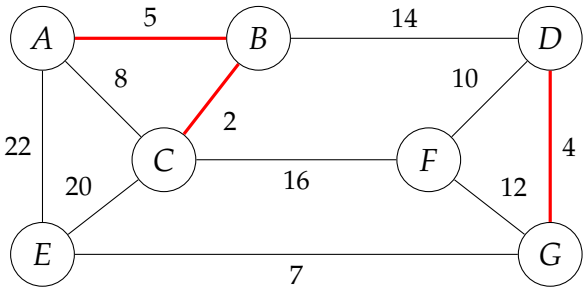
Beispiel 49 Anwendung des Algorithmus von Kruskal auf einen Graphen

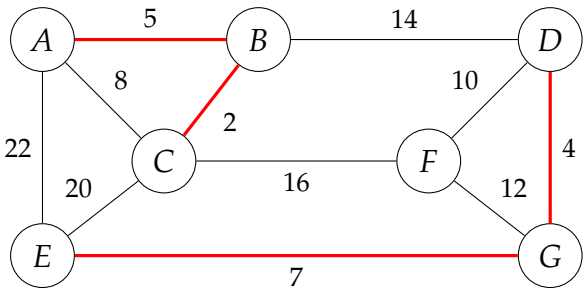
Downloaded by Maja Müller (blackcatgirl@freenet.de)



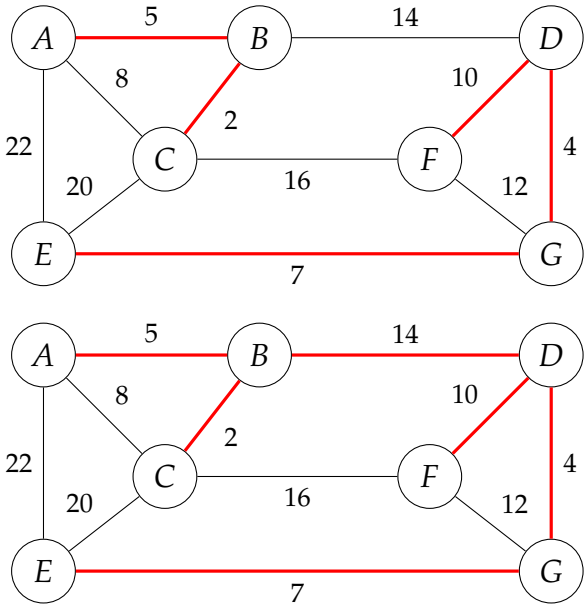








This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Konstruktionsprinzip:

- Der Algorithmus bestimmt Schritt für Schritt diejenigen Kanten, die zu einem MST gehören.
- Der Algorithmus verwaltet die Kantenmenge A. Für A gilt die Schleifeninvariante $(*)$: Unmittelbar vor jeder Iteration ist A Teilmenge der Kantenmenge eines MST.
- In jedem Schritt wird $\{u, v\}$ bestimmt, sodass für $A \cup \{u, v\}$ wieder $(*)$ gilt. Eine solche Kante heißt „sichere Kante“.

Algorithmus 66

algorithm GenericMST(G, w)
1 $A = \emptyset$
2 **while** A bildet keinen Spannbaum **do**
3 bestimme eine Kante $\{u, v\}$, die sicher für A ist
4 $A = A \cup \{\{u, v\}\}$.
5 **end while**
6 **return** A
end algorithm

Bemerkung 50

- Es ist klar, dass die Invariante gilt.
- Die Existenz einer sicheren Kante ist abgesichert, da es einen Spannbaum gibt.

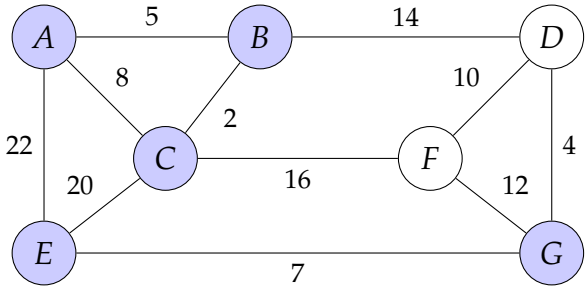
Definition 23 Sei $G = (V, E)$ ein zusammenhängender ungerichteter Graph.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

1. Ein *Schnitt* $(S, V \setminus S)$ von G ist eine Zerlegung von V.
2. Eine Kante $\{u, v\}$ kreuzt den Schnitt $(S, V \setminus S)$ genau dann, wenn $(u \in S \wedge v \in V \setminus S) \vee (v \in S \wedge u \in V \setminus S)$.
3. Eine Menge A von Kanten respektiert den Schnitt $(S, V \setminus S)$ genau dann, wenn keine Kante aus A den Schnitt kreuzt.
4. Eine Kante $\{u, v\}$ heißt *leichte Kante* eines Schnitts $(S, V \setminus S)$ genau dann, wenn $w(\{u, v\}) = \min\{w(e) \mid e \text{ kreuzt } (S, V \setminus S)\}$.

Beispiel 50

Schnitt von blau und weiß



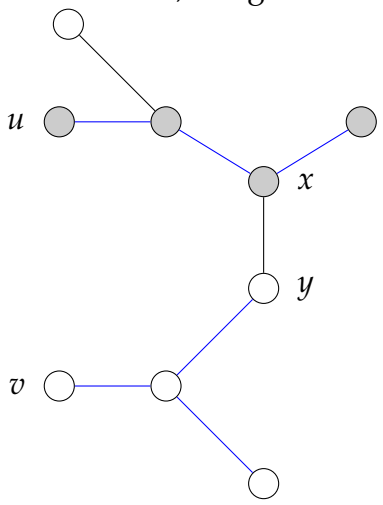
$\{B, D\}, \{C, F\}, \{F, G\}, \{D, G\}$ kreuzen den Schnitt. $\{D, G\}$ ist leichte Kante des Schnitts.

Satz 21

Sei $G = (V, E)$ ein zusammenhängender ungerichteter Graph mit Gewichtsfunktion w. Sei $A \subseteq E$ eine Teilmenge der Kantenmenge eines MST von G. Sei $(S, V \setminus S)$ ein Schnitt, der von A respektiert wird und $\{u, v\}$ eine leichte Kante des Schnitts. Dann ist $\{u, v\}$ eine sichere Kante von A.

Beweis: Sei $T = (V, E)$ ein MST mit $A \subseteq E$.

- Fall 1: $\{u, v\} \in E$. $A \cup \{\{u, v\}\} \subseteq E$, d.h. $\{u, v\}$ ist sicher für A.
- Fall 2: $\{u, v\} \notin E$. Da T ein Baum ist, gibt es einen eindeutigen Pfad von u nach v. u, v liegen auf verschiedenen Seiten des Schnitts.



Schnitt von grau und weiß

schwarz: Kanten von T

blau: Kanten von A in T

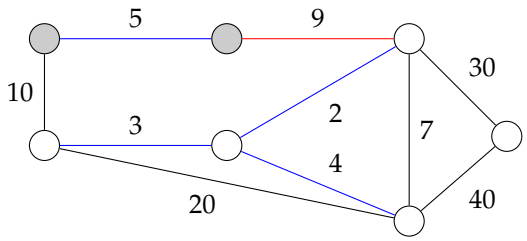
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$\{x, y\} \notin A$, da A den Schnitt respektiert. Sei $T' = (V, E')$ mit $E' = E \setminus \{\{x, y\} \cup \{u, v\}\}$. T' ist Spannbaum, denn er ist zusammenhängend und hat $n - 1$ Kanten.
T' ist auch MST, denn $w(\{u, v\}) \leq w(\{x, y\})$, da $\{u, v\}$ leichte Kante von $(S, V \setminus S)$ ist und $\{x, y\}$ ebenfalls kreuzt. $w(T') \leq w(T) - w(\{x, y\}) + w(\{u, v\}) \leq w(T)$. Da $A \subseteq E$ und $\{x, y\} \notin A$ gilt $A \subseteq E'$ und somit $A \cup \{u, v\} \in E'$. Daher ist $\{u, v\}$ sicher für A.

Algorithmus von Kruskal

- basiert auf GenericMST
- Regel in Zeile 3 sichere Kante zu finden: A ist Kantenmenge einer Menge von Bäumen. Die neue Kante ist die Kante mit minimalem Gewicht, die zwei Bäume verbindet.

Beispiel 51 Schnitt von grau und weiß. Die rote Kante mit dem Gewicht 9 ist die neue Kante mit minimalem Gewicht, die zwei Bäume verbindet.



Bemerkung 51 Behauptung: Eine so gewählte Kante $\{u, v\}$ ist eine sichere Kante für die Menge A.

Beweis:

- C_1, C_2 sind Bäume, die durch $\{u, v\}$ verbunden werden.
- Wir betrachten den Schnitt $(C_1, V \setminus C_1)$. $\{u, v\}$ ist leichte Kante für diesen Schnitt, da sie unter allen Kanten, die zwei Bäume verbinden, diejenige mit minimalem Gewicht ist. Daher ist sie auch die mit minimalem Gewicht, die C_1 mit $V \setminus C_1$ verbindet. Nach Satz 21 ist $\{u, v\}$ sicher.

algorithmische Umsetzung:

- Verwenden einer Union-Find-Struktur, um die Menge A abzuspeichern.
- Jede Menge enthält Knoten eines Baumes des aktuellen Waldes.

Algorithmus 67 **algorithm** MSTKruskal(G, w)

Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
1        A = ∅
2        for all Knoten v ∈ V[G] do
3            MakeSet(v)
4        end for
5        Sortiere die Kanten in E nach aufsteigendem Gewicht mit e_1 =
         {u_1, v_1}, ..., e_m = {u_m, v_m}.
6        for i = 1 to m do
7            if FindSet(u_i) ≠ FindSet(v_i) then
8                A = A ∪ {{u_i, v_i}}
9                Union(u_i, v_i)
10           end if
11       end for
12       return A
     end algorithm
```

Laufzeit:

- hängt von der Implementierung der Union-Find-Struktur ab
- verwenden Disjoint-Set-Forest mit Vereinigung nach Rang (und optional Pfadverkürzung)
- nach Zeilen:
  - Zeile 1: $O(1)$
  - Zeile 5: Sortieren in $O(|E| \cdot \log|E|)$
  - Zeilen 2–4: MakeSet auf alle Knoten in $O(|V|)$
  - Zeilen 6–11: Find-Set-Operationen in $O(|E|)$ und Union-Operationen in $O(|E| \cdot \alpha(|V|))$, also $O(|V| + |E| \cdot \log|V|)$
- wegen $|E| \leq |V|^2$[5]: $O(|V| + |E| \cdot \log(|V|^2)) = O(|V| + |E| \cdot \log|V|)$

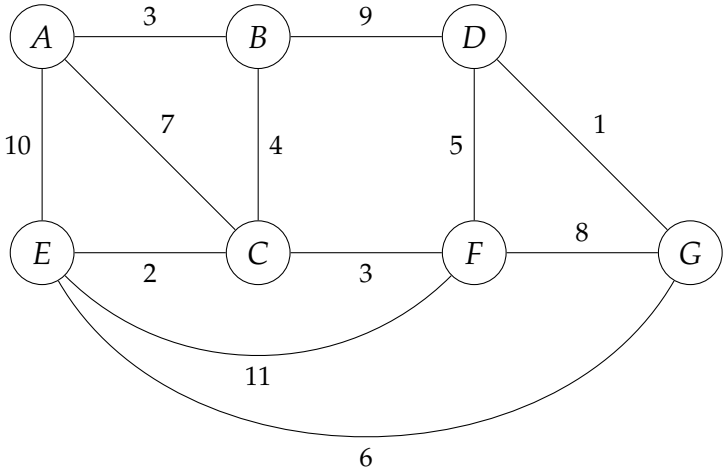
Algorithmus von Prim

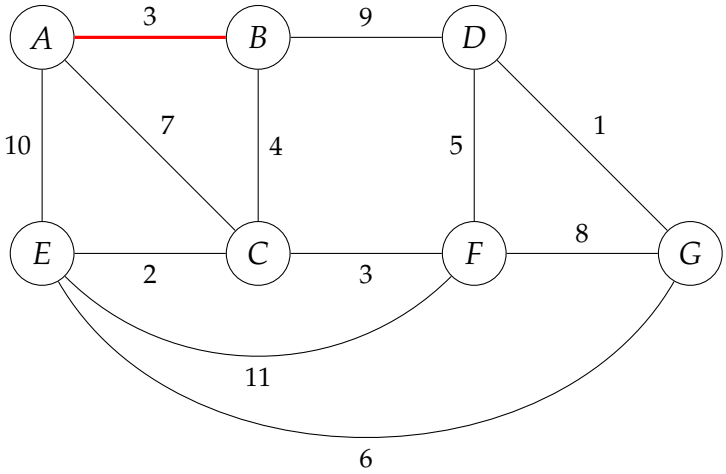
- beruht ebenfalls auf GenericMST
- Die Kanten der Menge A bilden stets einen Baum.
- In jedem Schritt wird eine leichte Kante für den Schnitt $(A, V \setminus A)$ hinzugefügt. Sie verbindet A mit einem isolierten Knoten $G' = (V, A)$.
- Die Kante ist nach Satz 21 sicher.

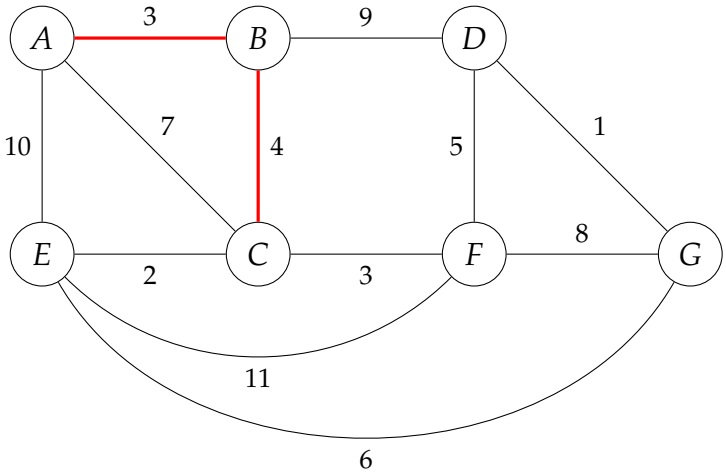
Beispiel 52 Ausführung des Algorithmus von Prim auf dem Graphen (beginnend bei A)

[5] Diesen Schritt benötigen wir, um die Laufzeit des Sortierens zu integrieren.

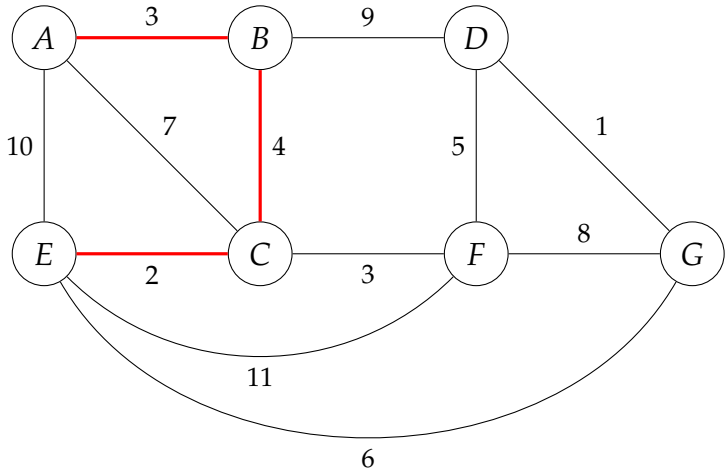
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

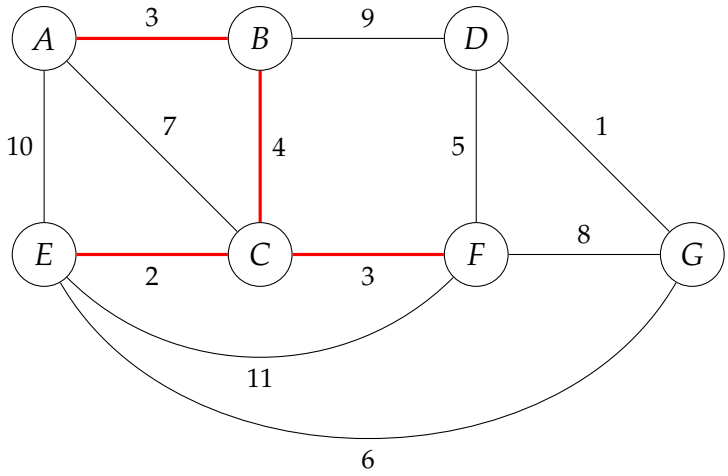


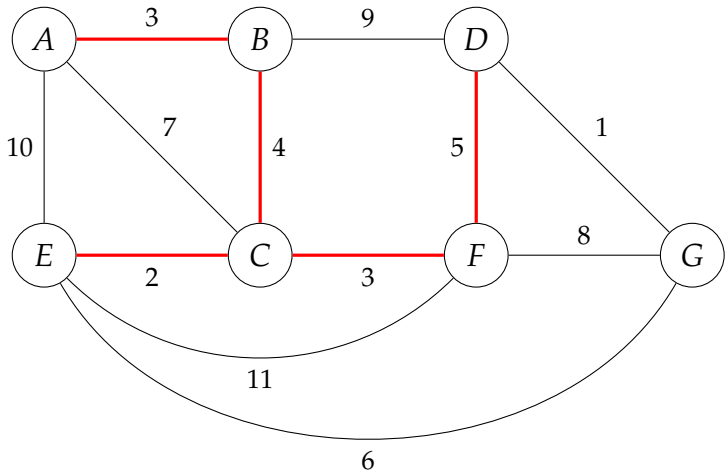




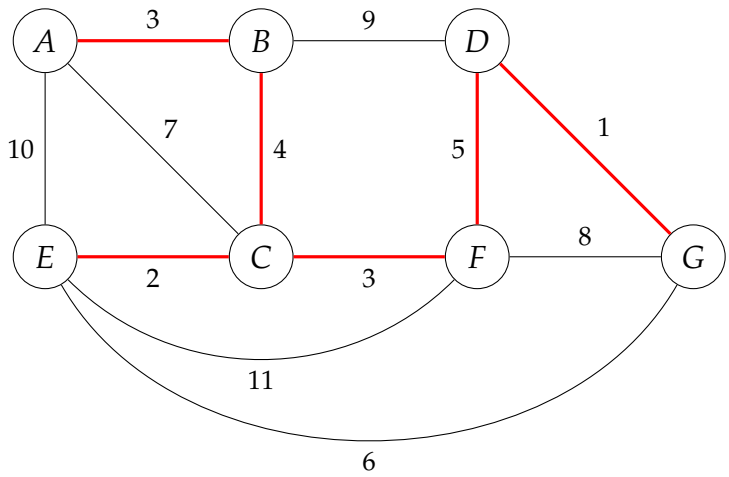
Downloaded by Maja Müller (blackcatgirl@freenet.de)







This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Bemerkung 52 Bei paarweise verschiedenen Gewichten (oder wenn alle gleichen Gewichte im MST vorkommen), ensteht mit diesem Algorithmus unabhängig vom Startknoten der gleiche Baum.

algorithmische Umsetzung:

- Alle Nachbarn der Knoten, die durch Kanten aus A verbunden sind, sind Kandidaten, als nächster Knoten aufgenommen zu werden.
- Diese Nachbarn werden in einer Priority-Queue gespeichert. Die Priorität ist das kleinste Kantengewicht, mit dem diese Knoten aus A erreichbar sind.

Beispiel 53 Erneutes Aufgreifen von Beispiel 52 mit Startknoten A:

Knoten	**Priority-Queue**	**Baum**
A	$B3, C7, E10$	$\emptyset$
B	$C4, E10, D9$	$\overline{AB}$
C	$E2, D9, F3$	$\overline{BC}$
E	$D9, F3, G6$	$\overline{CE}$
F	$D5, G6$	$\overline{CF}$
D	$G1$	$\overline{FD}$
G	$\emptyset$	$\overline{DG}$

Algorithmus 68

```
algorithm MSTPrim(G, w, r)
1     for all Knoten u ∈ V[G] do
2         key[u] = ∞
3         π[u] = NIL
4     end for
```


Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
5           key[r] = 0
6           Q = V[G]
7           while Q ≠ ∅ do
8               u = ExtractMin(Q)
9               for all Knoten v ∈ AL[u] do
10                  if v ∈ Q and w(u, v) < key[v] then
11                      π[v] = u
12                      key[v] = w(u, v)
13                  end if
14              end for
15          end while
        end algorithm
```

Bemerkung 53 Zeilen 9–14: Aktualisiert den Key und π für alle Knoten, die mit u benachbart sind, sich aber nicht in A befinden. Zeile 12 ist ein Decrease-Key.

Laufzeit: in Q höchstens $|V|$ Elemente

Operation	**binärer Heap**	**Fibonacci-Hep**
$\|V\|$ Mal Extract-Min	$\|V\| \log \|V\|$	$\|V\| \log \|V\|$
$\|E\|$ Mal Decrease-Key	$\|E\| \log \|V\|$	$\|E\| \cdot c$
Gesamt	$\|E\| \log \|V\|$	$\|E\| + \|V\| \log \|V\|$

11 Dynamische Programmierung

- Algorithmenentwurfstechnik
- Es werden Lösungen von Teilproblemen kombiniert.
- Teile und Herrsche: Die Teilprobleme werden in unabhängige Teilprobleme zerlegt, diese gelöst und daraus eine Gesamtlösung erstellt.
- dynamische Programmierung: Die Teilprobleme sind nicht unabhängig voneinander, d.h. sie benutzen ihrerseits gemeinsame Teilprobleme (Teile und Herrsche würde diese dann mehrmals lösen).
- Jedes Teilproblem wird gelöst und in einer Tabelle gespeichert, um später darauf zugreifen zu können.
- Dynamische Programmierung wird für Optimierungsprobleme angewendet (Suche nach *einer* optimalen Lösung).

Herangehensweise:

1. Charakterisierung der Struktur einer optimalen Lösung
2. Wert einer optimalen Lösung rekursiv definieren

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

3. Wert einer optimalen Lösung berechnen (bottom up)
4. eine optimale Lösung aus der berechneten Information rekonstruieren

11.1 Längste gemeinsame Teilfolge

- Vergleich von DNA-Strängen
- DNA-Strang ist ein String über $\{A, C, G, T\}$
- Wie ähnlich sind zwei DNA-Strings? Wie definiert man Ähnlichkeit?
  - Möglichkeit 1: Ein String ist ein Teilstring des anderen.
  - Möglichkeit 2: Die Zahl der möglichen Veränderungen, um einen in den anderen zu überführen, ist klein.
  - Möglichkeit 3: lange gemeinsame Teilfolgen (hier verwendet)

Beispiel 54 Wir betrachten zwei Folgen und ihre längste Teilfolge

A	C	G	A	A	C	T	A	C	T

G	C	A	A	T	C	T	T	A	T

Definition 24 Sei $x = x_1 ... x_n$ ein Wort über dem Alphabet Σ mit $x_i \in \Sigma$. $z = z_1 z_2 ... z_k$ heißt *Teilwort* von x genau dann, wenn es eine Folge $(i_1, ..., i_k)$ von Indizes mit $i_1 < i_2 < ... < i_k$ gibt, sodass für alle $j \in \{1, ..., k\}$ gilt $x_{i_j} = z_j$.

Definition 25 Ein Wort z heißt *gemeinsames Teilwort* zweier Wörter x und y genau dann, wenn z sowohl Teilwort von x als auch Teilwort von y ist.

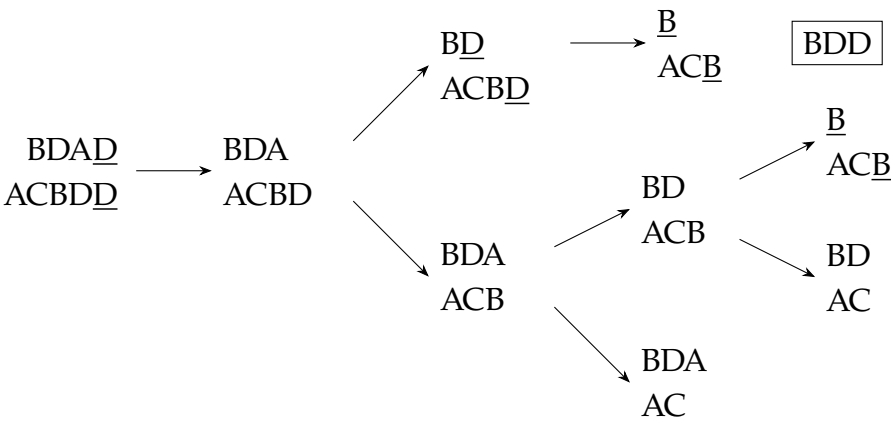
Algorithmisches Problem: Longest-Common-Subsequence (LCS)

- Input: zwei Wörter über dem Alphabet Σ
- Output: gemeinsames Teilwort von x und y mit maximaler Länge

Phase 1

Beispiel 55 Wir betrachten die Folgen BDAD und ACBDD.

Downloaded by Maja Müller (blackcatgirl@freenet.de)



Satz 22 Seien $x = x_1...x_m$ und $y = y_1...y_n$ und $z = z_1...z_k$ eine LCS von x und y.

1. Ist $x_m = y_n$, so gilt $z_k = x_m = y_n$ und $z_1...z_{k-1}$ ist LCS von $x_1...x_{m-1}$ und $y_1...y_{n-1}$.
2. Ist $x_m \neq y_n$, dann folgt $z_k \neq x_m$, dass z LCS von $x_1...x_{m-1}$ und $y_1...y_n$ ist.
3. Ist $x_m \neq y_n$, dann folgt $z_k \neq y_n$, dass z LCS von x und $y_1...y_{n-1}$ ist.

Beweis:

1. $z_k = x_m = y_n$. Annahme: $z_k \neq x_m$. Dann ist z x_m gemeinsames Teilwort von x und y. Daraus folgt $|zx_m| > |z|$. Das widerspricht z ist LCS von x und y. $z_k \neq y_n$ wird analog gezeigt.
   Es ist noch zu zeigen, dass $z_1...z_{k-1}$ LCS ist. $z_1...z_{k-1}$ ist Teilwort von $x_1...x_{m-1}$ und $y_1...y_{n-1}$, dann ist $|z_1...z_{k-1}| = k-1$. Annahme: Es gibt ein Teilwort w von $x_1...x_{m-1}$ und $y_1...y_{n-1}$ mit $|w| > k-1$. Dann ist wx_m Teilwort von x und y mit $|wx_m| > k$. Das ist ein Widerspruch.
2. elementar.
3. elementar.

Phase 2

$c(i,j)$ ist die Länge der LCS von $x_1...x_i$ und $y_1...y_j$.

$$c(i,j) = \begin{cases} 0 & i = 0 \text{ oder } j = 0 \\ c(i-1,j-1)+1 & i,j > 0 \text{ und } x_i = y_j \\ \max\{c(i,j-1), c(i-1,j)\} & i,j > 0 \text{ und } x_i \neq y_j \end{cases}$$

Bemerkung 54

- Der rekursive Algorithmus für $c(n,m)$ hat exponentielle Laufzeit.
- Es gibt nur $O(m \cdot n)$ viele verschiedene Teilprobleme von LCS($x_1...x_m$, $y_1...y_n$).

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Phase 3

Eingabe: $x = x_1 ... x_m$ und $y = y_1 ... y_n$. Im Programm:

$$c[0...m, 0...n] = \begin{matrix} c(0,0) & \dots & c(0,n) \\ \vdots & \ddots & \vdots \\ c(m,0) & \dots & c(m,n) \end{matrix},$$

$$b[1...m, 1...n] = \begin{matrix} b(1,1) & \dots & b(1,n) \\ \vdots & \ddots & \vdots \\ b(m,1) & \dots & b(m,n) \end{matrix}$$

$b(i,j)$ zeigt auf den c-Eintrag, der bei der Berechnung von $c(i,j)$ bestimmend war.

Algorithmus 69

```
algorithm LCSLength(x, y)
1    m = length[x]
2    n = length[y]
3    for i = 1 to m do
4        c[i, 0] = 0
5    end for
6    for j = 1 to n do
7        c[0, j] = 0
8    end for
9    for i = 1 to m do
10       for j = 1 to n do
11           if x_i = x_j then
12               c[i, j] = c[i − 1, j − 1] + 1
13               b[i, j] = ↖ (up left)
14           else if c[i − 1, j] ≥ c[i, j − 1] then
15               c[i, j] = c[i − 1, j]
16               b[i, j] = ↑ (up)
17           else
18               c[i, j] = c[i, j − 1]
19               b[i, j] = ← (left)
20           end if
21       end for
22   end for
23   return b, c
end algorithm
```

Laufzeit: $O(n \cdot m)$, da pro Tabelleneintrag konstante Zeit

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 56 Im Beispiel wird von links oben dem Algorithmus folgend die Tabelle aufgebaut. Von rechts unten wird der Weg zum längsten Teilwort über die Pfeilinformationen nachvollzogen und grau markiert. Die Buchstaben, die das Teilwort ausmachen, sind blau markiert.

	Y	C	D	A	A	D	C	A	B
X	0, -	0, -	0, -	0, -	0, -	0, -	0, -	0, -	0, -
A	0, -	0, -	0, -	1, ↖	1, ↖	1, ←	1, ←	1, ↖	1, ←
D	0, -	0, -	1, ↖	1, ↑	1, ↑	2, ↖	2, ←	2, ←	2, ←
D	0, -	0, -	1, ↖	1, ↑	1, ↑	2, ↖	2, ↑	2, ↑	2, ↑
B	0, -	0, -	1, ↑	1, ↑	1, ↑	2, ↑	2, ↑	2, ↑	3, ↖
C	0, -	1, ↖	1, ↑	1, ↑	1, ↑	2, ↑	3, ↖	3, ←	3, ↑
B	0, -	1, ↑	1, ↑	1, ↑	1, ↑	2, ↑	3, ↑	3, ↑	4, ↖
A	0, -	1, ↑	1, ↑	2, ↖	2, ↖	2, ↑	3, ↑	4, ↖	4, ↑

Phase 4

Konstruktion der LCS

Algorithmus 70

```
algorithm PrintLCS(b, X, i, j)
1     if i = 0 or j = 0 then
2         return
3     end if
4     if b[i, j] = ↖ then
5         PrintLCS(b, X, i − 1, j − 1)
6         write(X_i)
7     else if b[i, j] = ↑ then
8         PrintLCS(b, X, i − 1, j)
9     else
10        PrintLCS(b, X, i, j − 1)
11    end if
end algorithm
```

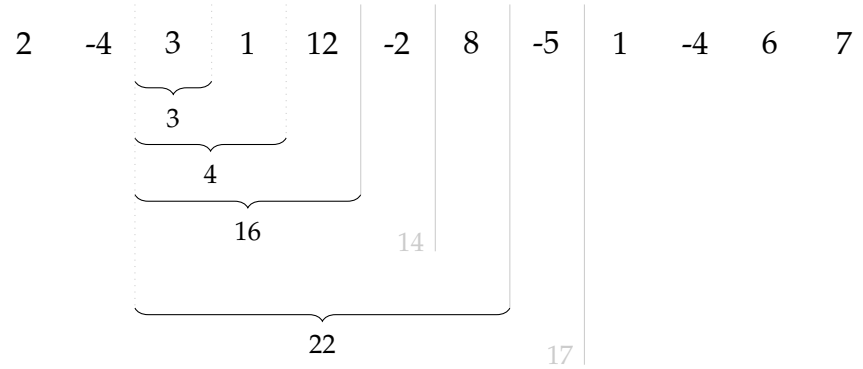
Laufzeit: $O(n + m)$

11.2 Summe maximaler Teilfolgen

Beispiel 57 In der folgenden Grafik ist beispielhaft eine Folge von Zahlen abgebildet, in der die maximalen Teilfolgen bestimmt werden sollen. Dabei

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

sind einige Teilintervalle durch geschweifte Klammern dargestellt und Endintervalle durch durchgezogene Linien.



Gegeben sei $A[1...n]$ mit $a_i \in \mathbb{Z}$. Gesucht sind i, j, sodass $\sum_{k=i}^{j} A[k]$ maximal ist. Das Ziel ist eine Laufzeit von $O(n)$.

Lösungsidee: Das maximale Teilintervall von $A[1], ..., A[k]$ kann die Gestalt $A[i], ..., A[j]$ mit $1 \leq i \leq j < k$ haben, dann ist das maximale Teilintervall von $A[1], ..., A[k-1]$, oder die Gestalt $A[i], ..., A[k]$ mit $1 \leq i \leq k$, dann ist es eine Vereinigung eines maximalen Endintervalls von $A[1], ..., A[k-1]$ (das im Fall $i = k$ verschwindet mit $A[k]$).

Wir definieren $a_T(k)$ als den Anfang eines maximalen Teilintervalles in $A[1]...A[k]$, $e_T(k)$ als Ende eines maximalen Teilintervalls und $s_T(k)$ die Summe.

Außerdem definieren wir $a_E(k)$ als den Anfang eines maximalen Endintervalls in $A[1], ..., A[k]$. Achtung: Hier ist $a_E(k) = k+1$ erlaubt, nämlich wenn alle echten Endintervalle eine negative Summe bilden. Ferner sei $s_E(k)$ die Summe des maximalen Endintervalles in $A[1], ..., A[k]$. Achtung: Es gilt immer $s_E(k) \geq 0$.

Rekursion:

- $k = 1$: $a_T(1) = e_T(1) = 1$ und $s_T(1) = A[1]$.

$$a_E(1) = \begin{cases} 1 & A[1] \geq 0 \\ 2 & A[1] < 0 \end{cases}$$

$$s_E(1) = \begin{cases} A[1] & A[1] \geq 0 \\ 0 & A[1] < 0 \end{cases}$$

- $k - 1 \to k$ $(k = 2, ..., n)$:
  - Bestimmung eines maximalen Teilintervalls
    - ⋆ Fall 1: $s_T(k-1) > s_E(k-1) + A[k]$

$$(a_T(k), e_T(k), s_T(k)) = (a_T(k-1), e_T(k-1), s_T(k-1))$$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

⋆ Fall 2: $s_T(k-1) \leq s_E(k-1) + A[k]$

$$a_T(k) = a_E(k-1)$$
$$e_T(k) = k$$
$$s_T(k) = s_E(k-1) + A[k]$$

– Bestimmung eines maximalen Endintervalls

⋆ Fall 1: $s_E(k-1) + A[k] \leq 0$ (wegen $s_E(k-1) \geq 0$ folgt dann $A[k] \leq 0$)

$$a_E(k) = k+1$$
$$s_E(k) = 0$$

⋆ Fall 2: $s_E(k-1) + A[k] > 0$

$$a_E(k) = a_E(k-1)$$
$$s_E(k) = s_E(k-1) + A[k]$$

Die Bestimmung von $a_T(k)$, $e_T(k)$, $s_T(k)$, $a_E(k)$, $s_E(k)$ für $k = 1, ..., n$ ist in $O(n)$ Zeit möglich. Ein optionales Paar ist letztendlich $(i,j) = (a_T(k), e_T(k))$.

11.3 Anzahl der aussagenlogischen Klammerungen

Gegeben sei ein String $w_1 o_1 w_2 o_2 ... o_{n-1} w_n$ mit $w_i \in \{0,1\}$ und $o_i \in \{\wedge, \vee, \dot{\vee}\}$. Gesucht ist die Anzahl der Klammerungen, bei denen der String den Wahrheitswert 1 ergibt.

Für $1 \leq j \leq n$:

$$k(i,j) = (k_1(i,j), k_0(i,j), k_*(i,j))$$

mit dem Teilstring $T_{ij} = w_i o_i ... o_{j-1} w_j$ und

- $k_1(i,j)$ als die Anzahl der Klammerungen des Teilstrings T_{ij}, die für T_{ij} den Wert 1 ergeben.
- $k_0(i,j)$ als die Anzahl der Klammerungen des Teilstrings T_{ij}, die für T_{ij} den Wert 0 ergeben.
- $k_*(i,j) = k_1(i,j) + k_0(i,j)$ als die Anzahl aller Klammerungen

Beispiel 58 Für $1 \wedge 0 \vee 1$ ergeben sich zwei Klammerungen: $(1 \wedge 0) \vee 1$ und $1 \wedge (0 \vee 1)$. Beide liefern den Wahrheitswert 1.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 59 Wir betrachten den Ausdruck $1 \wedge 0 \vee 1 \wedge 0$. Es ergeben sich die Klammerungen:

- $1 \wedge (0 \vee 1 \wedge 0)$
- $(1 \wedge 0) \vee (1 \wedge 0)$
- $(1 \wedge 0 \vee 1) \wedge 0$

Rekursion für $i = 1, ..., n$:

$$k_1(i,i) = \begin{cases} 1 & w_i = 1 \\ 0 & w_i = 0 \end{cases}$$
$$k_0(i,i) = \begin{cases} 0 & w_i = 1 \\ 1 & w_i = 0 \end{cases}$$
$$k_*(i,i) = 1$$

Sei nun $1 \leq i < j \leq n$.

$$k_*(i,j) = \sum_{l=i}^{j-1} k_*(i,l) \cdot k_*(l+1, j-1)$$
$$k_1(i,j) = \sum_{l=i}^{j-1} h_1(o_e, k(i,l), k(l+1,j))$$
$$h_1(\wedge, k(i,l), k(l+1,j)) = k_1(i,l) \cdot k_1(l+1,j)$$
$$h_1(\vee, k(i,l), k(l+1,j)) = k_1(i.l) \cdot k_1(l+1,j) + k_1(i.l) \cdot k_0(l+1,j) + k_0(i,l) \cdot k_1(l+1,j)$$
$$h_1(\dot{\vee}, k(i,l), k(l+1,j)) = k_1(i.l) \cdot k_0(l+1,j) + k_0(i,l) \cdot k_1(l+1,j)$$

Beispiel 60 Wir betrachten den Ausdruck $1 \wedge 0 \vee 1 \vee 0 \dot{\vee} 0 \wedge 1 \wedge 1$.

Beispielhafte Berechnungen:

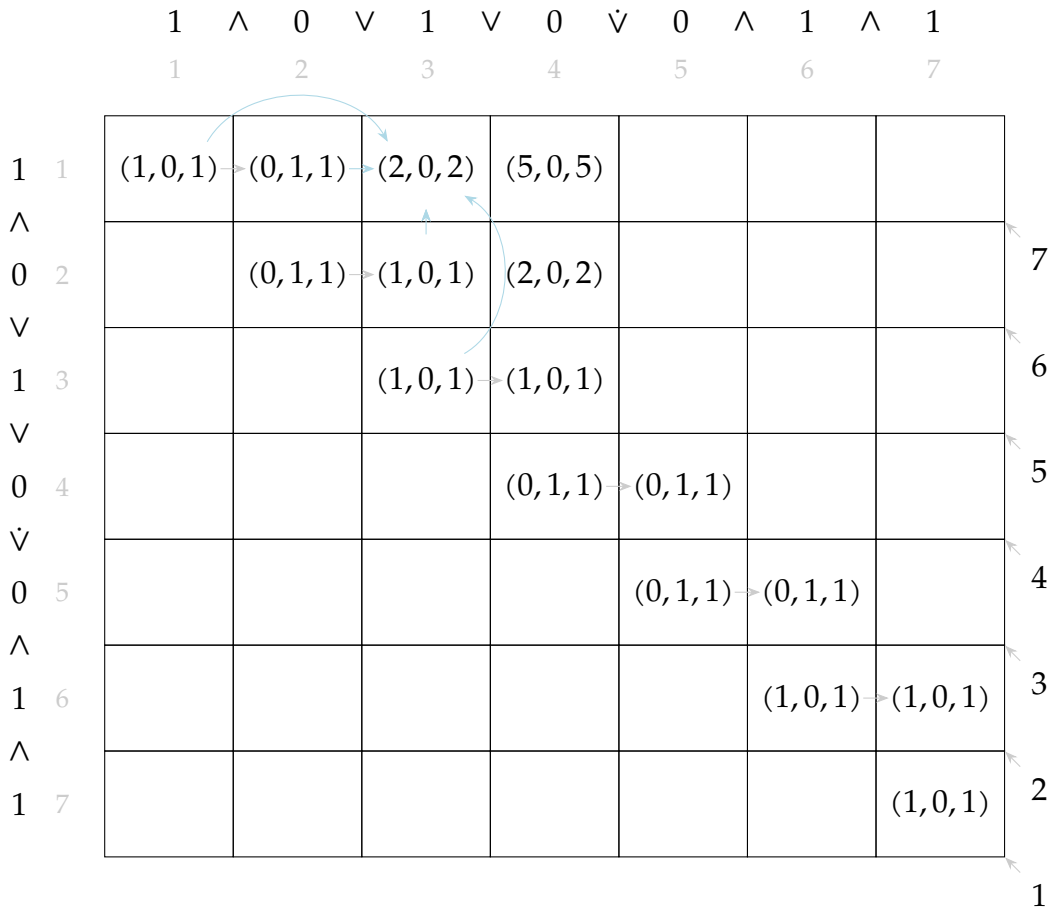
- $k(1,3)$ für $1 \overset{1.}{\wedge} 0 \overset{2.}{\vee} 1$
  1. $(1,1) \wedge (2,3) = (1,0,1)$
  2. $(1,2) \vee (3,3) = (1,0,1)$

  $(1,0,1) + (1,0,1) = (2,0,2)$
- $k(2,4)$ für $0 \overset{1.}{\vee} 1 \overset{2.}{\vee} 0$
  1. $(2,2) \vee (3,4) = (1,0,1)$
  2. $(2,3) \vee (4,4) = (1,0,1)$

  $(1,0,1) + (1,0,1) = (2,0,2)$
- $k(1,4)$ für $1 \overset{1.}{\wedge} 0 \overset{2.}{\vee} 1 \overset{3.}{\vee} 0$
  1. $(1,1) \wedge (2,4) = (1,0,1) \wedge (2,0,2) = (2,0,2)$
  2. $(1,2) \vee (3,4) = (0,1,1) \vee (1,0,1) = (1,0,1)$
  3. $(1,3) \vee (4,4) = (2,0,2) \vee (0,1,1) = (2,0,2)$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 23 Klammerungen zum Beispiel 60 (wenige Schritte)



$$(2,0,2) + (1,0,1) + (2,0,2) = (5,0,5)$$

11.4 Dame auf dem Schachbrett

Wir betrachten ein 8×8-Schachbrett mit einer Dame, die sich nur nach rechts, unten und diagonal rechts unten bewegen kann. Gewonnen hat, wer die Dame in das Feld rechts unten (8H) führt.

Wir setzen in ein Feld eine 0, wenn von dort aus keine Gewinnstrategie existiert (der Gegner gewinnt), und 1 anderenfalls. Die ist in Abbildung 24 dargestellt. Dabei beginnen wir in 8H mit einer 0 und setzen alle nicht gesetzten Felder in ihrer Reihe, Spalte und Diagonale auf 1. Ein Feld bekommt eine 0, wenn alle erreichbaren Felder 1-Felder sind.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Abbildung 24 Schachbrett als Gewinnmatrix

	A	B	C	D	E	F	G	H
1	1	1	1	0	1	1	1	1
2	1	1	1	1	1	1	1	1
3	1	1	1	1	0	1	1	1
4	0	1	1	1	1	1	1	1
5	1	1	0	1	1	1	1	1
6	1	1	1	1	1	1	0	1
7	1	1	1	1	1	0	1	1
8	1	1	1	1	1	1	1	0

Downloaded by Maja Müller (blackcatgirl@freenet.de)

12 Material aus den Übungsterminen

12.1 Algorithmik

Beispiel 61 „Wahl des Klassensprechers“ (Wahlsieger mit mehr als der Hälfte der Stimmen ermitteln)

- Strichliste anlegen (unsortiert):

```
1  for all Zettel do
2      if Name ∈ Liste then                          ▷ ∈ O(n²)
3          Strich machen
4      else
5          Aufschreiben
6      end if
7  end for
```

- Strichliste anlegen (sortiert):

```
1  for all Zettel do
2      if Name ∈ Liste then    ▷ Array: O(log(n)), verk. Liste: O(n)
3          Strich machen
4      else                              ▷ Array: O(n), verk. Liste: O(1)
5          Aufschreiben
6      end if
7  end for
```

- Mit anfangs leerem Stapel:

```
1  for all Zettel do
2      if Stack empty then
3          StackPush(Zettel)
4      else if x = top(Stack) then
5          StackPush(Zettel)
6      else
7          StackPop()
8      end if
9  end for
10 if Stack empty then
11     return ∅
12 else
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
13          y = StackPop()
14          if count(y ∈ Zettel) > n/2 then
15              return y
16          else
17              return ∅
18          end if
19      end if
```

- Laufzeit:
  - ⋆ For-Schleife: $O(n)$
  - ⋆ Zählen des Kandidaten in den Zetteln: $O(n)$
  - ⋆ insgesamt: $O(n)$
- Korrektheit:
  - ⋆ Invariante: Stapel hat nur gleiche Elemente (Beweis per Induktion)
  - ⋆ bei keinem Mehrheitselement: korrekt, wegen Durchzählen
  - ⋆ Mehrheitselement y: Wir nehmen an, es sei kein y im Stapel. Dann wurde jedes y mit einem Element ungleich y aus dem Stapel entfernt. Dies ist ein Widerspruch zur Mehrheit.

12.2 Landau-Notation

Beispiel 62 $(n+a)^b \in \Theta(n^b)$ mit $a \geq 0, b \geq 1$.

- Ω ist offensichtlich
- O: $(n+a)^b \leq (n(1+a))^b = (1+a)^b n^b \leq c \cdot n^b$
- alternativ für O: $(n+a)^b \overset{n \geq a}{\leq} (n+n)^b = (2n)^b = 2^b n^b$ ($c = 2^b, n_0 = \sqrt{a}$)

Bemerkung 55 $f \notin O(g) \Leftrightarrow \forall c > 0, n_0 > 0 : \exists n \geq n_0 : f(n) > c \cdot g(n)$ und damit $f \in \omega(g) \Rightarrow f \notin O(g)$.

Beispiel 63 $3n^3 \in \Omega(100n^2)$

- $3n^3 \geq 3n^2 = \frac{3}{100} \cdot 100n^2$ mit $c = \frac{3}{100}, n_0 = 1$
- $3n^3 \notin O(100n^2)$, wir prüfen auf $3n^3 \in \omega(100n^2)$, also zu zeigen: $\forall c > 0, n_0 > 0 : \exists n \geq n_0 : 3n^3 \geq c \cdot 100n^2$, und da $3n^3 \geq c \cdot 100n^2 \Leftrightarrow n \geq \frac{100}{3} \cdot c$. Da für $n > \frac{100}{3} \cdot c$ die Aussage erfüllt ist, liegt $3n^3 \in \omega(100n^2)$.

Beispiel 64 $2^n \in \Omega(2^{\frac{n}{2}})$ und $2^n \in \omega(2^{\frac{n}{2}})$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Ω ist offensichtlich
- ω: $\forall c > 0, n_0 > 0 : \exists n \geq n_0 : 2^n \geq c2^{\frac{n}{2}}$, also $n \geq \lceil 2 \cdot \log_2(c) \rceil$

Beispiel 65 $3^{\frac{n}{2}+100} \in o(3^n)$

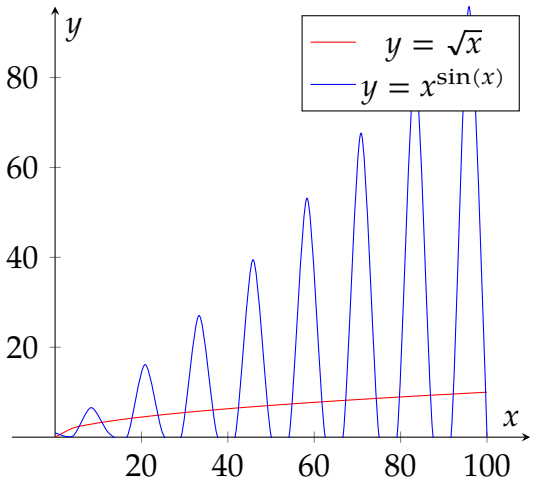
Tabelle 2 Elementbeziehungen zwischen Landau-Mengen

	O	Ω	Θ	o	ω
$f \in o(g)$	$\in$	$\notin$	$\notin$	–	$\notin$
$f \in \Theta(g)$	$\in$	$\in$	–	$\notin$	$\notin$
$f \in \omega(g)$	$\notin$	$\in$	$\notin$	$\notin$	–

Bemerkung 56 $\omega(g) \cap o(g) = \emptyset$, da $f \in \omega(g) \Rightarrow f \notin O(g) \Rightarrow f \notin o(g)$.

Beispiel 66 $\sqrt{n} \notin O(n^{\sin n})$ und $\sqrt{n} \notin \Omega(n^{\sin n})$

Abbildung 25 Visualisierung von Beispiel 66 ($\sqrt{n}$ und $n^{\sin n}$)



$\notin O$: zu zeigen: $\forall c > 0, n_0 > 0 : \exists n \geq n_0 : \sqrt{n} > c \cdot n^{\sin n}$. $\forall n_0 : \exists n \geq n_0 : 0 \leq \sin < \frac{1}{2}$ und $\forall c > 0, n_0 > 0 : \exists n \geq n_0 : n^{\frac{1}{2}-x} > c$ (für $0 < x < \frac{1}{2}$). Daraus folgt $\forall c > 0, n_0 > 0 : \exists n \geq n_0 : n^{\frac{1}{2}-\sin n} > c$.

Bemerkung 57 $f \in O(g) \Leftrightarrow g \in \Omega(f)$.

Beweis:

$$
\begin{aligned}
f \in O(g) &\Leftrightarrow \exists c > 0, n_0 > 0 : \forall n \geq n_0 : f(n) \leq c \cdot g(n) \\
&\Leftrightarrow \exists c > 0, n_0 > 0 : \forall n \geq n_0 : \frac{1}{c} \cdot f(n) \leq g(n) \\
&\Leftrightarrow \exists d > 0, n_0 > 0 : \forall n \geq n_0 : d \cdot f(n) \leq g(n) \\
&\Leftrightarrow g \in \Omega(f)
\end{aligned}
$$

□

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel 67 $\max\{f(n), g(n)\} \in \Theta(f(n) + g(n))$

- O: $\max\{f, g\} \leq f + g$
- Ω: $\max\{f, g\} \geq \frac{1}{2} \cdot (f + g)$

Beispiel 68 $\log^{40}(n) \in O(n^{\frac{1}{4}})$. Zu zeigen: $\log^{40}(n) \leq c \cdot n^{\frac{1}{4}}$:

$$40 \log\log(n) \leq \frac{1}{4} \log(n) \text{ für großes } n_0$$

$$\log(\log^{40}(n)) \leq \log(n^{\frac{1}{4}})$$

$$\log^{40}(n) \leq n^{\frac{1}{4}}$$

Alternativ: $\log^k(n) \in O(n^p)$ mit $k \geq 1, p > 0$: $\log(n) = \log(n^{\frac{k}{p}\frac{p}{k}}) = \frac{k}{p} \log(n^{\frac{p}{k}}) \leq \frac{k}{p} \cdot n^{\frac{p}{k}}$, durch Potenzieren mit k: $\log^k(n) \leq \left(\frac{k}{p}\right)^k n^p$.

Beispiel 69 $n^{2(-1)^n} \notin \{O, \Omega, \Theta\}(n)$. Für O: Wir wählen n gerade und $n \geq \max\{c, n_0\}$. Für Ω: Wir wählen n ungerade und $n \geq \max\left\{n_0, \sqrt[3]{\frac{1}{c}}\right\}$.

12.3 Rekursionsgleichungen

Beispiel 70 Sei $T(1) = 1$ und $T(n) = T(n-1) + n$.

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + n - 1 + n \\ &= T(n-3) + n - 2 + n - 1 + n \\ &\vdots \\ &= T(1) + 2 + 3 + \ldots + n - 1 + n \\ &= \sum_{i=1}^{n} i = \frac{n(n+1)}{2} \in \Theta(n^2) \end{aligned}$$

Beispiel 71 Sei $T(1) = 1$ und $T(n) = 2T\left(\frac{n}{2}\right) + 1$.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + 1 \\ &= 2\left(2T\left(\frac{n}{4}\right) + 1\right) + 1 \\ &= 2\left(2\left(T\left(\frac{n}{8}\right) + 1\right) + 1\right) + 1 \\ &\vdots \end{aligned}$$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$= 2^i \cdot T\Big(\frac{n}{2^i}\Big) + \sum_{k=0}^{i-1} 2^k$$
$$= 2^{\lceil \log(n) \rceil} T(1) + 2^{\lceil \log(n) \rceil} - 1 \in \Theta(n)$$

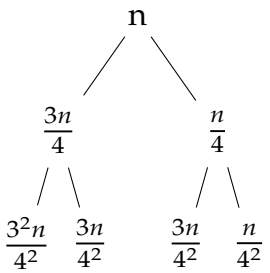
Beispiel 72 Sei $T(2) = 1$ und $T(n) = T(\sqrt{n}) + 1$.

$$\begin{aligned} T(n) &= T(\sqrt{n}) + 1 \\ &= T(\sqrt[4]{n}) + 2 \\ &= T(\sqrt[8]{n}) + 3 \\ &\vdots \\ &= T(n^{\frac{1}{2^i}}) + i \\ &= T(2) + \lceil \log\log(n) \rceil \in \Theta(\log\log(n)) \end{aligned}$$

Beispiel 73 Sei $T(1) = 1$ und $T(n) = T(\sqrt{n}) + \log(n)$.

$$\begin{aligned} T(n) &= T(\sqrt{n}) + \log(n) \\ &= T(\sqrt[4]{n}) + \frac{1}{2}\log(n) + \log(n) \\ &= T(\sqrt[8]{n}) + \frac{1}{4}\log(n) + \frac{1}{2}\log(n) + \log(n) \\ &= T(\sqrt[2^i]{n}) + \log(n) \cdot \sum_{k=0}^{i} \frac{1}{2^k} \\ &= T(\sqrt[2^{\log(n)}]{n}) + \log(n) \cdot \Big(1 + 1 - \frac{1}{2^{\log(n)}}\Big) \\ &= T(\sqrt[2^{\log(n)}]{n}) + \log(n) \cdot \Big(2 - \frac{2}{n}\Big) \in \Theta(\log(n)) \end{aligned}$$

Beispiel 74 Rekursionsbaum für $T(n) = T\big(\frac{3}{4}n\big) + T\big(\frac{1}{4}n\big) + n$. Anfang des Baumes:



$h_2 \cdot n \geq T(n) \geq h_1 \cdot n$. Bestimmen von h_1 als Abbruchstiefe für den $\frac{1}{4}$-Zweig ($4^{-h_1} n \leq 1$, also $h_1 = \lceil \log_4(n) \rceil$). Bestimmen von h_2 als Abbruchstiefe, wenn sich der Baum fortsetzen würde, bis der $\frac{3}{4}$-Zweig abbricht

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$(h_2 = \lceil \log_{\frac{4}{3}}(n) \rceil)$. Daraus ergibt sich $n\lceil \log_{\frac{4}{3}}(n) \rceil \geq T(n) \geq n\lceil \log_4(n) \rceil$ und deshalb $T(n) \in \Theta(n \log(n))$.

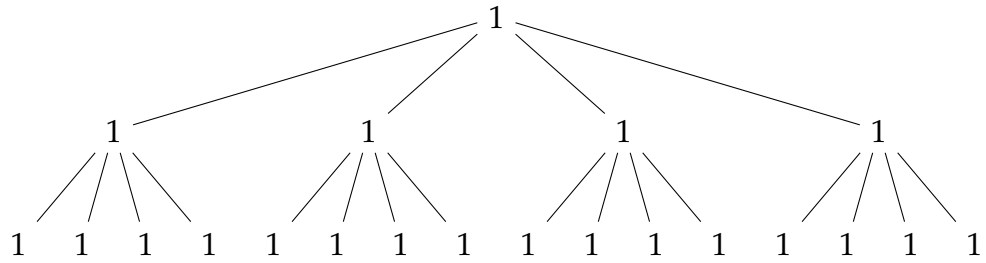
Bemerkung 58 Bei Rekursionsgleichungen der Form $T(n) = T(\alpha n) + T(\beta n) + n$, gilt:

- Fall 1: $\beta = 1 - \alpha$, dann $\Theta(n \log(n))$.
- Fall 2: $\beta + \alpha < 1$, dann $\Theta(n)$.
- Fall 3: $\alpha + \beta > 1$, dann $\Theta(n^c)$ mit $c > 1$.

Das kann sogar verallgemeinert werden, um statt n ein $f(n)$ zu verwenden.

Beispiel 75 $T(n) = 4 \cdot T(\sqrt{n}) + 1$. Wir wählen $m = \log(n), S(m) = T(2^m)$. Daher ist $S(m) = 4S\left(\frac{m}{2}\right) + 1$. Das fällt in den ersten Fall des Mastertheorems, also $S(m) \in \Theta(m^2)$. Durch Resubstitution ist $T(n) \in \Theta(\log^2(n))$.

Alternativer Lösungsweg mit Rekursionsbaum (nur 1en). Daraus folgt die Summe der Viererpotenzen bis zu einer Abbruchshöhe.



Beispiel 76 $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$. Das liegt in $\Theta(n \log\log(n))$, z.B. lösbar über einen Rekursionsbaum.

12.4 Suchen und Sortieren

Löschen im binären Heap für Index i:

- Increase Key auf das Indexelement (größerer Key als Maximum)
- danach Extract Max

Binärer Heap:

- Insert: $O(\log(n))$
- Extract Max: $O(\log(n))$
- Increase Key: $O(\log(n))$
- Find Max: $O(1)$
- Find Min: $\Omega(n)$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

- Search: $\Omega(n)$
- Delete: $O(\log(n))$
- Max Heap to Min Heap: $O(n)$

Beispiel 77 Ein beispielhafter Algorithmus zur Ausgabe der k größten Elemente einer Liste in sortierter Reihenfolge:

- BuildMaxHeap
- Extract Max
- $k-1$ Mal neuer Heap H_K mit Insert für beide Kinder des entfernten Knoten und Extract Max auf H_k
- H_k hat höchstens k Knoten (Höhe $\log(k)$)

$O(n + k\log(k))$ ist eine untere Schranke für diesen Vorgang.

Beispiel 78 Seien paarweise verschiedene Punkte in der Ebene gegeben. Es ist ein y so zu bestimmen, dass für die Gerade $y = m$ die Summe der Abstände aller Punkte minimal ist.

Dabei ist m der Median der Folge $(y_1, ..., y_n)$. Falls n ungerade, ist es die einzig richtige Position. Wenn n gerade ist, sind alle Positionen zwischen den y-Koordinaten der Elemente vom Rang $\frac{n}{2}$ und $\frac{n}{2} + 1$ korrekt.

Korrektheit: Für ungerade n verursacht eine Verschiebung der y-Koordinate vom Median eine Vergrößerung des Abstandes für $\lceil \frac{n}{2} \rceil$ Elemente und eine Verkleinerung für $\lfloor \frac{n}{2} \rfloor$ Elemente um jeweils den gleichen Betrag. Jede andere Position ist verbesserbar, indem man die Leitung dorthin verlegt, wo sich mehr Punkte befinden. Ist n gerade bleibt die Summe der Abstände im Streifen gleich, außerhalb verhält es sich wie oben.

Beispiel 79 Es ist zu zeigen, dass Inorder-Tree-Walk eine sortierte Folge ausgibt.

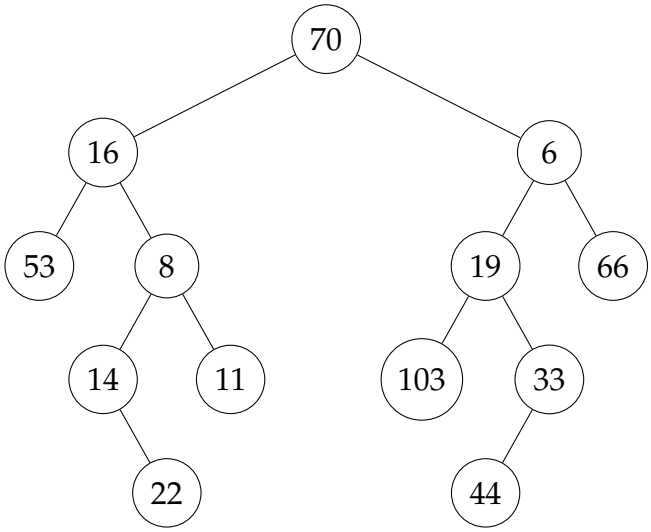
- Induktionsanfang: 1 Knoten, Inorder gibt eine sortierte Folge aus
- Induktionsvoraussetzung: Für $n \leq k$ Knoten wird eine sortierte Folge ausgegeben.
- Induktionsbehauptung: Für $n \leq k + 1$ Knoten wird eine sortierte Folge ausgegeben.

Beweis des Induktionsschrittes: Der Baum mit $k + 1$ Knoten hat zwei Kinder, die jeweils maximal k Knoten enthalten. Diese geben nach Induktionsvoraussetzung sortierte Folgen aus. Nach Suchbaumeigenschaft ist auch das Einfügen der Wurzel mit Inorder sortiert. □

Beispiel 80 Seien Pre- und In-Order-Reihenfolge eines binären Nicht-Suchbaumes gegeben. Es ist gefragt, ob der Baum eindeutig rekonstruierbar ist.

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

Beispiel:

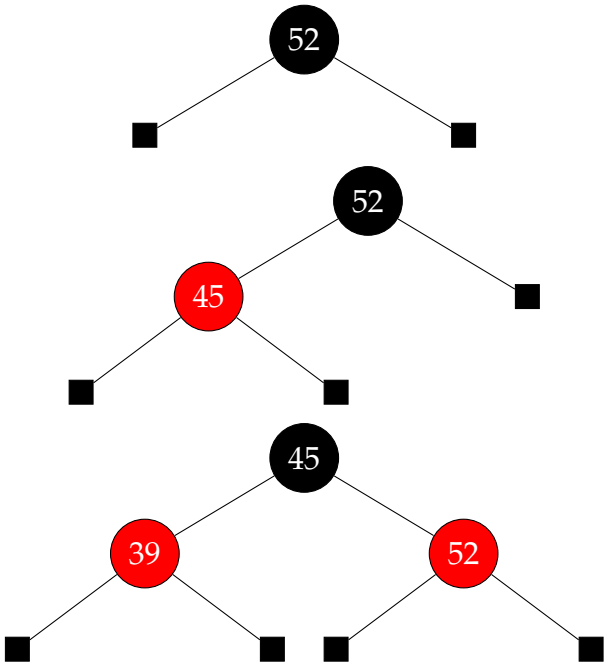


In-Order: 53, 16, 14, 22, 8, 11, 70, 103, 19, 44, 33, 6, 66

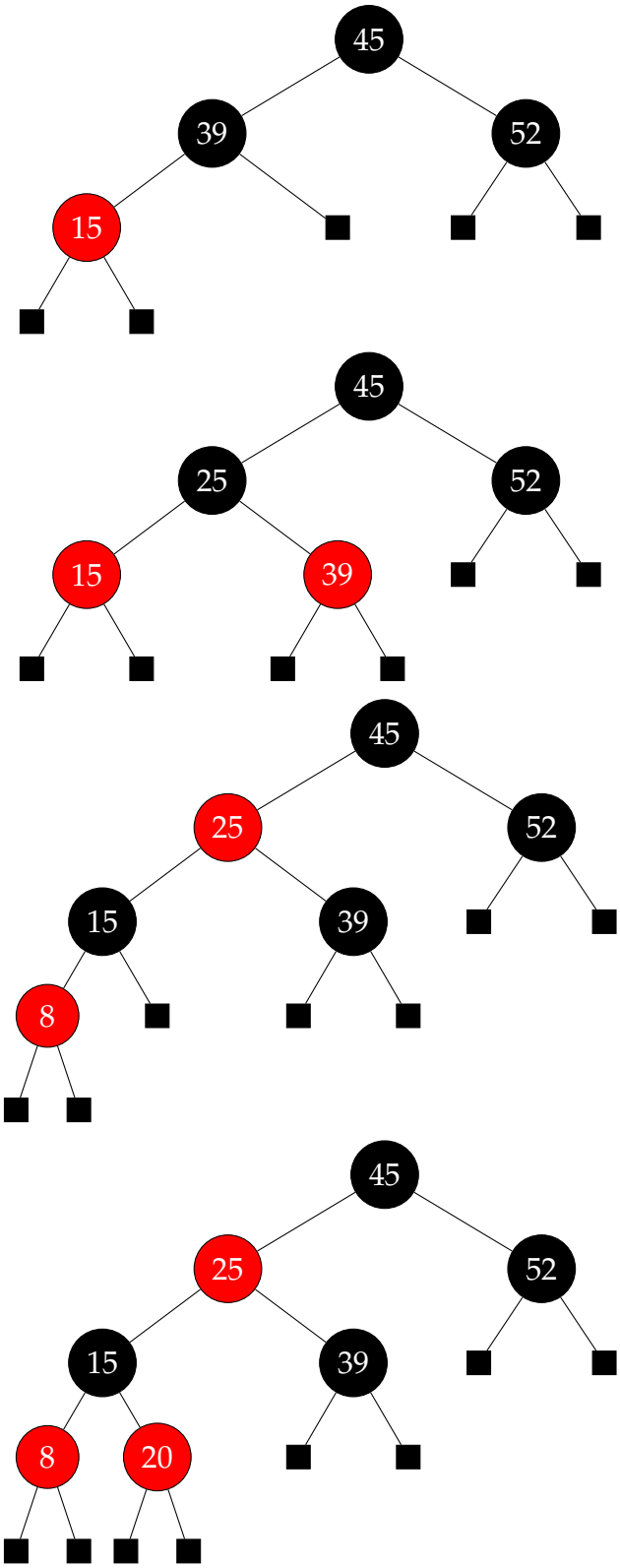
Pre-Order: 70, 16, 53, 8, 14, 22, 11, 6, 19, 103, 33, 44, 66

Antwort: Ja, die Rekonstruktion ist eindeutig möglich.

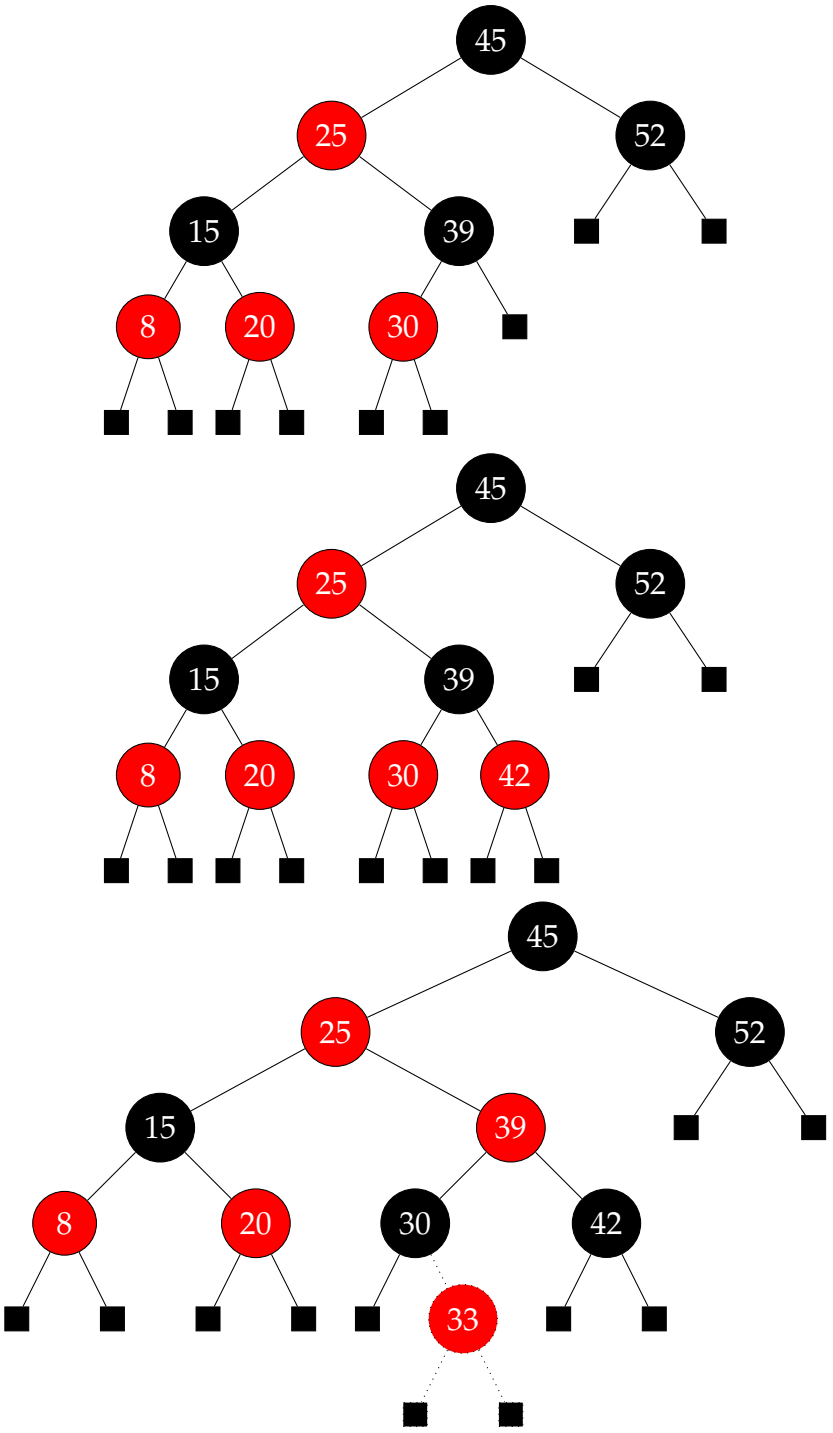
Beispiel 81 Es sind die Schlüssel 52, 45, 39, 15, 25, 8, 20, 30, 42, 33 in einen anfangs leeren Rot-Schwarz-Baum einzufügen.



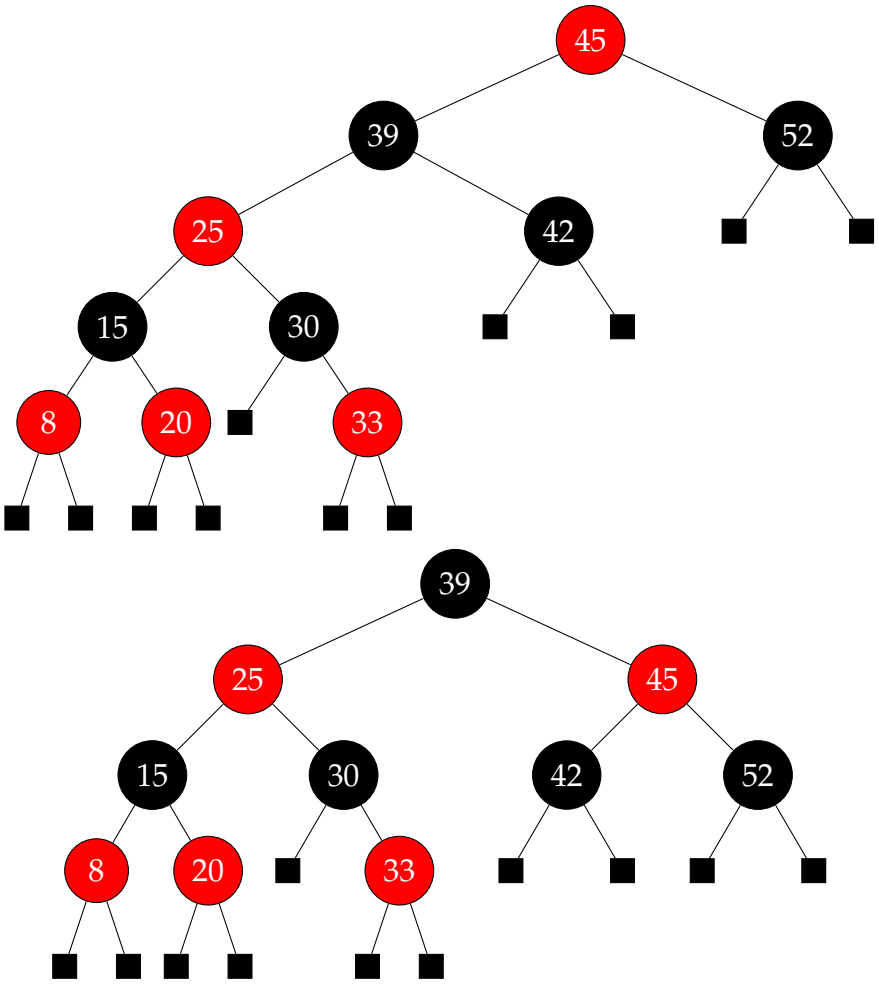
Downloaded by Maja Müller (blackcatgirl@freenet.de)



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Downloaded by Maja Müller (blackcatgirl@freenet.de)



Beispiel 82 Es ist ein Algorithmus RBSelection zur Ausführung von Selection auf einem Rot-Schwarz-Baum in $O(\log(n))$ anzugeben und seine Laufzeit zu analysieren.

Zur Vereinfachung der Aufgabe wird ein Rot-Schwarz-Baum genutzt, der für jeden Knoten x die Anzahl seiner inneren Knoten speichert (wir nennen dies $|x|$, Nil-Knoten haben den Wert 0). Da die Worst-Case-Laufzeit nicht verändert ist, wird maximal das Preprocessing erschwert. Der folgende Algorithmus führt Selection auf einem solchen Rot-Schwarz-Baum T aus und sucht den Schlüssel des Elementes vom Rang k.

```
    algorithm RBSelection(T, k)
1       c = root[T]
2       if k > count(c) + 1 then
3           return nil[T]
4       end if
5       r = count(left[c]) + 1                ▷ Rang der Wurzel von T
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
6           if k = r then
7               return c
8           else if k > r then
9               return RBSelection(right[c], k − r)
10          else
11              return RBSelection(left[c], k)
12          end if
        end algorithm
```

Der Algorithmus terminiert in jedem Fall, da er von der Wurzel bis zu dem Knoten gelangt, welcher den gesuchten Rang hat. Dabei nähert er in jeder Rekursionsstufe um eine Ebene an. Der Worst-Case dieses Algorithmus, der an der Wurzel beginnt, ist, bis zu einem Blatt durchlaufen zu müssen. Damit liegt er in $O(h) = O(\log(n))$.

Beispiel 83

Für $k = 1, 2, 3$ ist die Anzahl der Permutationen der Zahlen $1, ..., 2^k - 1$ so zu berechnen, dass beim Einfügen der durch die Permutationen gegebenen Folge in einen Suchbaum ein vollständiger Binärbaum entsteht. Ferner ist die Rekursiongleichung für den allgemeinen Fall anzugeben.

Sei $p(k)$ die Anzahl der Permutationen. Für $k = 1$ ist $p(1) = 1$. Für $k = 2$ ist $p(2) = 2$ (die Permutationen 213 und 231). Für $k = 3$ ist die Wurzel festgesetzt auf 4. Die anderen 6 Positionen sind variabel. Dabei gibt es zwei Möglichkeiten für den linken Teilbaum und zwei Möglichkeiten für den rechten Teilbaum. Außerdem müssen wir die Position der Elemente des linken Teilbaums (3 Knoten) aus den Restplätzen auswählen, also $\binom{6}{3}$. Daher ist $p(3) = 2 \cdot 2 \cdot \binom{6}{3} = 80$.

Für den allgemeinen Fall ergeben sich $\binom{2^k-2}{2^{k-1}-1}$ Möglichkeiten, welche Positionen von Elementen aus den Schlüsseln des linken Teilbaums belegt sind. Für jede dieser Belegungen gibt es $p(n-1)$ Möglichkeiten für die Reihenfolge der Schlüssel des linken Teilbaums und $p(n-1)$ Möglichkeiten für die Schlüssel des rechten Teilbaumes. Daraus ergibt sich mit $p(1) = 1$

$$p(k) = (p(k-1))^2 \cdot \binom{2^k - 2}{2^{k-1} - 1}.$$

Beispielhaft ergibt sich $p(4) = 80^2 \cdot \binom{14}{7} = 21\,964\,800$.

Beispiel 84

Es ist ein Algorithmus RBJoin gesucht, der die RS-Bäume T_1, T_2 mit $n_1 + n_2 = n$ Knoten vereinigt, sodass T alle Schlüssel aus T_1 und T_2 enthält.

Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
algorithm RBJoin(T_1, T_2)
1     i = InorderTreeWalk(T_1)
2     j = InorderTreeWalk(T_2)
3     ▷ zwei sortierte Folgen, dann in O(n) mischen
4     return RBTreeFromSortedArray(Merge(i, j))
end algorithm
```

Für den Aufbau eines Rot-Schwarz-Baumes aus einem sortierten Feld ist die Zeitkomplexität ebenfalls $O(n)$, da es als rekursives Problem die Rekursionsgleichung $T(n) = 2 \cdot T(\frac{n}{2}) + 1$ besitzt.

Die Vereinigung zweier Rot-Schwarz-Bäume ist nicht in $O(\log(n))$ möglich. Seien $a_1, ..., a_n$ Schlüssel. Wir bauen rekursiv aus $a_1, ..., a_{\lceil \frac{n}{2} \rceil}$ einen RS-Baum T_1 auf und aus $a_{\lceil \frac{n}{2} \rceil + 1}, ..., a_n$ den RS-Baum T_2. Wir vereinigen T_1 und T_2. Falls das in $O(\log(n))$ möglich ist, erhält man in $T(n) = 2 \cdot T(\frac{n}{2}) + c \cdot \log(n)$ ($O(n)$ nach Mastertheorem) einen RS-Baum, der $a_1, ..., a_n$ enthält. Das Auslesen per Inorder-Tree-Walk liefert eine sortierte Folge. Das ist ein Widerspruch zu $\Omega(n \log(n))$ für das Sortieren.

Beispiel 85

Es ist zu beschreiben, wie mithilfe eines Rot-Schwarz-Baumes T, in welchem jeder Knoten die Anzahl innerer Knoten in dem an ihm gewurzelten Teilbaum enthält, die Größe $\text{inv}(K)$ in $O(n \log(n))$ bestimmbar sind.

Idee: Elemente sukzessive in RS-Baum einfügen und zählen, wie viele im Baum befindliche Elemente größer sind als das einzufügende. Dies ist die Anzahl der Inversionen, die das aktuelle Element x mit links von ihm stehenden Elementen bildet. Über alle x summieren liefert die Anzahl der Inversionen.

```
algorithm RBInv(T, x)
1     y = root[T]
2     inv = 0
3     while y ≠ nil[T] do
4         if key[y] > key[x] then
5             inv = inv + 1 + count(right[y])
6             y = left[y]
7         else
8             y = right[y]
9         end if
10    end while
11    return inv
```


This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

end algorithm

12.5 Amortisierte Analyse

Beispiel 86 Es ist eine Potentialfunktion für die amortisierten Kosten von Insert auf einem Bionmial-Heap gesucht und die amortisierten Kosten zu bestimmen.

Φ bildet auf die Anzahl der Bäume in der Wurzelliste ab. $\Phi(D_0) = 0, \Phi(H) \geq 0$. Wir führen weiter ein $(n)_2 = 100\ldots0\underbrace{111\ldots1}_{m}$ mit der Anzahl der Bäume, die verlinkt werden m.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(H') - \Phi(H)\\ &= m + 1 + \Phi(H) - m + 1 - \Phi(H)\\ &= 2 \in O(1)\end{aligned}$$

Beispiel 87 Wir betrachten eine Datenstruktur aus k sortierten Feldern, wobei jedes Feld A_i die Größe 2^i hat mit $0 \leq i \leq k-1$.

Wir entwerfen den folgenden Suchalgorithmus:

```
    algorithm SEARCH(A, x)
1       for i = 0 to k − 1 do
2           if A_i[1] ≠ NIL then                    ▷ prüfe, ob Feld leer
3               index = BINARYSEARCH(A_i, x)
4               if index ≠ NIL then
5                   return index
6               end if
7           end if
8       end for
9       return NIL
    end algorithm
```

Diese Suche hat den Worst-Case, dass alle Felder besetzt sind. Dabei ist $k = \lceil \log(n+1) \rceil$. Wir betrachten $\sum_{i=0}^{k-1} \log(2^i) + 1 = \sum_{i=0}^{k-1} i + 1 = k + \frac{k\cdot(k-1)}{2} \in O(\log^2(n))$.

$T(i)$ sei die Zeit für ein Insert, falls $A[i]$ das erste freie Feld ist. Es gilt $T(0) = 0$. Dann betrachten wir $T(i) = \sum_{j=0}^{i-1} 2^{j+1} = 2 \cdot \sum_{j=0}^{i-1} 2^j = 2 \cdot (2^i - 1) = 2^{i+1} - 2$.

n sei eine Zweierpotenz. Es sollten n Insert-Operationen durchgeführt werden. Ansatz: Wie oft ist $A[i]$ das erste freie Feld. $T(n) = \frac{n}{2} \cdot T(0) +$

Downloaded by Maja Müller (blackcatgirl@freenet.de)

$\frac{n}{4} \cdot T(1) + ... + \frac{n}{2^i} \cdot T(i-1) + ...$ Also $\sum_{i=0}^{\lceil \log(n+1) \rceil} \frac{n}{2^{i+1}} \cdot T(i) = n \cdot \underbrace{\sum_{i=0}^{\lceil \log(n+1) \rceil} \frac{2^{i+1}-2}{2^{i+1}}}_{\text{const}} \in O(n \log(n))$.

Beispiel 88 Seien S_1 und S_2 Stacks. Es ist eine Priority-Queue mit diesen Stapeln zu beschreiben.

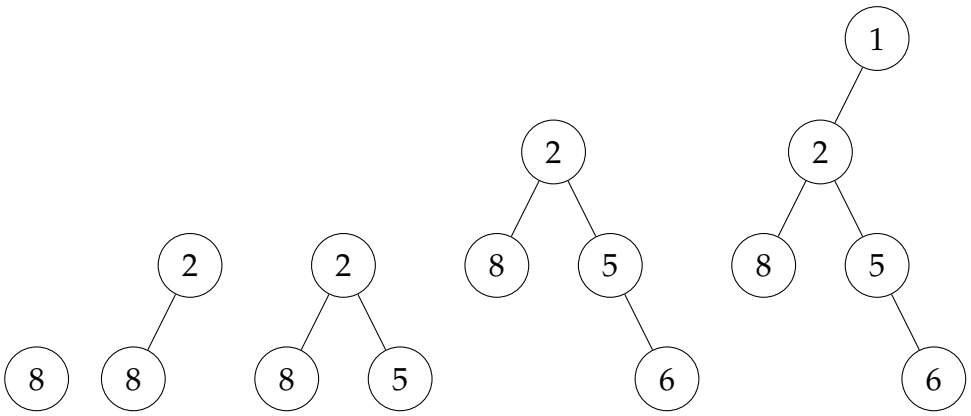
Insert setzen wir gleich mit $\textsc{Push}(S_1, x)$. Damit liegt es in $O(1)$.

```
    algorithm DEQUEUE(Q)
1       if S_2 is empty then
2           while S_1 is not empty do
3               PUSH(S_2, POP(S_1))
4           end while
5       end if
    end algorithm
```

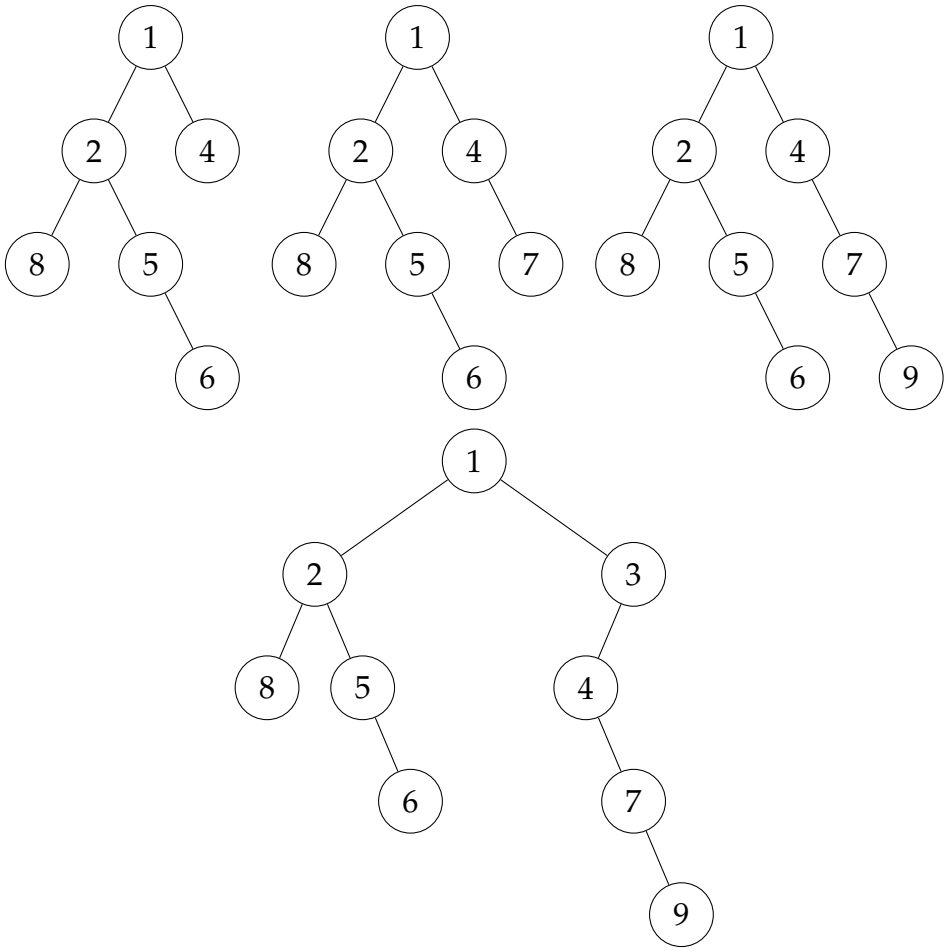
Nun soll eine amortisierte Analyse durchgeführt werden. Wir wählen $\Phi(Q)$ als Abbildung auf die Anzahl der Elemente in S_1.

- Insert: $\hat{c}_i = c_i + \Phi(Q') - \Phi(Q) = 1 + 1 \in O(1)$
- Dequeue:
  - Fall 1: S_2 ist nicht leer: $\hat{c}_i = c_i + \Phi(Q') - \Phi(Q) = 1 + 0 \in O(1)$
  - Fall 2: S_2 ist leer: $\hat{c}_i = c_i + \Phi(Q') - \Phi(Q) = 2 \cdot S_1 + 1 + 0 - 2 \cdot S_1 \in O(1)$

Beispiel 89 Ein kartesischer Baum aus $8, 2, 5, 6, 1, 4, 7, 9, 3$:



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Für die amortisierte Analyse des Anfügens von $A[i]$ wählen wir Φ als die Anzahl der Knoten im rechtesten Pfad und R_i als die Anzahl der Knoten im rechtesten Pfad, deren Schlüssel größer als $A[i]$ ist.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(B') - \Phi(B) \\ &= r_i + c + \Phi(B) - r_i + 1 - \Phi(B) \\ &= c + 1 \in O(1)\end{aligned}$$

Beispiel 90 Für einen binären Suchbaum B bezeichnen wir mit $|B|$ die Zahl der Knoten in B. Für einen Knoten x von B bezeichnen wir mit $T(x)$ den Teilbaum von B mit der Wurzel x, mit $L(x)$ den linken Teilbaum von x und mit $R(x)$ den rechten Teilbaum von x. Es gelte $\frac{1}{2} \leq \alpha < 1$. Ein binärer Suchbaum B heißt α-balanciert, wenn für alle Knoten x von B gilt, dass $|L(x)| \leq \alpha|T(x)|$ und $|R(x)| \leq \alpha|T(x)|$.

Sei $H(n)$ die maximale Höhe eines α-balancierten Baumes mit n Knoten. Es gilt

Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$
\begin{aligned}
H(n) &\leq 1 + H(\alpha n) \\
H(n) &\leq 2 + H(\alpha^2 n) \\
&\vdots \\
H(n) &\leq k + H(\alpha^k n)
\end{aligned}
$$

Gesucht ist das kleinste k mit $\alpha^k \cdot n \leq 1$, also $\alpha^k \leq \frac{1}{n}$. Daraus ergibt sich $\log(\alpha) \leq \log \frac{1}{n}$. bzw. $k \geq -\frac{\log(n)}{\log(\alpha)} \in O(\log(n))$.

Algorithmus für Rebuild in $O(|T(x)|)$ mit $|T(x)| = n$:

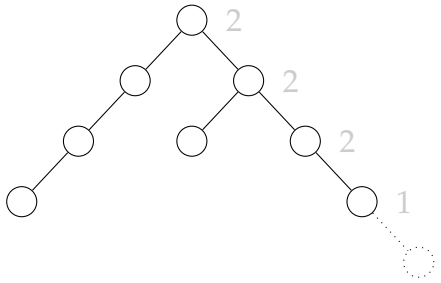
- mit InorderTreeWalk sortiert auslesen in ein Feld der Länge n $(O(n))$
- neuen Baum aufbauen mit Wurzel $A\left[\left\lceil \frac{n}{2} \right\rceil\right]$ (rekursive Anwendung auf linke und rechte Teilliste)
- $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 1 \in O(n)$

Wir setzen $\alpha = \frac{3}{4}$ und

$$
\Phi(B) = 2 \cdot d \cdot \sum_{\substack{x \in B, \\ \Delta x \geq 2}} \Delta x \qquad \text{mit } \Delta x = ||L(x)| - |R(x)||.
$$

Wir betrachten ein Insert ohne Rebuild:

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi(B') - \Phi(B) \\
&= h + 2d \cdot \sum_{\substack{x \in B', \\ \Delta x \geq 2}} \Delta x - 2d \sum_{\substack{x \in B, \\ \Delta x \geq 2}} \Delta x \\
&= h + 2d \cdot \left(\sum_{\substack{x \in B', \\ \Delta x \geq 2}} \Delta x - \sum_{\substack{x \in B, \\ \Delta x \geq 2}} \Delta x \right) \\
&\leq h + 2d \cdot 2h \in O(h) = O(\log(n))
\end{aligned}
$$

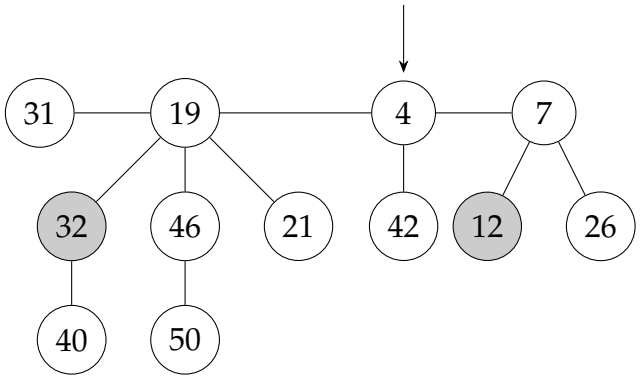


Nun betrachten wir ein Insert mit Rebuild:

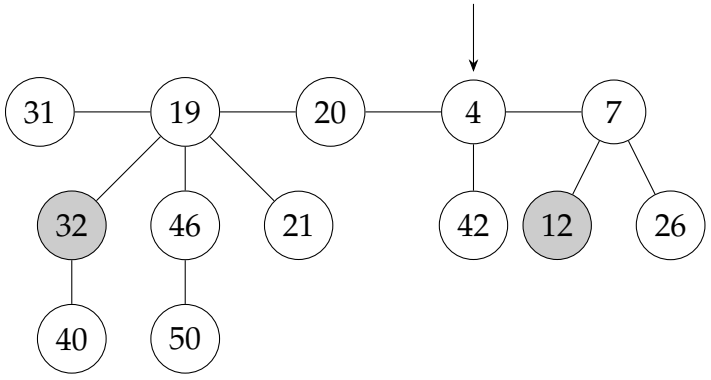
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

$$
\begin{aligned}
\Delta\Phi &= \Phi(B') - \Phi(B) \\
&= \overbrace{\Phi(B) - \Phi(T(x))}^{\Phi(B')} - \Phi(B) \\
&= -\Phi(T(x)) \\
\Phi(T(x)) &= 2d \sum_{\substack{y \in T(x), \\ \Delta y \geq 2}} \Delta y \\
&\geq 2d \cdot \Delta x \\
&= 2d||L(x)| - |R(x)|| \\
&\geq 2d \cdot \left| \frac{3}{4}|T(x)| - \frac{1}{4}|R(x)| \right| \\
&= 2d \cdot \frac{1}{2}|T(x)| = d \cdot |T(x)| \\
\Delta\Phi &= -\Phi(T(x)) \leq -d \cdot |T(x)| \\
\hat{c}_i &= c_i + \Delta\Phi \\
&\leq \overbrace{h + \underbrace{d \cdot |T(x)|}_{\text{Rebuild}}}^{c_i} - d \cdot |T(x)| \\
&= h \in O(\log(n))
\end{aligned}
$$

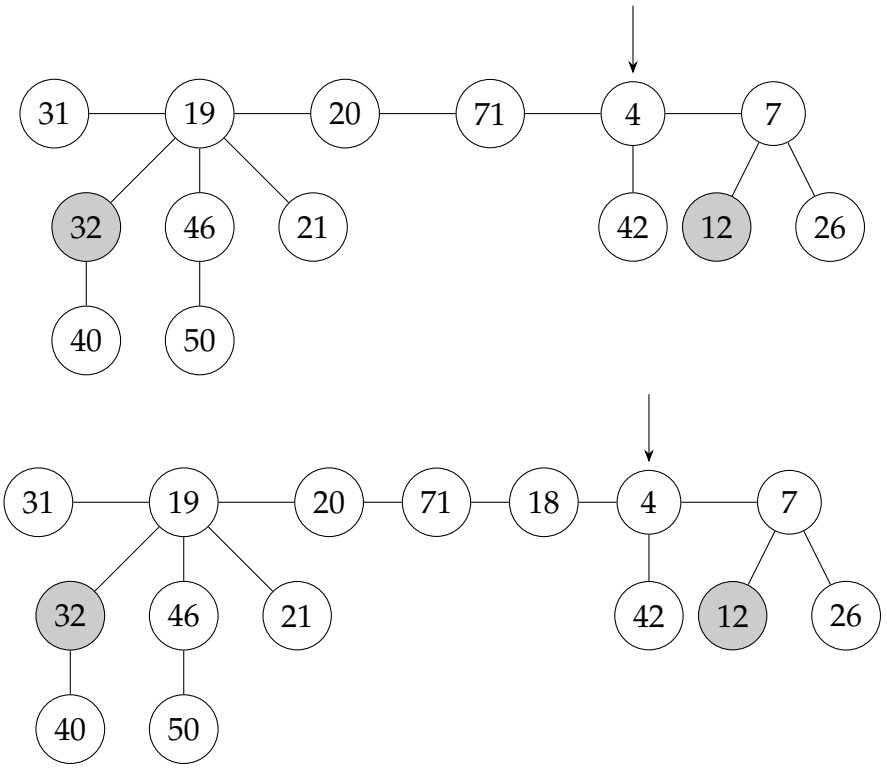
Beispiel 91 Gegeben sei



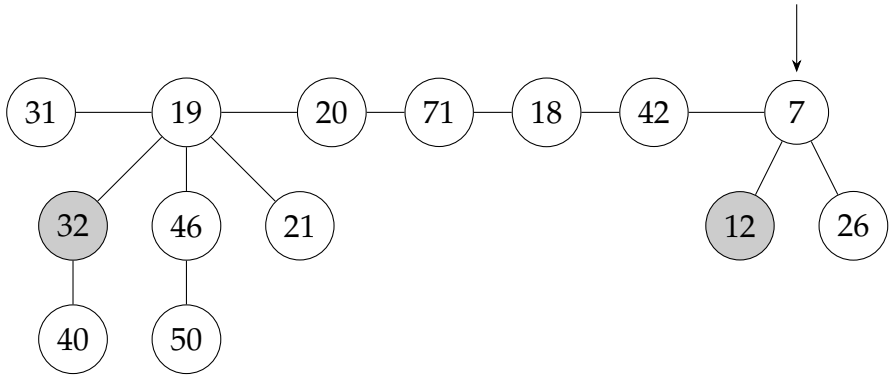
Es sind die Schlüssel $20, 71, 18$ einzufügen.



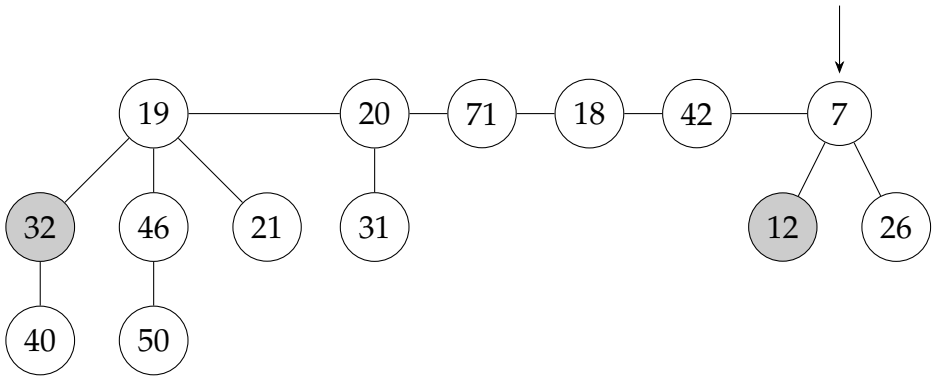
Downloaded by Maja Müller (blackcatgirl@freenet.de)



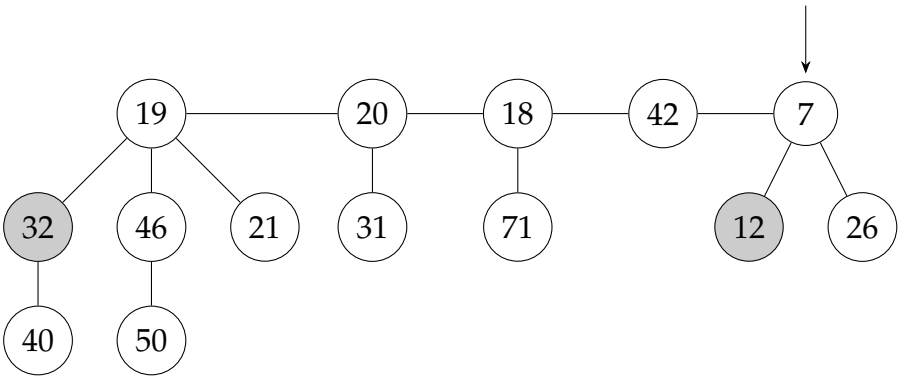
Es ist Extract-Min auszuführen. Dabei wird die 4 als Minimum ausgegeben. Dann:

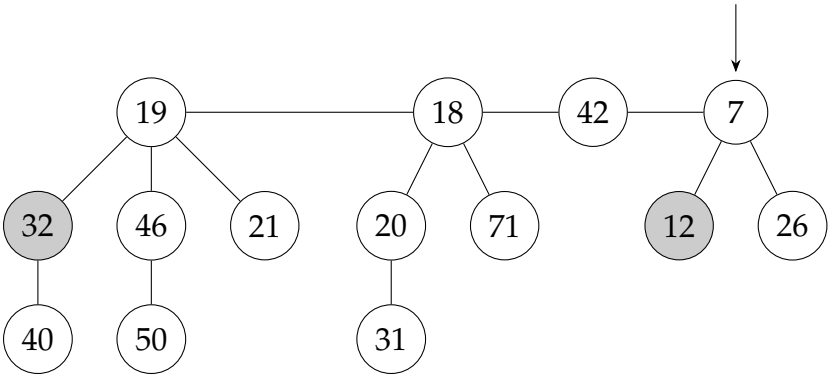


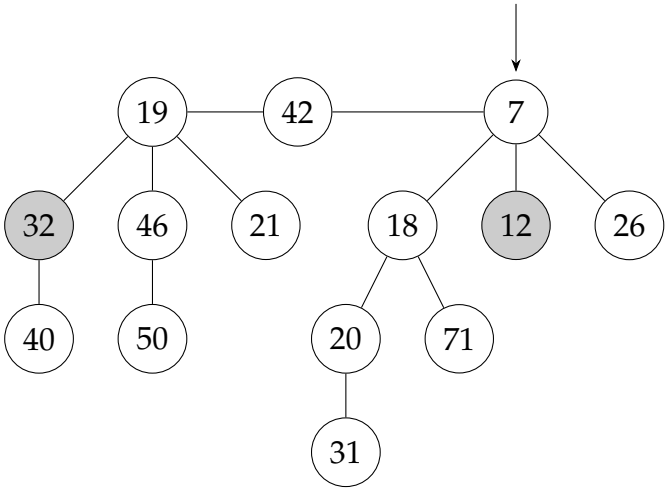
Wir gehen die Wurzelliste vom Minimum aus nach rechts durch.



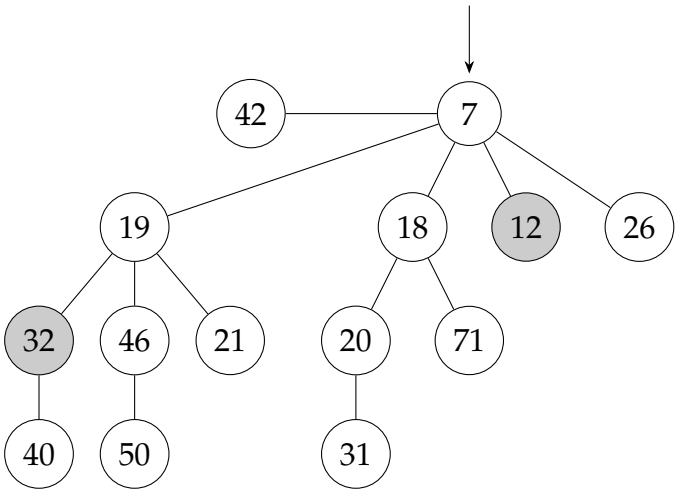
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)





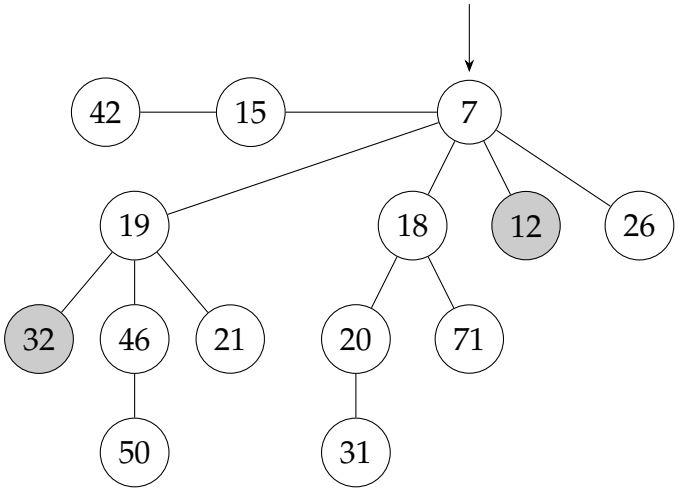


Downloaded by Maja Müller (blackcatgirl@freenet.de)



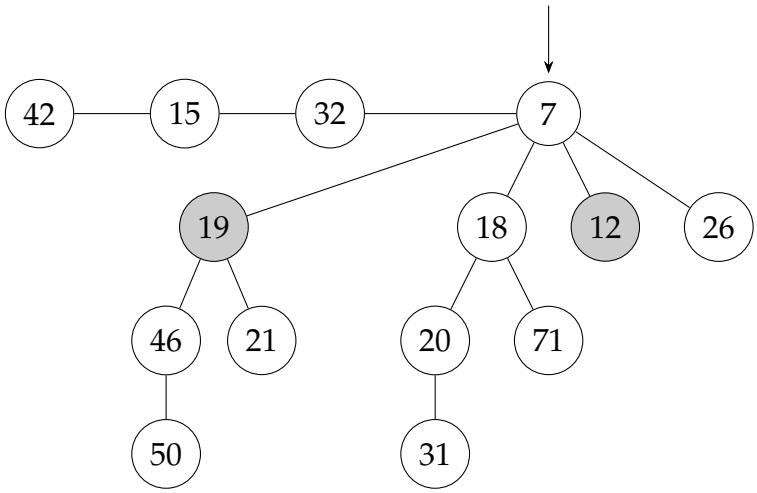
Es ist der Schlüssel 40 auf 15 herabzusetzen und anschließend 21 auf 16.

Zuerst wird 40 auf 15 herabgesetzt und deshalb in die Wurzelliste geschoben.

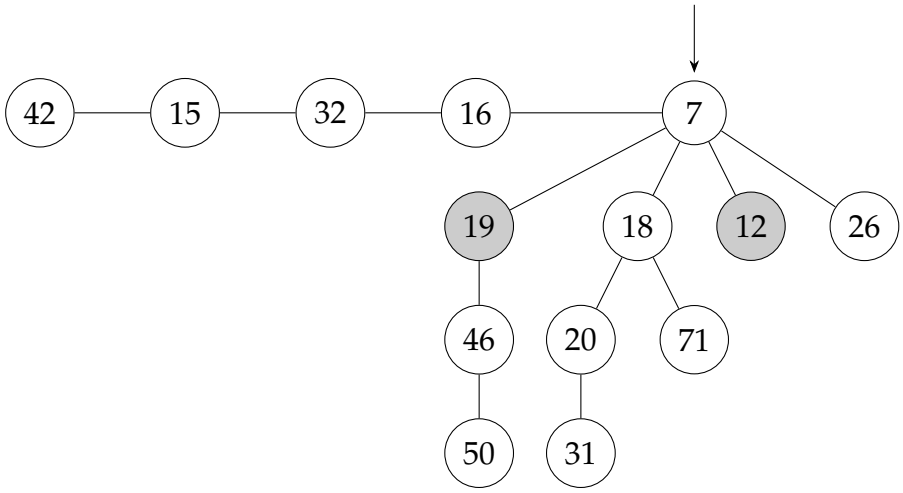


Nun wird die 32 im Cascading-Cut ebenso in die Wurzelliste geschoben.

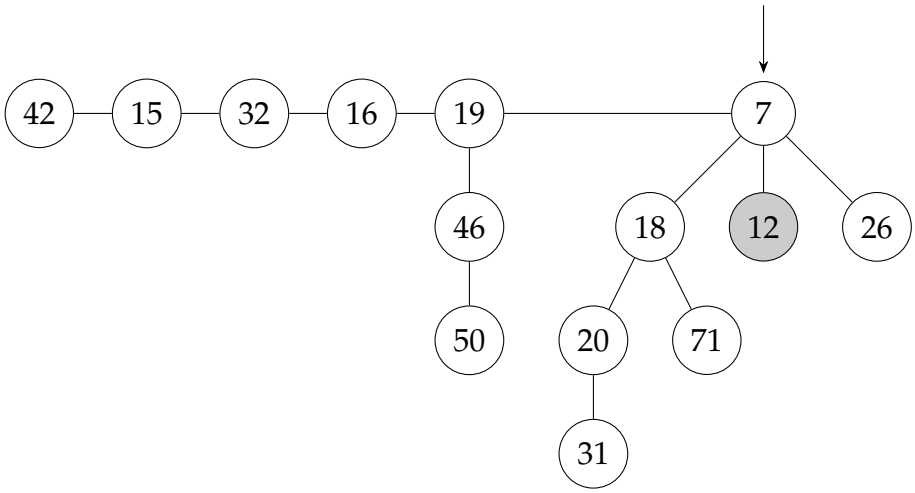
This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



Danach wird 21 auf 16 reduziert.



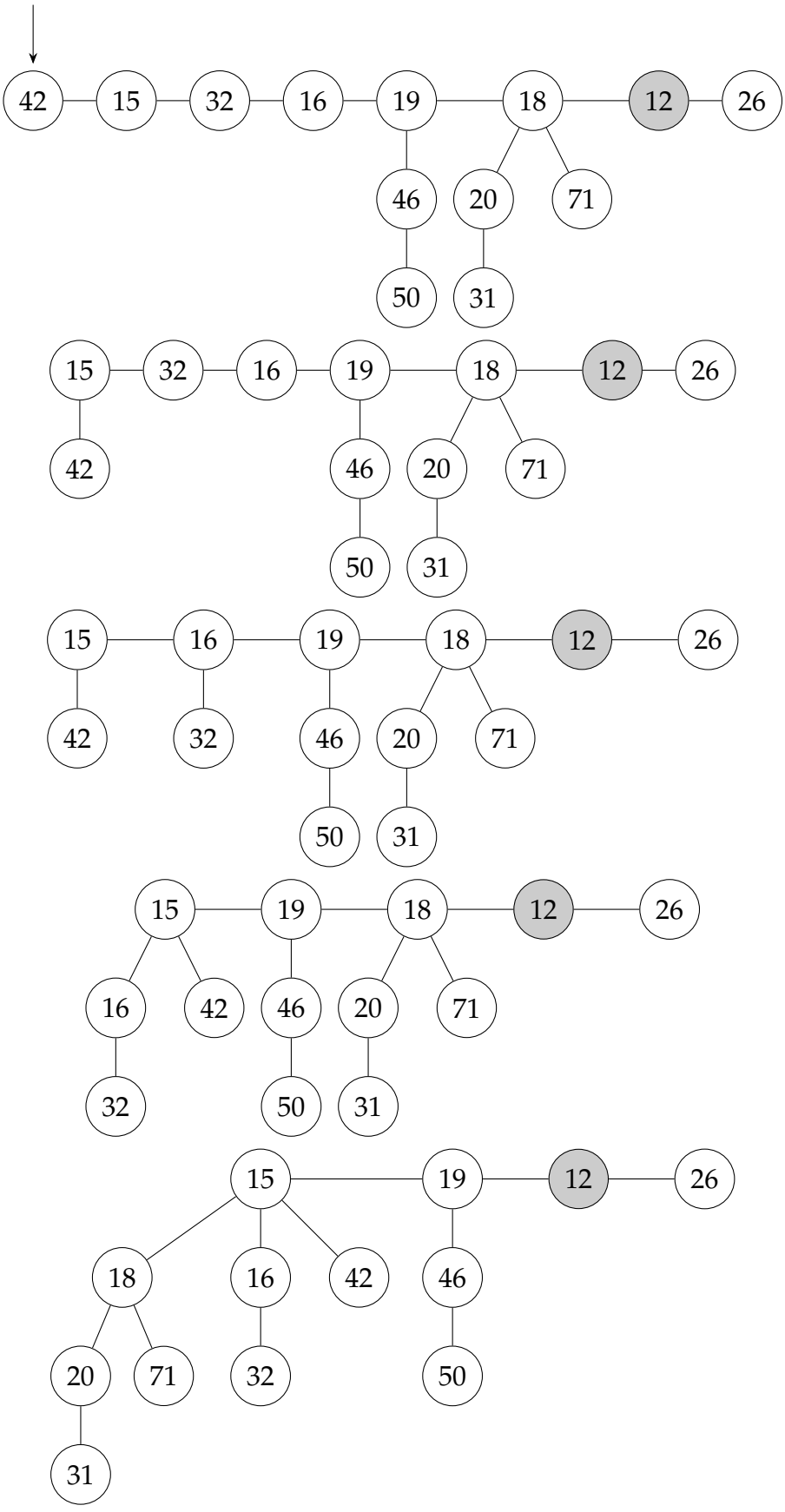
Im Cascading-Cut wird dann die 19 in die Wurzelliste geschoben.



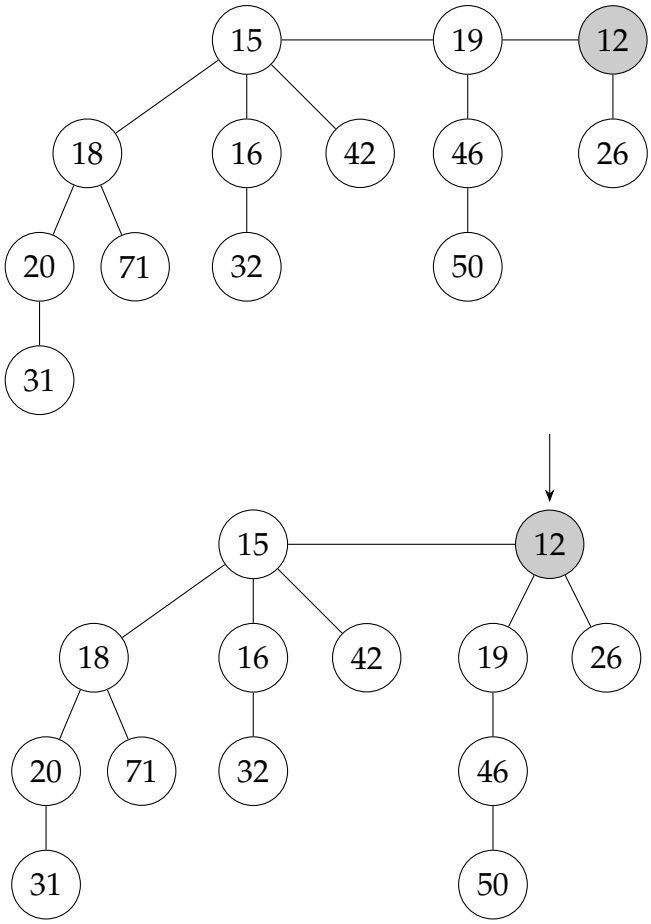
Es ist Extract-Min anzuwenden.

Es wird die 7 extrahiert.

Downloaded by Maja Müller (blackcatgirl@freenet.de)



This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)



12.6 Graphenalgorithmen

Beispiel 92 Es ist ein Algorithmus anzugeben, der in $O(|V|)$ unabhängig von $|E|$ überprüft, ob ein ungerichteter Graph einen Kreis enthält.

CHECKCYCLES(G):
01 FOR EACH Knoten $v \in V[G]$: // Initialisierung der DFS
02 color[v] = weiß
03 π[v] = NIL
04 FOR EACH Knoten $v \in V[G]$:
05 IF color[v] = weiß:
06 CYCLE-VISIT(v)
07 RETURN kein Kreis
CYCLE-VISIT(v):
01 IF color[v] = grau:
02 RETURN Kreis gefunden
03 color[v] = grau
04 FOR EACH $u \in AL[v]$:

Downloaded by Maja Müller (blackcatgirl@freenet.de)

```
05  π[u] = v
06  CYCLE-VISIT(u)
07 color[v] = schwarz
```

Der Algorithmus basiert auf einer Tiefensuche und durchläuft alle Knoten. Stößt er dabei mit einer freien Kante auf einen bereits besuchten Knoten ist ein Kreis gefunden. Damit ist der Algorithmus auch in $O(|V|)$ und unabhängig von $|E|$, da man im Worst Case $|V|-1$ Kanten betrachten muss (aufspannender Baum), bevor ein Kreis gefunden werden muss oder man ohne noch zu bearbeitende Kanten dasteht.

Beispiel 93

Sei G ein ungerichteter Graph mit $V = A \cup B \cup C$ und es existieren nur Kanten zwischen A, B und B, C. Es ist ein Algorithmus zu bestimmen, der den Graph G' mit Knotenmenge $A \cup C$ und denjenigen Kanten zwischen $u \in A, v \in C$ bestimmt, für die ein u-v-Weg in G existiert.

Es wird davon ausgegangen, dass A, B, C disjunkt sind.

```
GETACGRAPH(G)
01 G' = (∅, ∅)
02 FOR EACH Knoten v ∈ V[G]:
03   color[v] = white
04   π[v] = NIL
05   IF v ∈ A ∪ C: V(G') += v
06 FOR EACH Knoten v ∈ V[G]:
07   IF color[v] = white:
08     anodes = []
09     cnodes = []
10     ACGRAPHVISIT(anodes, cnodes, v)
11     ADDEDGES(E(G'), anodes, cnodes)
12 RETURN G'
ACGRAPHVISIT(anodes, cnodes, v)
01 IF v in A: anodes += v
02 ELSE IF v in C: cnodes += v
03 color[v] = gray
04 FOR EACH u ∈ AL[v]
05   IF color[u] = white:
06     π[u] = v
07     ACGRAPHVISIT(anodes, cnodes, u)
08 color[v] := black
ADDEDGES(E, X, Y)
01 FOR EACH e = {x, y} ∈ X × Y:
02   E += e
```


This document is available on studocu

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Der Algorithmus baut auf der Tiefensuche auf, um auch Folgen wie $A \to B \to A \to B \to C$ abfassen zu können. Die Laufzeit der Tiefensuche bleibt unverändert, da die Speicheroperationen konstante Zeit benötigen. Allerdings wird durch die Verkettung der Knoten zu Kanten zusätzlich ein Summand von $|A| \cdot |C|$ angefügt. Das ergibt insgesamt $O(|V| + |E| + |A| \cdot |C|)$.

Beispiel 94

Sei $T = (V, E)$ ein Baum. Es ist sein Durchmesser gesucht. Der Durchmesser ist dabei die Länge des längsten Weges im Baum.

Algorithmus:

1. Führe Breitensuche von einem beliebigen Startknoten aus.
2. Wähle den Knoten mit der größten Breitensuchnummer. Dies ist ein Endpunkt des längsten Weges.
3. Von dort aus führen wir wiederum eine Breitensuche aus. Der Knoten mit der größten Breitensuchnummer ist der zweite Endknoten des längsten Weges.

Alternativer Algorithmus:

1. Wir zeichnen einen Knoten als Wurzel aus.
2. Bestimme rekursiv die Länge des längsten Weges in jedem Kind sowie die Höhe des am Kind gewurzelten Teilbaumes.
3. Bilde das Maximum aus den längsten Wegen in den Teilbäumen sowie den paarweisen Summen der Höhen plus 1 (über Wurzel).

Downloaded by Maja Müller (blackcatgirl@freenet.de)

Index

a
Adjazenzliste *97*
Adjazenzmatrix *97*
abstrakter Datentyp *16*

b
Baumkanten *108*
BFS *99*, *103*
BinaryIncrement *76*
BinarySearch *143*
Binomialbäume *71*
BinomialHeapDecreaseKey *74*, *75*
BinomialHeapDelete *75*
BinomialHeapExtractMin *74*, *75*
BinomialHeapInsert *73*
BinomialHeapUnion *72*, *73*, *74*
Binomialheap *71*
BinomialLink *72*
BuildMaxHeap *28*, *30*
binärer Suchbaum *40*

c
CascadingCut *87*, *88*, *90*
ChainedHashDelete *65*
ChainedHashInsert *64*
ChainedHashSearch *64*, *65*
Countingsort *37*, *38*, *39*
Cut *87*, *88*

d
Datenstruktur *16*
Dequeue *99*, *144*
DFS *104*, *105*, *107*, *109*, *110*, *111*
DFSVisit *105*, *106*
Dictionary *17*
Disjoint-Set-Forest *92*
DSFLink *94*, *95*
DSFUnion *93*, *94*, *95*, *96*

e
Enqueue *99*
ExtractMin *120*

f
FHeapConsolidate *82*, *84*, *86*
FHeapDecreaseKey *87*, *88*, *89*
FHeapExtractMin *82*
FHeapInsert *80*
FHeapLink *83*, *84*
FHeapMinimum *80*
FHeapUnion *81*
FindSet *93*, *94*, *96*, *116*

g
GenericMST *113*, *115*, *116*
gemeinsames Teilwort *121*
geordnet *70*
gerichteter Graph *96*

h
HashDelete *67*
HashInsert *67*, *68*
HashSearch *67*
Heap *23*
Heapsort *23*, *30*

i
InorderTreeWalk *41*, *42*, *142*, *146*
InsertionSort *8*, *36*
IterativeTreeSearch *43*

k
Kruskal *3*, *111*, *115*
korrekt *8*

This document is available on studocu
Downloaded by Maja Müller (blackcatgirl@freenet.de)

l
LCSLength *123*
LeftRotate *51*, *53*, *60*
leichte Kante *114*

m
MakeBinomialHeap *73*, *74*
MakeFHeap *80*, *81*
MakeSet *93*, *94*, *96*, *116*
MaxHeapExtractMax *24*, *27*
MaxHeapIncreaseKey *26*
MaxHeapInsert *24*, *25*, *26*
MaxHeapify *24*, *25*, *27*, *28*, *29*, *30*
Median *34*
Merge *22*, *23*, *142*
Mergesort *21*, *23*, *30*
MSTKruskal *115*
MSTPrim *119*
minimaler Spannbaum *111*

o
OrthoSegmentSchnitt *16*

p
Partition *31*
Pop *144*
PostorderTreeWalk *41*
PreorderTreeWalk *41*
Prim *3*, *116*
PrintLCS *124*
Priority Queue *23*
Push *144*

q
Querkanten *108*
Quicksort *30*, *31*, *33*

r
Radixsort *39*
RandomizedQuicksort *33*
Rang *33*
RBDelete *58*, *63*
RBDeleteFixup *58*, *59*, *60*, *63*
RBInsert *52*, *56*, *57*, *63*
RBInsertFixup *52*, *56*, *57*
RBInv *142*
RBJoin *141*, *142*
RBSelection *140*, *141*
RBTreeFromSortedArray *142*
RightRotate *51*, *53*, *60*
Rot-Schwarz-Baum *48*
Rückwärtskanten *108*

s
Schnitt *114*
Schwarzhöhe *49*
Search *143*
Selection *34*
Spannbaum *111*
StackEmpty *75*
StackMultiPop *75*
StackPop *75*, *130*, *131*
StackPush *130*
stabil *38*
swap *25*, *26*, *30*, *31*, *74*, *84*

t
Teilwort *121*
TopologicalSort *109*, *111*
TreeDelete *47*, *58*
TreeInsert *45*, *46*, *52*
TreeMaximum *44*
TreeMinimum *43*, *44*, *45*
TreeSearch *43*
TreeSuccessor *45*, *47*

u
Union *116*
universell *66*

Downloaded by Maja Müller (blackcatgirl@freenet.de)

v
Vorwärtskanten *108*

w
wurzelgerichteter Baum *92*

This document is available on studocu

Downloaded by Maja Müller (blackcatgirl@freenet.de)